



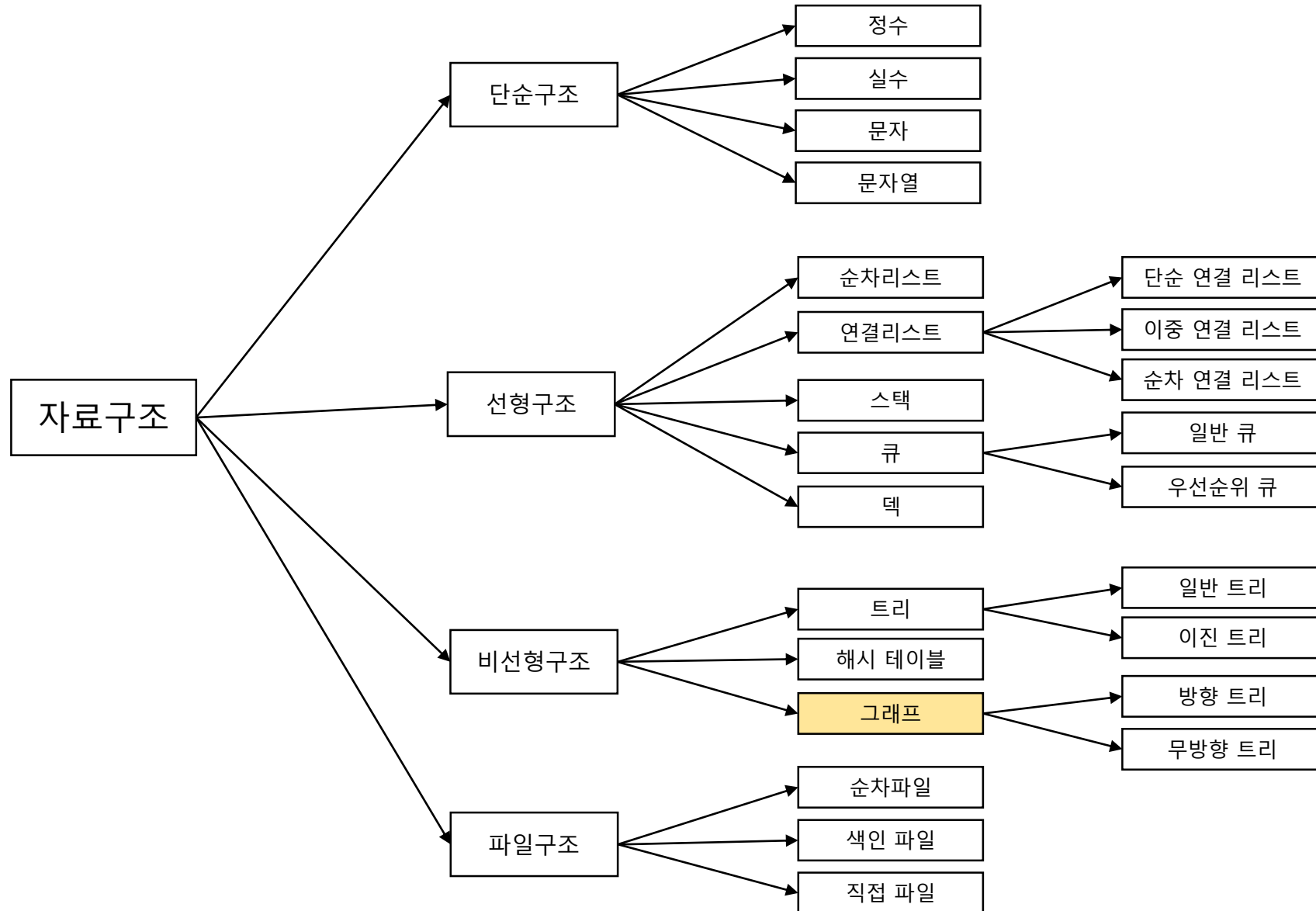
국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론

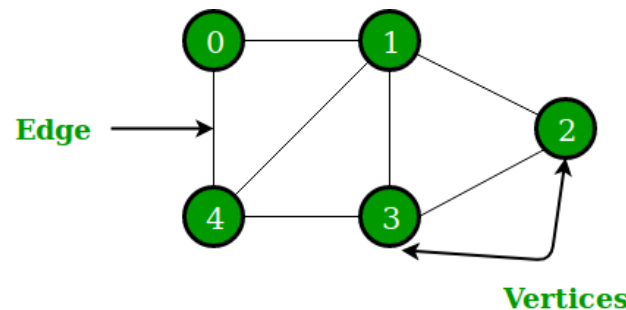


Data structure



Graph

- vertex와 edge로 구성된 한정된 자료구조를 의미한다. vertex는 정점, edge는 정점과 정점을 연결하는 간선이다.
- 단순히 노드(N, node)와 그 노드를 연결하는 간선(E, edge)을 하나로 모아 놓은 자료 구조
- 연결되어 있는 원소간의 관계를 표현한 자료구조.
- 연결할 객체를 나타내는 정점(Vertex)과 객체(정점)를 연결하는 간선(Edge)의 집합으로 구성.
- 그래프는 여러 점들이 서로 복잡하게 연결되어 있는 관계를 표현한 자료구조.
- 정확히는 정점(Vertex)간의 관계를 표현한 조직도이다.
- 직접적인 관계가 있다면 두 정점 사이를 이어주는 간선(edge)이 있고, 간접적인 관계가 있다면 몇 개의 점과 선에 걸쳐서 이어진다.



Graph

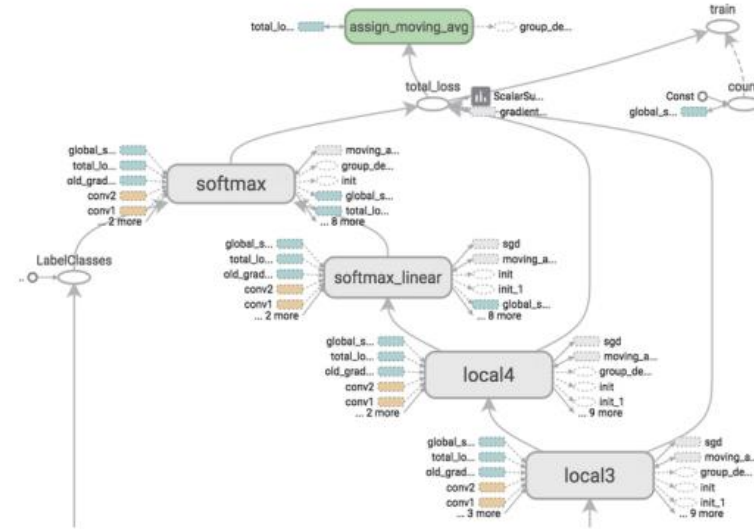
- 연결되어 있는 객체 간의 관계를 표현하는 자료구조
- 가장 일반적인 자료구조 형태
- 선형 자료구조들이나 트리도 그래프로 표현할 수 있음



지하철 노선도



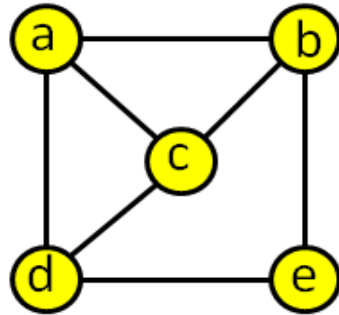
항공 노선도



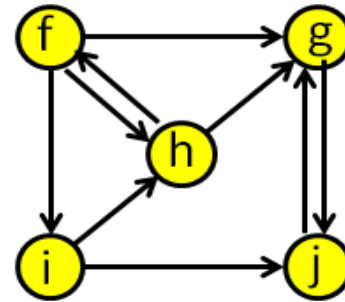
텐서플로우 그래프

Graph 용어

- 그래프는 정점(Vertex)과 간선(Edge)의 집합으로 하나의 간선은 두 개의 정점을 연결
- 그래프는 $G=(V, E)$ 로 표현, V =정점의 집합, E =간선의 집합
- 방향 그래프(Directed Graph): 간선에 방향이 있는 그래프
- 무방향 그래프(Undirected Graph): 간선에 방향이 없는 그래프
- 정점 a 와 b 를 연결하는 간선을 (a, b) 로 표현
- 정점 a 에서 b 로 간선의 방향이 있는 경우 $\langle a, b \rangle$ 로 표현



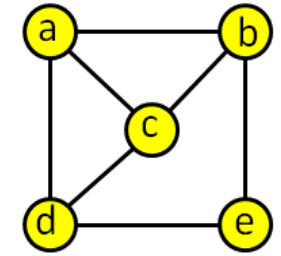
(a) 무방향그래프



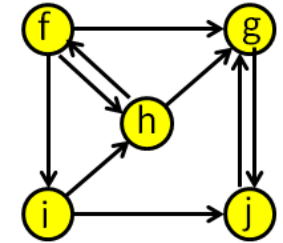
(b) 방향 그래프

Graph 용어

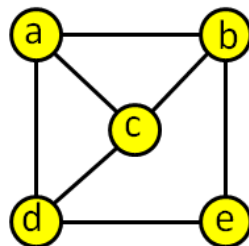
- 차수(Degree): 정점에 인접한 정점의 수
- 방향 그래프에서는 차수를 진입 차수(In-degree)와 진출 차수(Out-degree)로 구분
- 그림(a) 정점 a의 차수 = 3, 정점 e의 차수 = 2.
- 그림(b) 정점 g의 진입 차수 = 3, 진출 차수 = 1.
- 경로(Path)는 시작 정점 u부터 도착점 v까지의 정점들을 나열하여 표현
 - ✓ [a, c, b, e]: 정점 a로부터 도착점 e까지의 여러 경로들 중 하나
- 단순 경로(Simple Path): 경로 상의 정점들이 모두 다른 경로
- '일반적인' 경로: 동일한 정점을 중복하여 방문하는 경우를 포함
 - ✓ [a, b, c, b, e]: 정점 a로부터 도착점 e까지의 경로
- 사이클(Cycle): 시작 정점과 도착점이 동일한 단순 경로
 - ✓ [a, b, e, d, c, a]



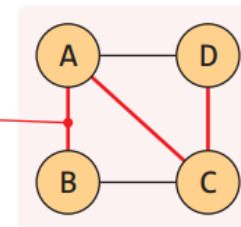
(a) 무방향그래프



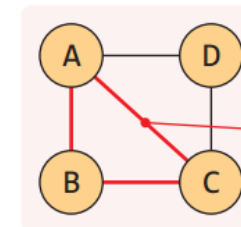
(b) 방향 그래프



경로 B, A, C, D는 단순 경로

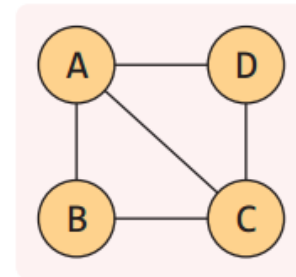
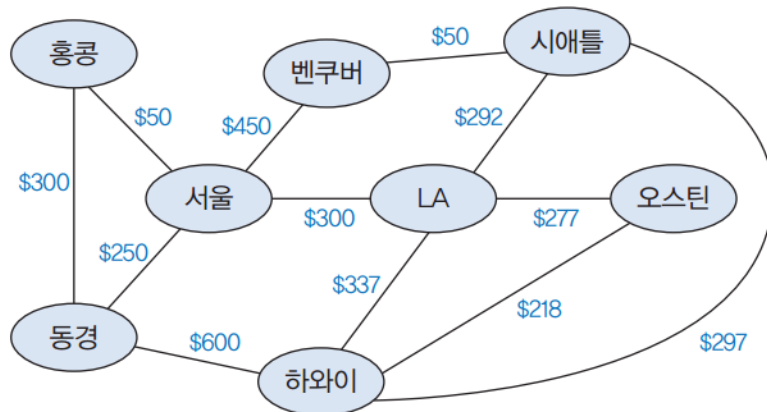


경로 B, A, C, B는 사이클

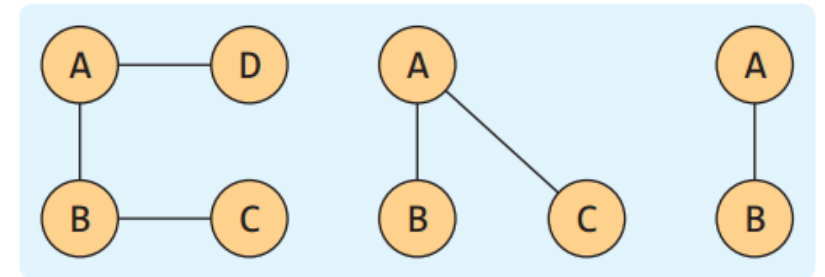


Graph 용어

- 가중치(Weighted) 그래프: 간선에 가중치가 부여된 그래프
 - ✓ 가중치는 두 정점 사이의 거리, 지나는 시간이 될 수도 있다. 또한 음수인 경우도 존재
- 부분그래프(Subgraph): 주어진 그래프의 정점과 간선의 일부분(집합)으로 이루어진 그래프
 - ✓ 부분그래프는 원래의 그래프에 없는 정점이나 간선을 포함하지 않음
- 트리(Tree): 사이클이 없는 그래프
- 신장트리(Spanning Tree): 주어진 그래프가 하나의 연결성분으로 구성되어 있을 때, 그래프의 모든 정점들을 사이클 없이 연결하는 부분그래프



G1



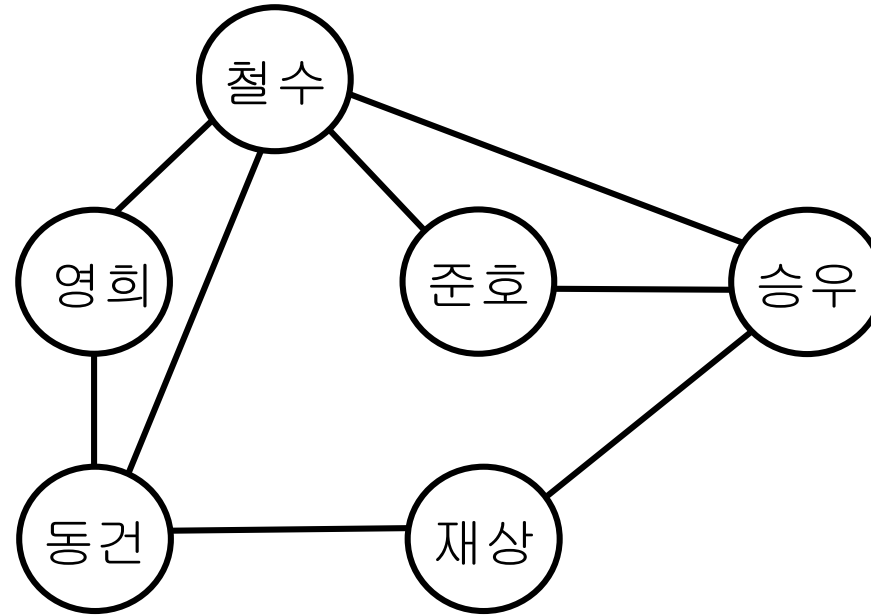
G1의 부분 그래프들

Graph 용어

- 경로의 길이(length): 경로를 구성하는데 사용된 간선의 수
- 가중치(Weighted) 그래프: 간선에 가중치가 부여된 그래프
 - ✓ 가중치는 두 정점 사이의 거리, 지나는 시간이 될 수도 있다. 또한 음수인 경우도 존재
- 부분그래프(Subgraph): 주어진 그래프의 정점과 간선의 일부분(집합)으로 이루어진 그래프
 - ✓ 부분그래프는 원래의 그래프에 없는 정점이나 간선을 포함하지 않음
- 트리(Tree): 사이클이 없는 그래프
- 신장트리(Spanning Tree): 주어진 그래프가 하나의 연결성분으로 구성되어 있을 때, 그래프의 모든 정점들을 사이클 없이 연결하는 부분그래프

Graph 종류

- 기본적인 그래프
- 무향 그래프라고도 한다

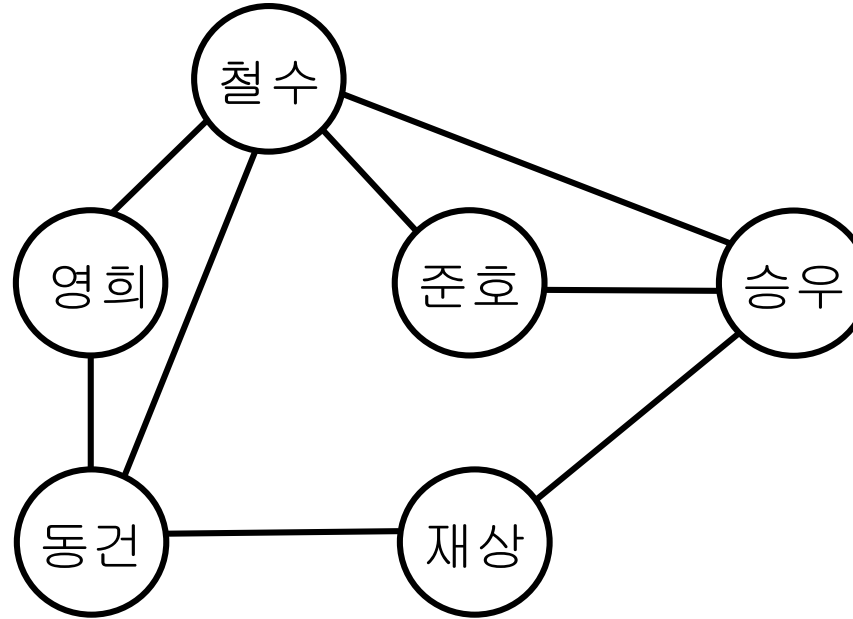


(a) 친밀도의 존재 여부 표시

사람들 간의 친분 관계를 나타낸 그래프

Graph 종류

- 가중 그래프(Weighted Graph)

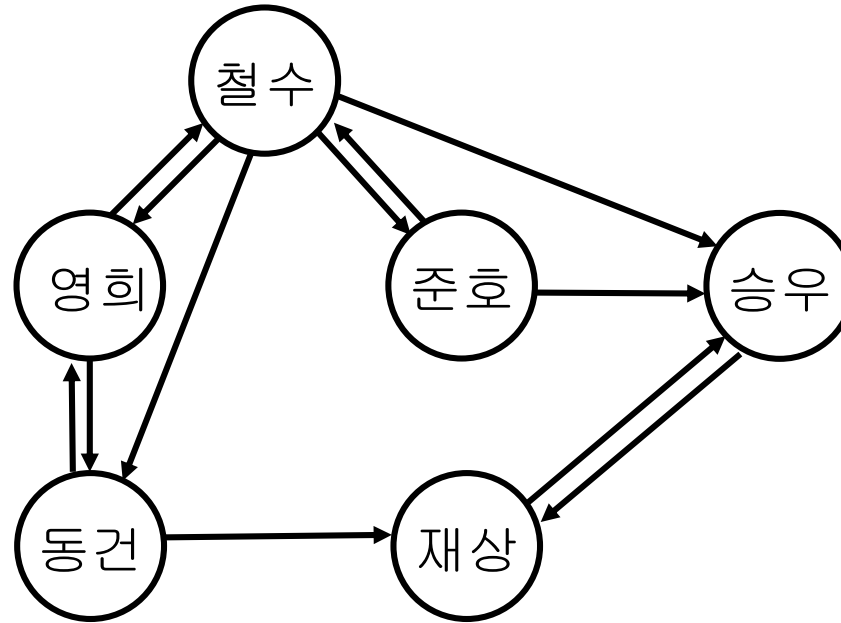


(b) 친밀도의 크기까지 표시

사람들 간의 친분 관계를 나타낸 그래프

Graph 종류

- 방향 그래프(유향 그래프)(Directed Graph)

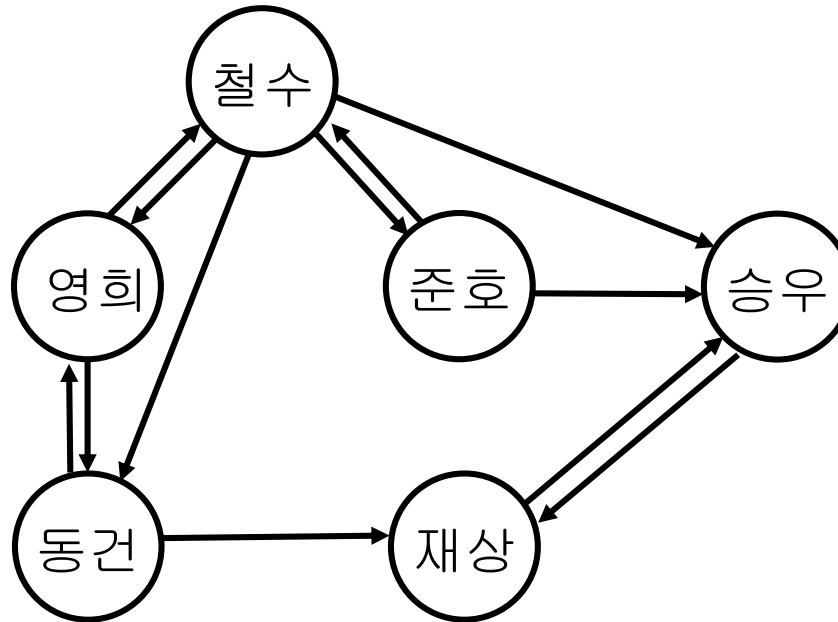


(a) 호감 여부 표시

방향을 고려한 친분 관계 그래프

Graph 종류

- 가중 방향 그래프(Weighted Directed Graph)



(b) 호감의 크기까지 표시

방향을 고려한 친분 관계 그래프

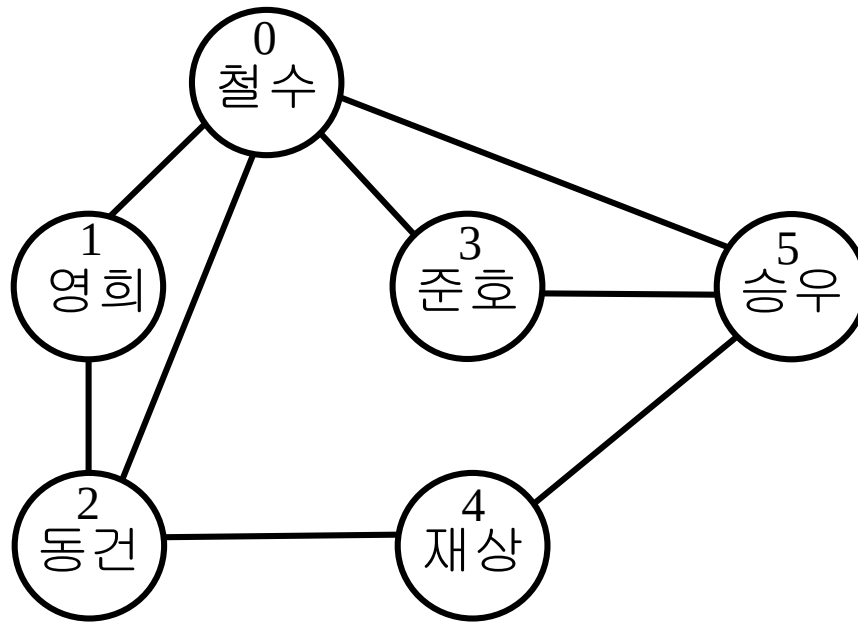
Graph의 표현

- 인접 행렬Adjacency Matrix

- 그래프가 $n \times n$ 행렬 $A[i][j]$ 로 표현된다.
- 정점 i 와 정점 j 사이에 간선의 존재 여부를 $A[i][j]$ 에 기록한다

Graph의 표현

- 무방향 그래프의 인접 행렬



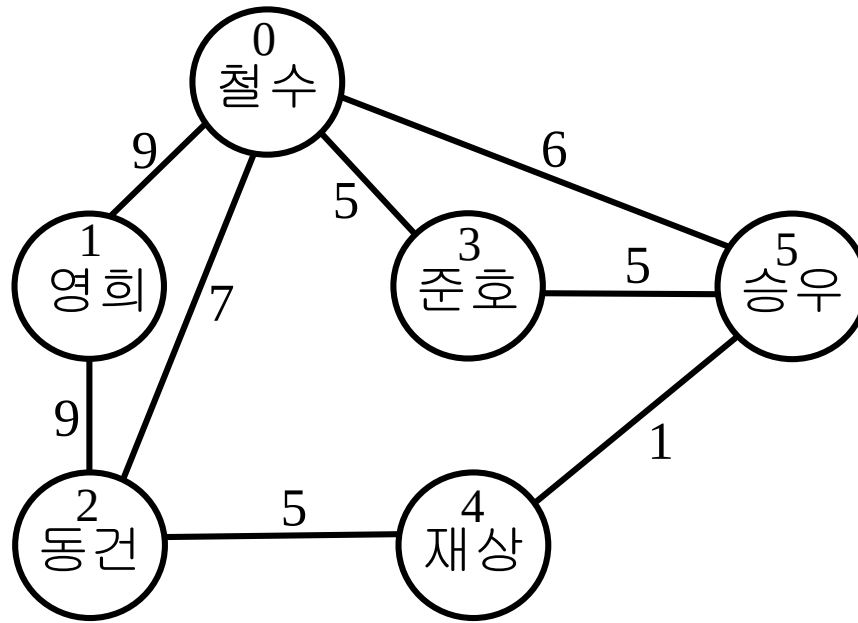
(a) 가중치가 없는 경우

	0	1	2	3	4	5
0	0	1	1	1	0	1
1	1	0	1	0	0	0
2	1	1	0	0	1	0
3	1	0	0	0	0	1
4	0	0	1	0	0	1
5	1	0	0	1	1	0

무방향 그래프와 인접 행렬 표현

Graph의 표현

- 가중 무방향 그래프의 인접 행렬



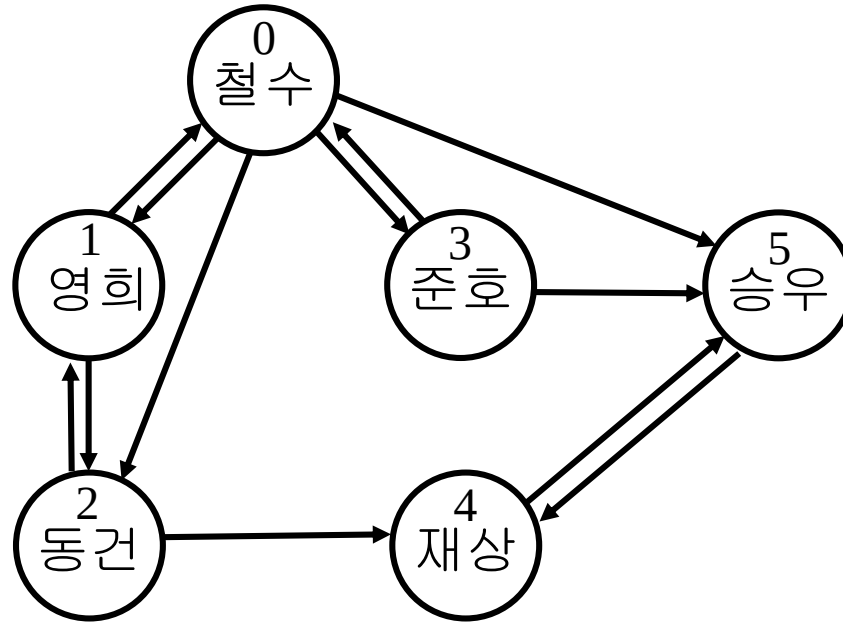
(b) 가중치가 있는 경우

	0	1	2	3	4	5
0	0	9	7	5	0	6
1	9	0	9	0	0	0
2	7	9	0	0	5	0
3	5	0	0	0	0	5
4	0	0	5	0	0	1
5	6	0	0	5	1	0

무방향 그래프와 인접 행렬 표현

Graph의 표현

- 방향 그래프의 인접 행렬



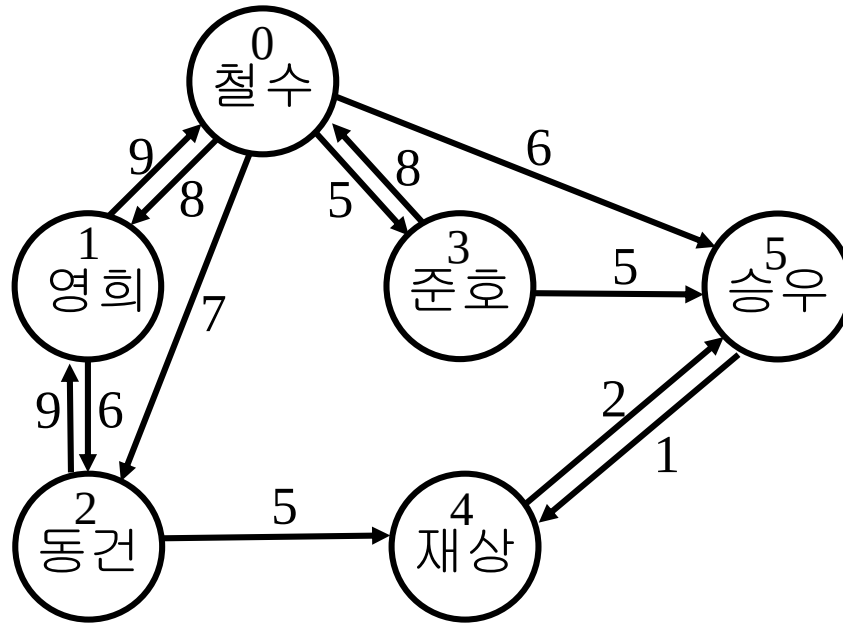
(a) 가중치가 없는 경우

	0	1	2	3	4	5
0	0	1	1	1	0	1
1	1	0	1	0	0	0
2	0	1	0	0	1	0
3	1	0	0	0	0	1
4	0	0	0	0	0	1
5	0	0	0	0	1	0

방향 그래프와 인접 행렬 표현

Graph의 표현

- 가중 방향 그래프의 인접 행렬



(b) 가중치가 있는 경우

	0	1	2	3	4	5
0	0	8	7	5	0	6
1	9	0	6	0	0	0
2	0	9	0	0	5	0
3	8	0	0	0	0	5
4	0	0	0	0	0	2
5	0	0	0	0	1	0

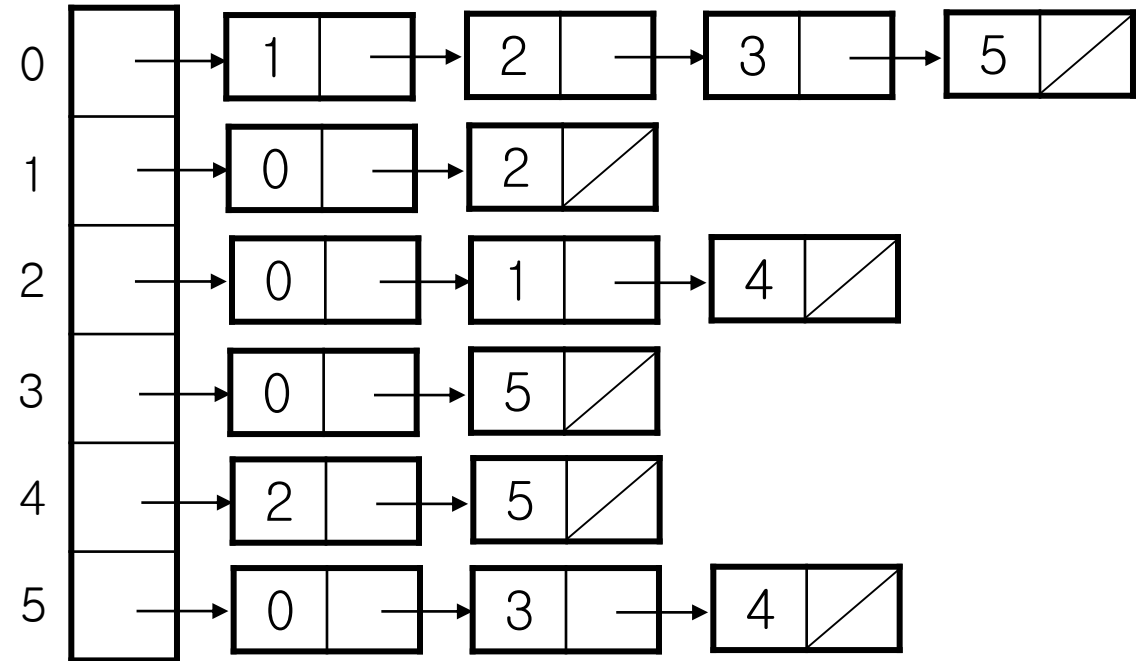
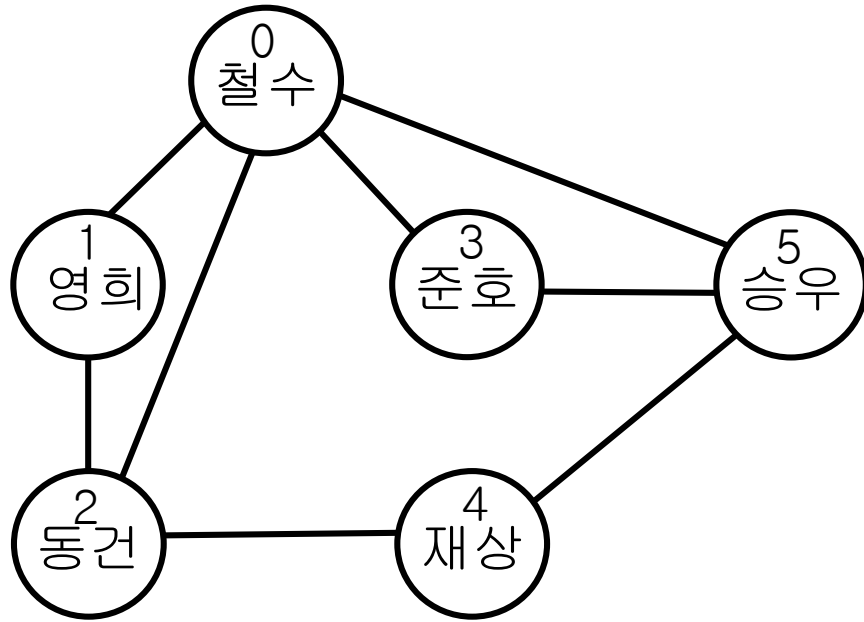
가중 방향 그래프와 인접 행렬 표현

Graph의 표현

- 인접 리스트(Adjacency List)
 - 그래프가 n 개의 연결 리스트로 표현된다.
 - 정점 i 에 연결된 정점들은 연결 리스트 i 로 관리된다
 - 가중치가 있으면 함께 기록한다

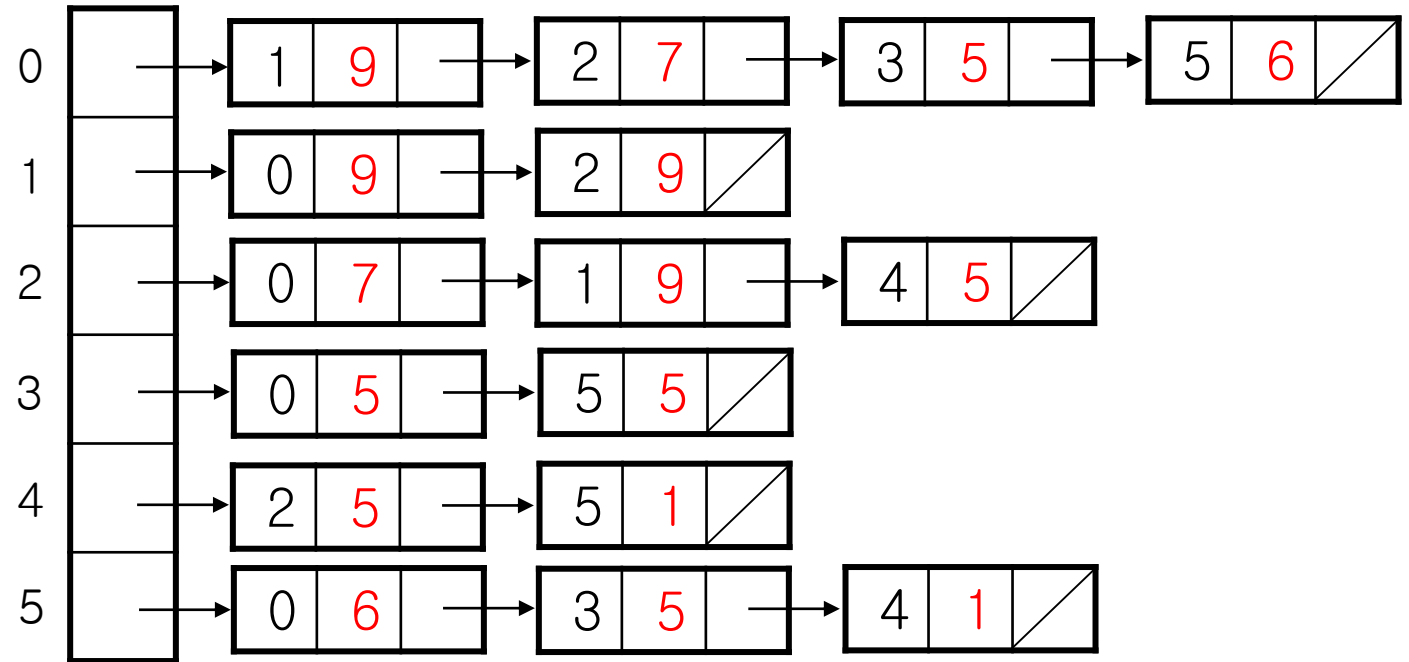
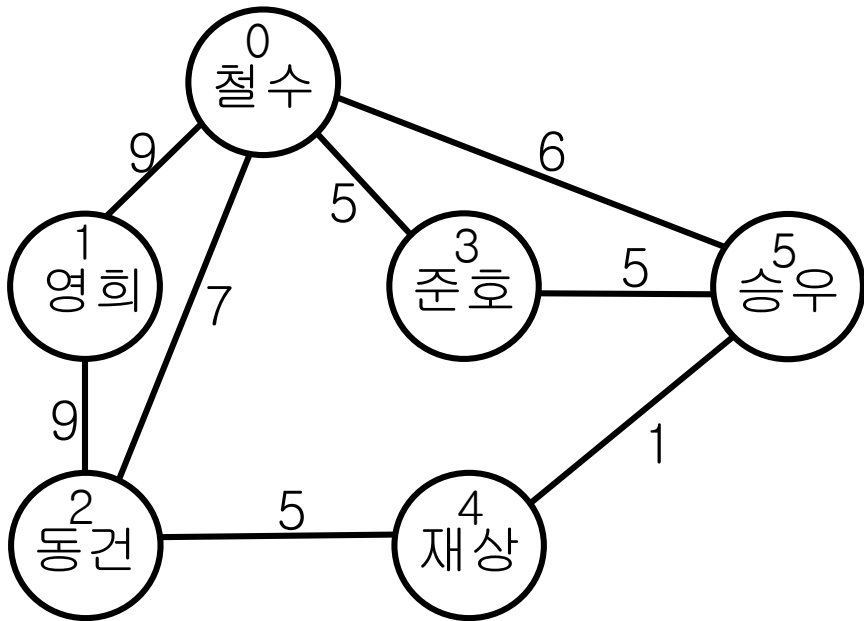
Graph의 표현

- 무향 그래프의 인접 리스트



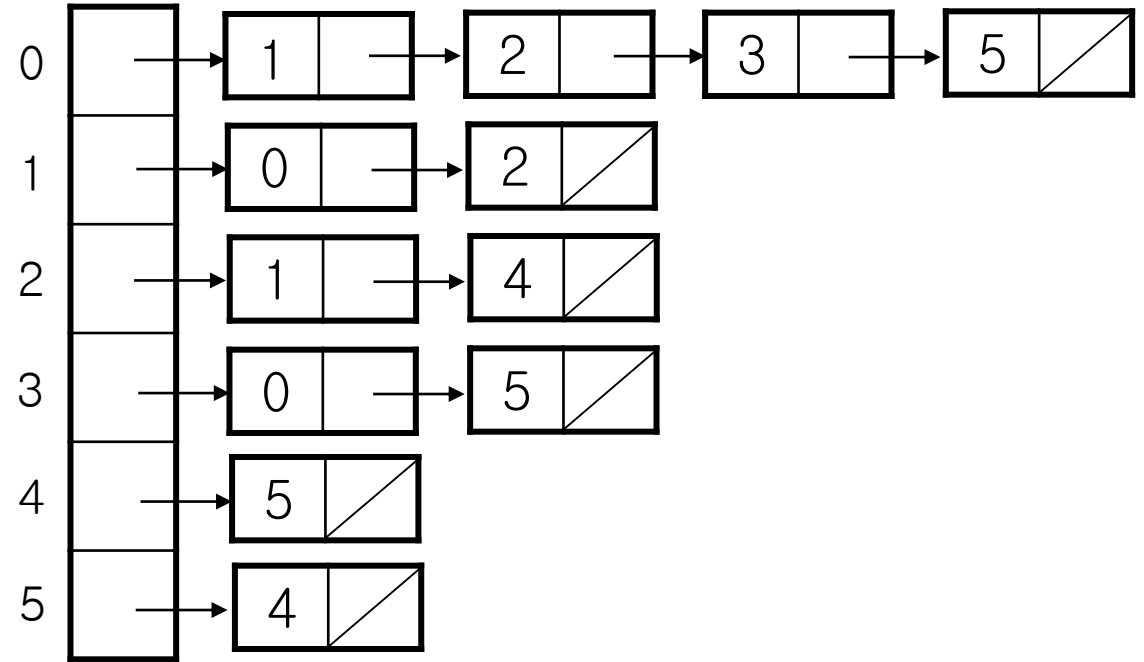
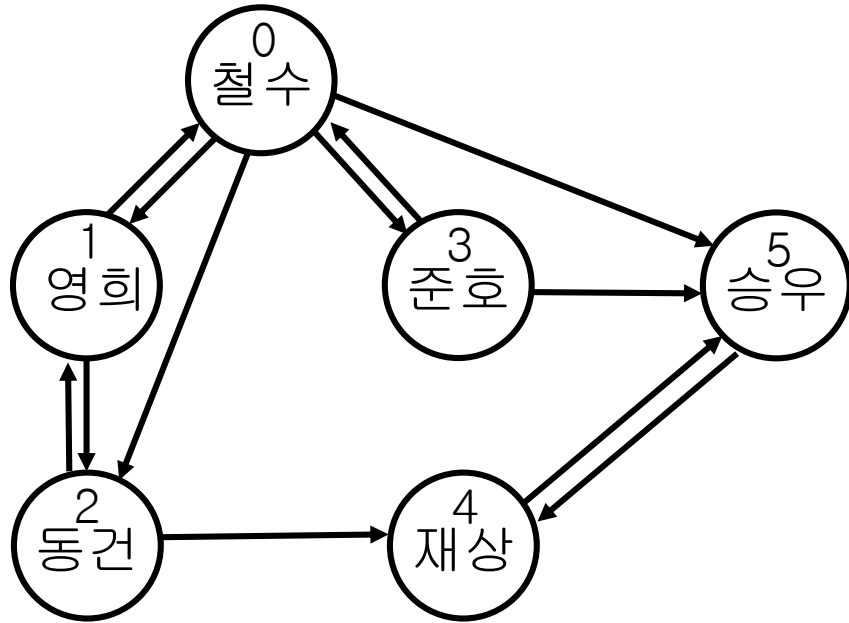
Graph의 표현

- 가중 그래프의 인접 리스트



Graph의 표현

- 방향 그래프의 인접 리스트



Graph의 표현

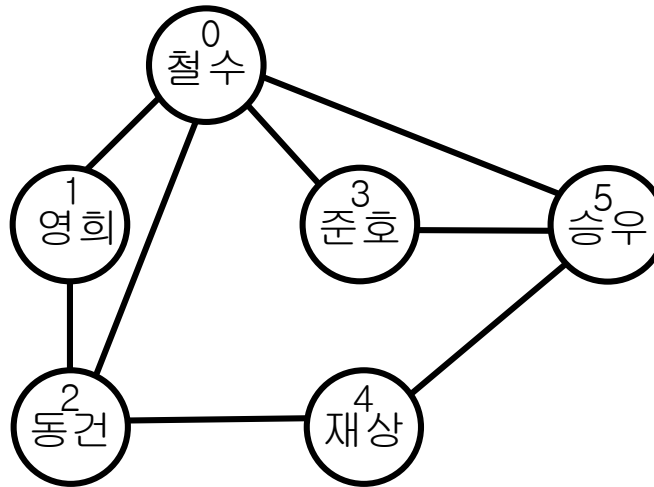
- 인접 배열(Adjacency Array)
 - 그래프가 n 개의 배열(리스트)로 표현된다
 - 필요하면 가중치도 함께 기록한다

Graph의 표현

- 인접 배열(Adjacency Array)

- 그래프가 n 개의 배열(리스트)로 표현된다
- 필요하면 가중치도 함께 기록한다

무방향 그래프와 인접 배열 표현

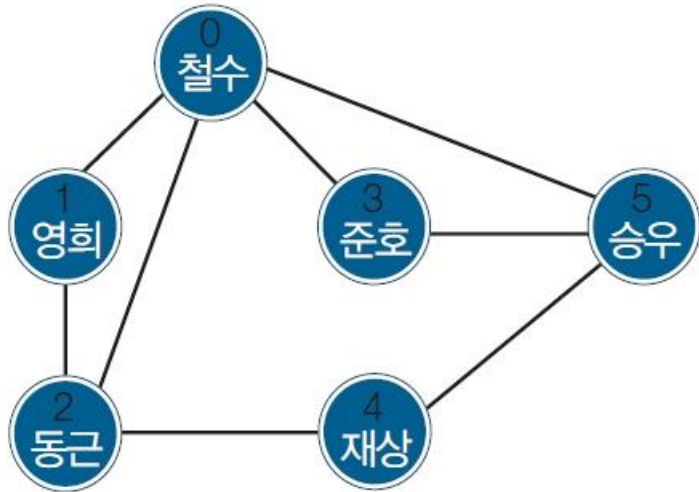


4	—	→	1	2	3	5
2	—	→	0	2		
3	—	→	0	1	4	
2	—	→	0	5		
2	—	→	2	5		
3	—	→	0	3	4	

Graph의 표현

- 인접 해시 테이블(Adjacency Hash Table)
 - 그래프가 n 개의 해시 테이블로 표현된다
 - 필요하면 가중치도 함께 기록한다

무방향 그래프와 인접 해시 테이블 표현



0	7	→		1	2	3		5		
1	11	→	0		2					
2	17	→	0	1				4		
3	21	→	0	5						
4	25	→		5	2					
5	31	→	0				3	4		

$$h(x) = x \% 8$$

$$h(x) = x \% 4$$

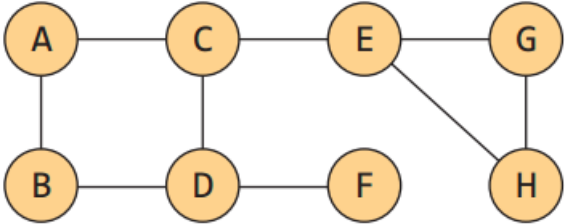
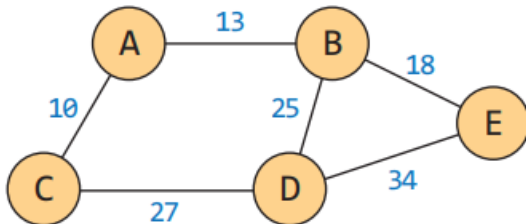
...

소수 아니어도..

인접 행렬과 인접 리스트의 복잡도 비교

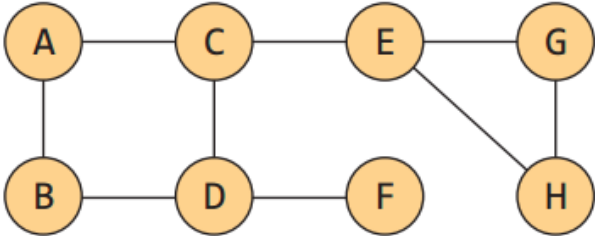
인접 행렬	인접 리스트
간선의 수에 무관하게 항상 n^2 개의 메모리 공간이 필요하다. 따라서 정점에 비해 간선의 수가 매우 많은 조밀 그래프(dense graph)에서 효과적이다.	n 개의 연결 리스트가 필요하고, $2e$ 개의 노드가 필요하다. 즉 $n+2e$ 개의 메모리 공간이 필요하다. 따라서 정점에 비해 간선의 개수가 매우 적은 희소 그래프(sparse graph)에서 효과적이다.
u 와 v 를 연결하는 간선의 유무는 $M[u][v]$ 를 조사하면 바로 알 수 있다. 따라서 $\text{getEdge}(u,v)$ 의 시간 복잡도는 $O(1)$ 이다.	$\text{getEdge}(u,v)$ 연산은 정점 u 의 연결 리스트 전체를 조사해야 한다. 정점 u 의 차수를 d_u 라고 한다면 이 연산의 시간 복잡도는 $O(d_u)$ 이다.
정점의 차수를 구하는 $\text{degree}(v)$ 는 정점 v 에 해당하는 행을 조사하면 되므로 $O(n)$ 이다. 즉, 정점 v 에 대한 차수는 다음과 같이 계산된다. $\text{degree}(v) = \sum_{k=0}^{n-1} M[v][k]$	정점 v 의 차수 $\text{degree}(v)$ 는 v 의 연결 리스트의 길이를 반환하면 된다. 따라서 시간 복잡도는 $O(d_v)$ 이다.
정점 v 의 인접 정점을 구하는 $\text{adjacent}(v)$ 연산은 해당 행의 모든 요소를 검사하면 되므로 $O(n)$ 의 시간이 요구된다.	정점 v 에 간선으로 직접 연결된 모든 정점을 구하는 $\text{adjacent}(v)$ 연산도 해당 연결리스트의 모든 요소를 방문해야 되므로 $O(d_v)$ 이다.
그래프에 존재하는 모든 간선의 수를 알아내려면 인접 행렬 전체를 조사해야 하므로 n^2 번의 조사가 필요하다. 따라서 $O(n^2)$ 의 시간이 요구된다.	전체 간선의 수를 알아내려면 헤더 노드를 포함하여 모든 인접 리스트를 조사해야 하므로 $O(n+e)$ 의 연산이 요구된다.

파이썬을 이용한 인접 행렬 표현

무방향 그래프	인접 행렬 표현
	<pre> vertex = ['A','B','C','D','E','F','G','H'] adjMat = [[0, 1, 1, 0, 0, 0, 0, 0], [1, 0, 0, 1, 0, 0, 0, 0], [1, 0, 0, 1, 1, 0, 0, 0], [0, 1, 1, 0, 0, 1, 0, 0], [0, 0, 1, 0, 0, 0, 1, 1], [0, 0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0, 1], [0, 0, 0, 0, 1, 0, 1, 0]]</pre>
가중치 그래프	인접 행렬 표현
	<pre> vertex = ['A', 'B', 'C', 'D', 'E'] adjMat = [[0, 13, 10, None, None], [13, 0, None, 25, 18], [10, None, 0, 27, None], [None, 25, 27, 0, 34], [None, 18, None, 34, 0]]</pre>

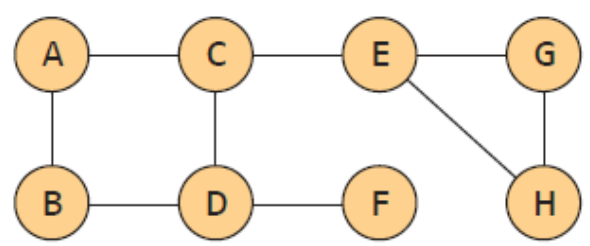
파이썬을 이용한 인접 리스트 표현

- 다양한 방법으로 인접 리스트를 표현할 수 있음
- 인접 정점의 인덱스 리스트를 사용한 경우

그래프	인접 정점 인덱스의 리스트
	<pre>vertex = ['A','B','C','D','E','F','G','H'] adjList = [[1, 2], # 'A'의 인접정점 인덱스 [0, 3], # 'B'의 인접정점 인덱스 [0, 3, 4], # 'C' [1, 2, 5], # 'D' [2, 6, 7], # 'E' [3], # 'F' [4, 7], # 'G' [4, 6]] # 'H'</pre>

파이썬의 딕셔너리를 이용한 표현

- 파이썬의 딕셔너리와 인접 정점 집합 이용한 경우

그래프	딕셔너리와 집합을 이용한 표현
	<pre>graph = { 'A': {'B', 'C'}, 'B': {'A', 'D'}, 'C': {'A', 'D', 'E'}, 'D': {'B', 'C', 'F'}, 'E': {'C', 'G', 'H'}, 'F': {'D'}, 'G': {'E', 'H'}, 'H': {'E', 'G'} }</pre> graph['C']

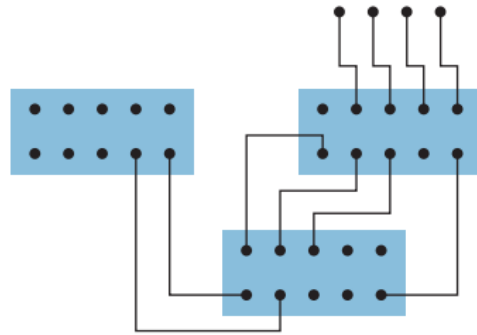
- graph['C'] : 정점 'C'의 인접 정점 집합 { 'A', 'D', 'E' }
- graph['C']의 모든 원소 출력 코드

```

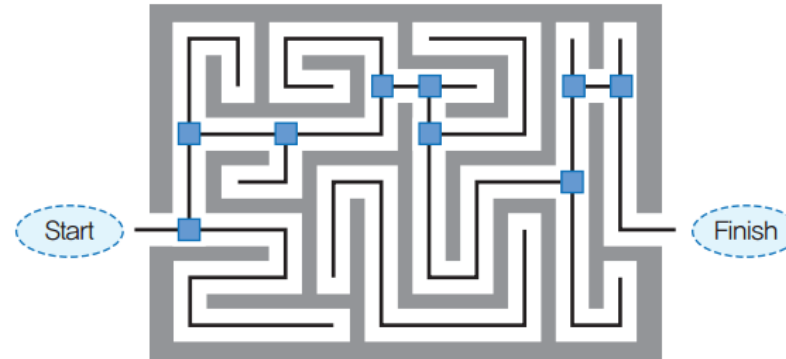
for v in graph['C'] :
    print(v)
# 정점 C의 인접 정점 집합의 모든 원소에 대해
# 그 원소를 화면에 출력
    
```

그래프의 탐색

- 가장 기본적인 연산
 - 시작 정점부터 차례대로 모든 정점들을 한 번씩 방문
 - 많은 문제들이 단순히 탐색만으로 해결됨
 - 도로망 예: 특정 도시에서 다른 도시로 갈 수 있는지 여부
 - 전자회로 예: 특정 단자와 다른 단자의 연결 여부



단자들 간의 연결성 검사

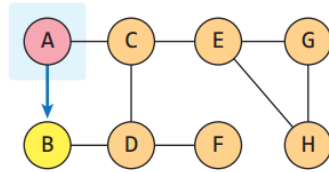


미로 탐색 문제

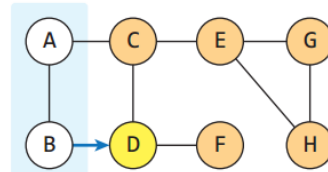
- 기본 탐색 전략
 - 깊이 우선 탐색 (Depth First Search, DFS)
 - 너비 우선 탐색 (Breadth First Search: BFS)

깊이 우선 탐색

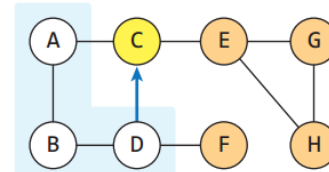
- 한쪽으로 끝까지 가다가 더 이상 갈 수 없으면 가장 가까운 갈림길로 돌아와서 다른 방향으로 다시 탐색
- 되돌아가기를 위해 스택이 사용됨(또는 순환 호출 이용)



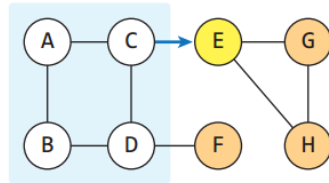
(a) A에서 시작: A→B



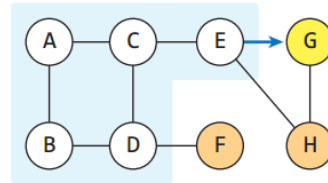
(b) B→D(A는 방문했음)



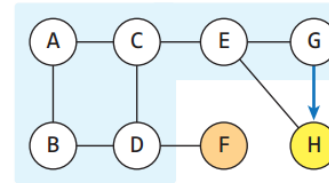
(c) D→C(B는 방문했음)



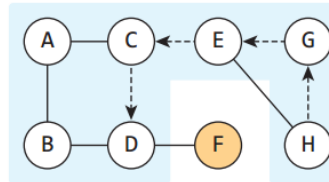
(d) C→E(A, D는 방문했음)



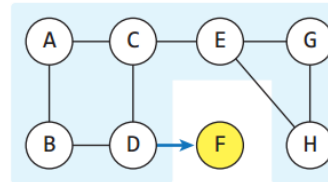
(e) E→G(C는 방문했음)



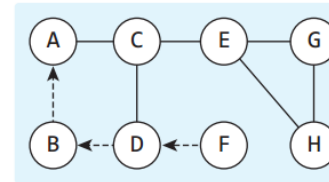
(f) G→H(E는 방문했음)



(g) H에서는 모두 방문했음
G, E, C, D순으로 되돌아 감.
D에서는 가지 않은 F가 있음.



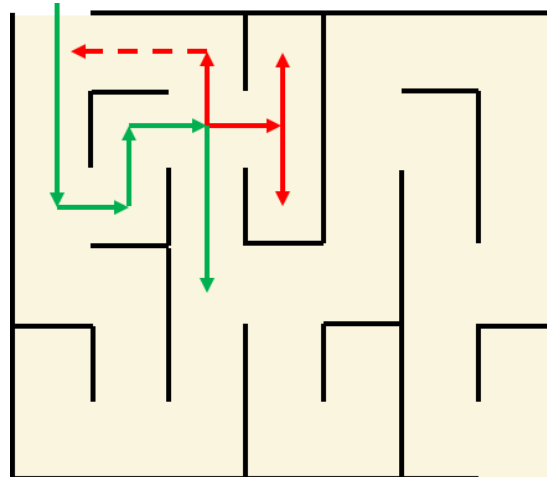
(h) D→F



(i) F에서도 모두 방문했음
D, B, A순으로 되돌아 감
탐색 완료
방문 순서: ABDCEGHF

깊이 우선 탐색

- DFS는 실타래를 가지고 미로에서 출구를 찾는 것과 유사하다.
- 새로운 곳으로 갈 때는 실타래를 풀면서 진행하고, 길이 막혀 진행할 수 없을 때에는 실타래를 되감으며 왔던 길을 되돌아가 같은 방법으로 다른 경로를 탐색하여 출구를 찾는다.
- 그래프에서의 DFS는 임의의 정점에서 시작하여 이웃하는 하나의 정점을 방문하고, 방금 방문한 정점의 이웃 정점을 방문하며, 이웃하는 정점들을 모두 방문한 경우에는 이전 정점으로 되돌아 가서 탐색을 수행하는 방식으로 진행



깊이 우선 탐색

```

01 adj_list = [[2, 1], [3, 0], [3, 0], [9, 8, 2, 1],
02             [5], [7, 6, 4], [7, 5], [6, 5], [3], [3]]
03 N = len(adj_list)
04 visited = [None] * N
05
06 def dfs(v):
07     visited[v] = True
08     print(v, ' ', end='')
09     for w in adj_list[v]:
10         if not visited[w]:
11             dfs(w)
12
13 print('DFS 방문 순서:')
14 for i in range(N):
15     if not visited[i]:
16         dfs(i)

```

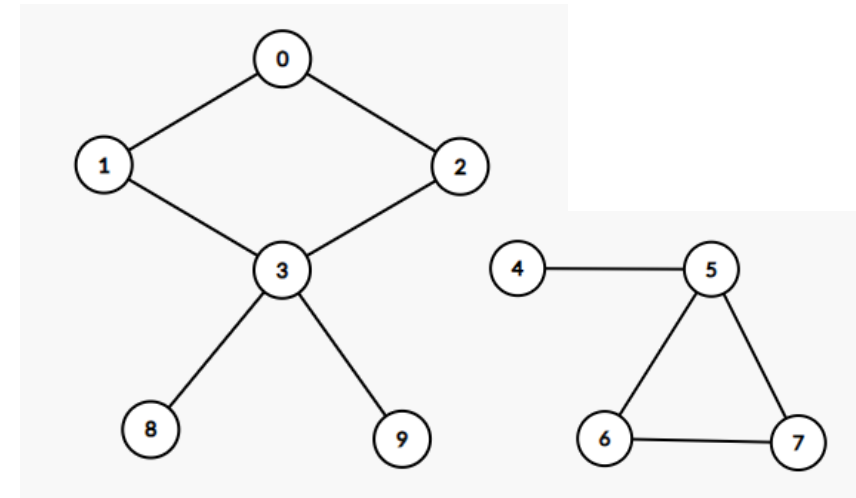
그래프 인접리스트

정점 방문 여부 확인 용

정점 v 방문

정점 v에 인접한 정점으로 dfs() 재귀호출

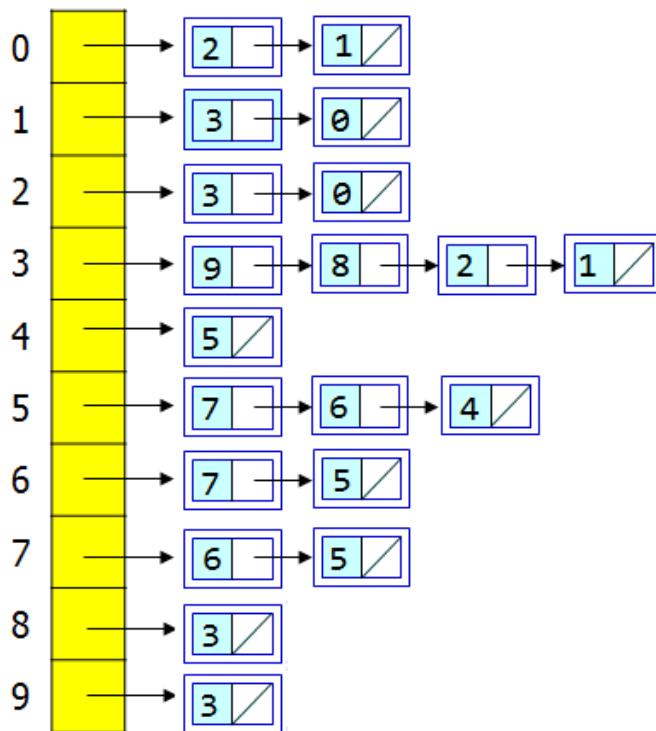
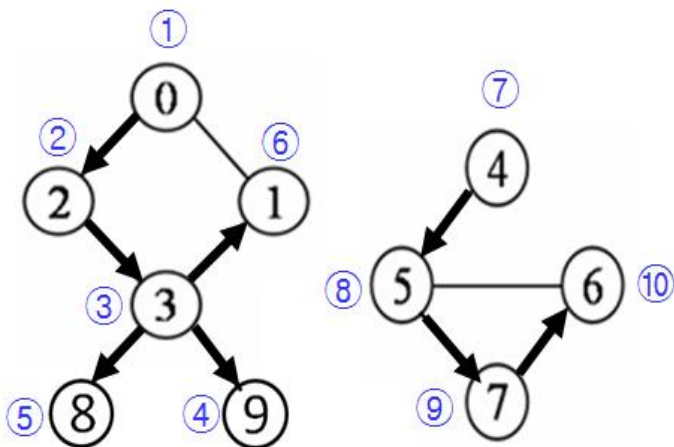
dfs() 호출



DFS 방문 순서:

0 2 3 9 8 1 4 5 7 6

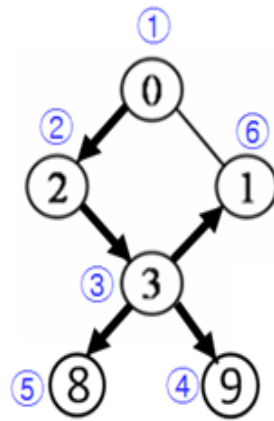
DFS 수행 과정



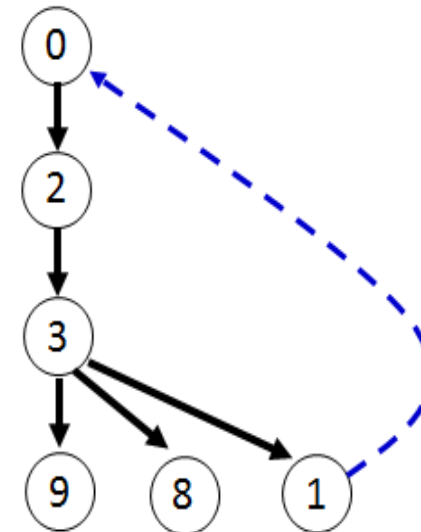
방문순서	dfs()호출	visited[]	출력
①	dfs(0)	visited[0] = True	0
②	dfs(2)	visited[2] = True	2
③	dfs(3)	visited[3] = True	3
④	dfs(9)	visited[9] = True	9
⑤	dfs(8)	visited[8] = True	8
⑥	dfs(1)	visited[1] = True	1
⑦	dfs(4)	visited[4] = True	4
⑧	dfs(5)	visited[5] = True	5
⑨	dfs(7)	visited[7] = True	7
⑩	dfs(6)	visited[6] = True	6

깊이 우선 탐색

- (a)의 DFS 방문순서대로 정점 0부터 위에서 아래방향으로 정점들을 그리면 (b)와 같은 트리가 만들어진다.
- 실선은 탐색하며 처음 방문할 때 사용된 간선이고, 점선은 뒷간선(Back Edge)으로서 탐색 중 이미 방문 된 정점에 도달한 경우를 나타낸다.
- 그래프가 1개의 연결성분으로 되어있을 때 DFS를 수행하며 만들어지는 트리를 깊이우선 신장 트리 (Depth First Spanning Tree)라고 한다.



(a)



(b)

깊이 우선 탐색

- 순환 호출을 이용한 DFS 알고리즘
 - ✓ 인접 행렬 표현을 사용한 경우
 - vtx: 정점 리스트
 - adj: 인접 행렬
 - s: 시작 정점의 인덱스
 - visited: 정점의 방문 확인 리스트

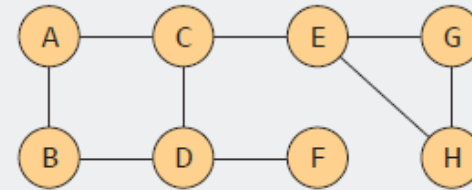
```
def DFS(vtx, adj, s, visited) :
    print(vtx[s], end=' ')
    visited[s] = True

    for v in range(len(vtx)) :
        if adj[s][v] != 0 :
            if visited[v] == False:
                DFS(vtx, adj, v, visited)
```

```
vtx = ['A','B','C','D','E','F','G','H']
edge= [[0, 1, 1, 0, 0, 0, 0, 0], ... ]
```

```
print('DFS(출발:A) : ', end='')
DFS(vtx, edge, 0, [False]*len(vtx))
print()
```

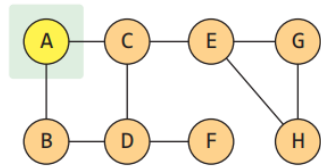
visited를 False 배열로 초기화해 전달



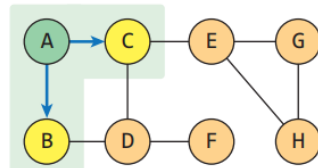
C:\WINDOWS\system32\cmd.exe
DFS(출발:A) : A B D C E G H F

너비 우선 탐색

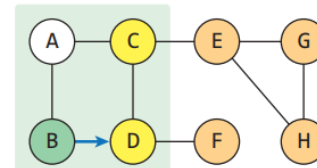
- 시작 정점으로부터 가까운 정점을 먼저 방문하고 멀리 떨어져 있는 정점을 나중에 방문하는 순회 방법
- 큐를 사용하여 구현됨



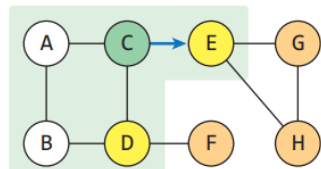
(a) A에서 시작
큐 내용: A



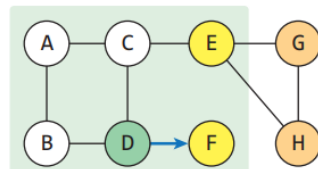
(b) A → B, C
큐 내용: BC



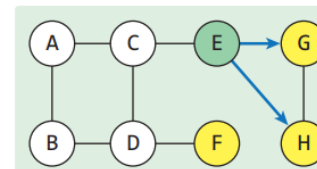
(c) B → D
큐 내용: CD



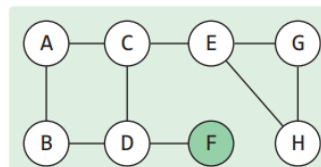
(d) C → E
큐 내용: DE



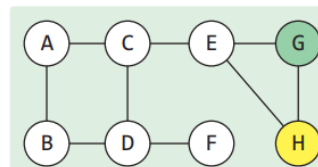
(e) D → F
큐 내용: EF



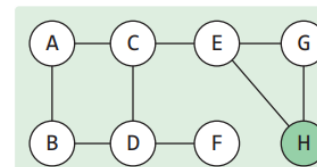
(f) E → G, H
큐 내용: FGH



(g) F에서는 모두 방문했음
큐 내용: GH



(h) G에서는 모두 방문했음
큐 내용: H



(i) H에서도 모두 방문했음
큐 공백상태 → 탐색 완료
방문 순서: ABCDEFGH

너비 우선 탐색

- BFS는 연못에 돌을 던져서 만들어지는 동심원의 물결이 퍼져나가는 것 같이 정점들을 방문한다.
- BFS는 임의의 정점 s 에서 시작하여 s 의 모든 이웃하는 정점들을 방문하고, 방문한 정점들의 이웃 정점들을 모두 방문하는 방식으로 그래프의 모든 정점을 방문
- BFS는 이진트리에서의 레벨순회와 유사



너비 우선 탐색

```

01 adj_list = [[2, 1], [3, 0], [3, 0], [9, 8, 2, 1],
02             [5], [7, 6, 4], [7, 5], [6, 5], [3], [3]]
03 N = len(adj_list)
04 visited = [None] * N
05
06 def bfs(i):
07     queue = []
08     visited[i] = True
09     queue.append(i)
10     while len(queue) != 0:
11         v = queue.pop(0)
12         print(v, ' ', end='')
13         for w in adj_list[v]:
14             if not visited[w]:
15                 visited[w] = True
16                 queue.append(w)
17
18 print('BFS 방문 순서:')
19 for i in range(N):
20     if not visited[i]:
21         bfs(i)

```

그래프 인접리스트

정점 방문 여부 확인 용

큐를 리스트로 구현

큐의 맨 앞에서 제거된 정점을 v가 참조하게 함

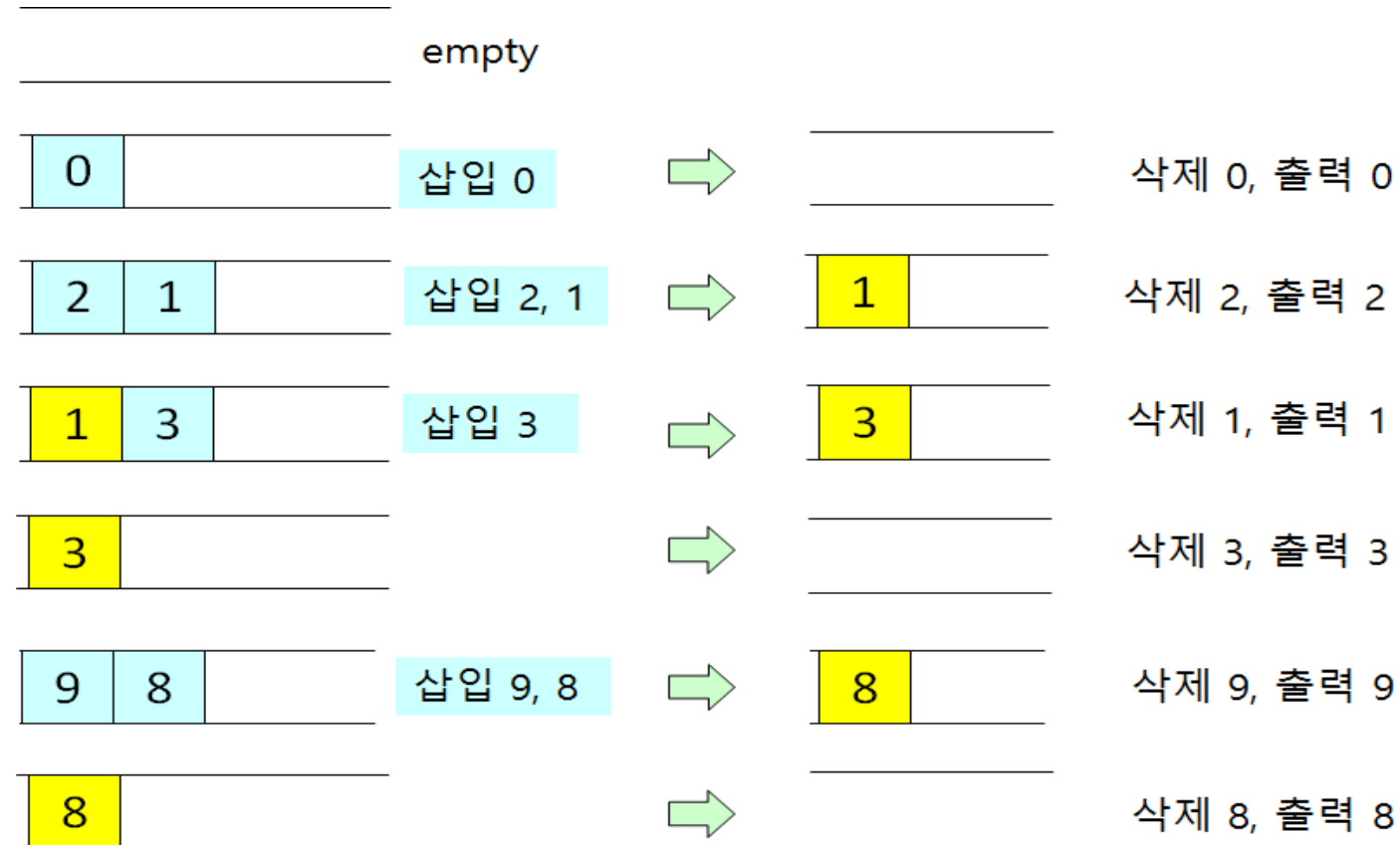
정점 v 방문

v에 인접하면서 방문 안된 정점 큐에 삽입

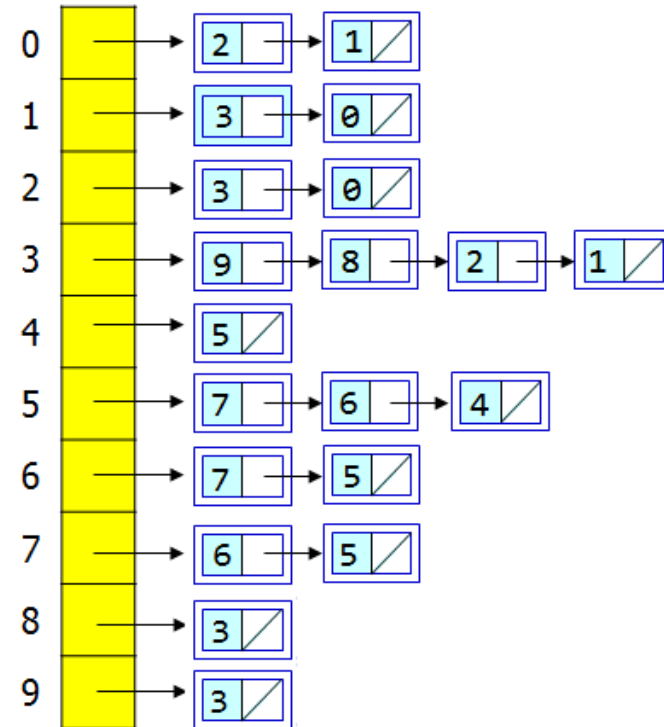
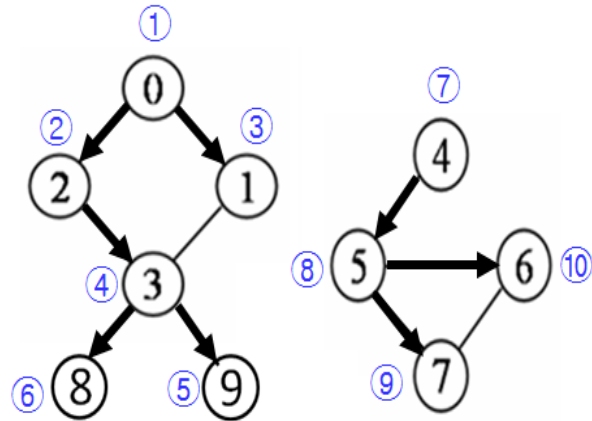
bfs() 호출

너비 우선 탐색

- bfs(0)부터 수행되며 첫 번째 연결성분의 정점들을 모두 방문할 때까지 큐에 정점들이 삽입, 삭제되며 정점들이 출력(방문)될 때의 큐의 상태



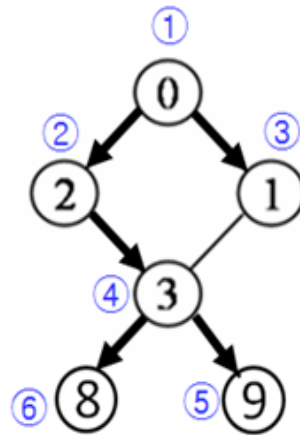
BFS 수행 과정



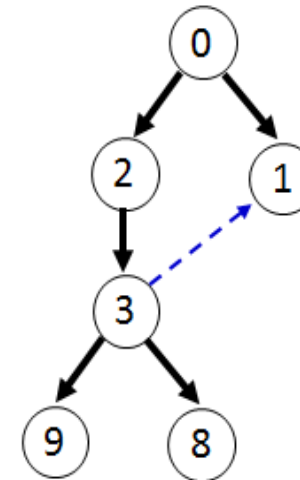
방문순서	visited[]	출력
①	visited[0] = True	0
②	visited[2] = True	2
③	visited[1] = True	1
④	visited[3] = True	3
⑤	visited[9] = True	9
⑥	visited[8] = True	8
⑦	visited[4] = True	4
⑧	visited[5] = True	5
⑨	visited[7] = True	7
⑩	visited[6] = True	6

너비 우선 탐색

- (a)의 그래프에서 BFS 방문순서대로 정점 0부터 위에서 아래방향으로 그려보면 (b)와 같은 트리가 만들어짐
- 실선은 탐색하며 처음 방문할 때 사용된 그래프의 간선이고, 점선은 교차 간선(Cross Edge)으로서 탐색 중 이미 방문된 정점에 도달한 경우를 나타냄
- 그래프가 1개의 연결성분으로 되어 있을 때 BFS를 수행하며 만들어지는 트리: 너비우선 신장 트리 (Breadth First Spanning Tree)



(a)



(b)

너비 우선 탐색

- 인접 리스트로 표현한 그래프의 BFS 알고리즘
 - vtx: 정점 리스트
 - aList: 인접 리스트(파이썬 리스트 이용)
 - s: 시작 정점의 인덱스

```
vtx = ['A','B','C','D','E','F','G','H']  
alist= [[1,2], [0,3], ... ]  
  
print('BFS_AL(출발:U): ', end=" ")  
BFS_AL(vtx, aList, 0)  
print()
```

```
C:\WINDOWS\system32\cmd.exe  
BFS_AL(출발:A): A B C D E F G H
```

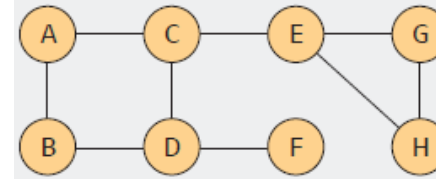
```
from queue import Queue  
def BFS_AL(vtx, aList, s):  
    n = len(vtx)  
    visited = [False]*n  
    Q = Queue()  
    Q.put(s)  
    visited[s] = True  
    while not Q.empty():  
        s = Q.get()  
        print(vtx[s], end=' ')  
        for v in aList[s]:  
            if visited[v]==False:  
                Q.put(v)  
                visited[v] = True
```

탐색 알고리즘 성능

- 깊이 우선 탐색 / 너비 우선 탐색
 - 인접 행렬 표현: $O(n^2)$
 - 인접 리스트로 표현: $O(n + e)$
- 완전 그래프와 같은 조밀 그래프 → 인접 행렬이 유리
- 희소 그래프 → 인접리스트가 유리

딕셔너리와 집합으로 표현한 그래프 탐색

- 딕셔너리와 집합으로 그래프 표현
 - graph: 딕셔너리
 - visited: 정점 방문 확인 집합



```

mygraph = { "A" : {"B","C"},
            "B" : {"A", "D"},
            "C" : {"A", "D", "E"},
            "D" : {"B", "C", "F"},
            "E" : {"C", "G", "H"},
            "F" : {"D"},
            "G" : {"E", "H"},
            "H" : {"E", "G"}
        }
    
```

```

def DFS2(graph, v, visited):
    if v not in visited :
        visited.add(v)
        print(v, end=' ')
        nbr = graph[v] - visited
        for u in nbr:
            DFS2(graph, u, visited)
    
```

```

print('DFS2(출발:A) : ', end="")
DFS2(mygraph, "A", set())
print()
    
```

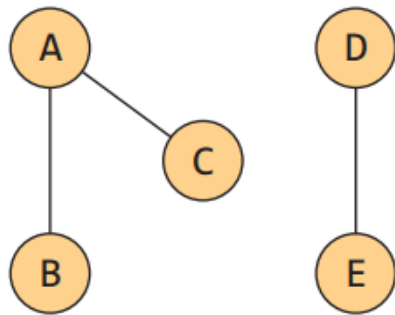
맨 처음에는 공집합을 visited로 전달

```

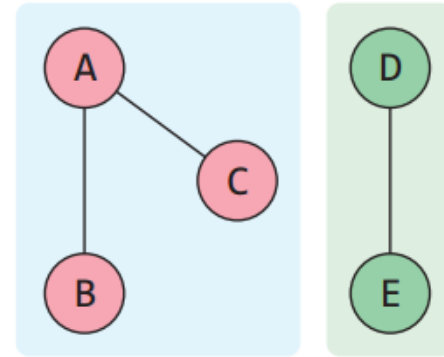
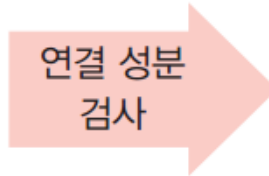
C:\WINDOWS\system32\cmd.exe
DFS2(출발:A) : A C D F B E G H
    
```

연결 성분 검사

- 연결 성분 검사란?
 - 연결 성분: 최대로 연결된 부분 그래프
 - 그래프의 모든 연결 성분을 찾는 작업
 - DFS 또는 BFS를 반복적으로 이용해 해결할 수 있음



원래의 그래프



부분 그래프들

A	1
B	1
C	1
D	2
E	2
label	

위의 그래프에는 2개의 연결된 부분 그래프, 즉 2개의 연결 성분이 있다.

연결 성분 검사 알고리즘

- BFS를 이용한 연결 성분 검사
- 인접 행렬 표현 그래프에 대한 알고리즘

```
def find_connected_component(vtx, adj) :  
    n = len(vtx)  
    visited = [False]*n  
    groups = [] # 연결 성분 리스트  
  
    for v in range(n) :  
        if visited[v] == False :  
            color = bfs_cc(vtx, adj, v, visited)  
            groups.append( color )  
  
    return groups
```

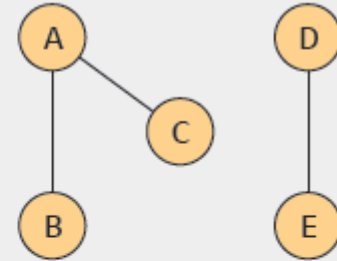
```
from queue import Queue  
def bfs_cc(vtx, adj, s, visited):  
    group = [s]  
    Q = Queue()  
    Q.put(s)  
    visited[s] = True  
    while not Q.empty() :  
        s = Q.get()  
        for v in range(len(vtx)) :  
            if visited[v]==False and adj[s][v] != 0 :  
                Q.put(v)  
                visited[v] = True  
                group.append(v)  
    return group
```

연결 성분 검사 알고리즘

- 테스트 프로그램

```

vertex = ['A', 'B', 'C', 'D', 'E']
adjMat = [ [ 0, 1, 1, 0, 0 ],
            [ 1, 0, 0, 0, 0 ],
            [ 1, 0, 0, 0, 0 ],
            [ 0, 0, 0, 0, 1 ],
            [ 0, 0, 0, 1, 0 ] ]
    
```



```

colorGroup = find_connected_component(vertex, adjMat)
print("연결성분 개수 = %d " % len(colorGroup))
print(colorGroup)    # 정점 리스트들을 출력
    
```

```

C:\WINDOWS\system32\cmd.exe
연결성분 개수 = 2
[[0, 1, 2], [3, 4]]
    
```

연결 성분 검사

- 주어진 정점으로부터 DFS를 수행하여 방문되는 정점들을 리스트로 모아서 1개의 연결성분을 만든다. 다음엔 방문 안된 정점으로부터 같은 과정을 반복하여 다른 연결성분들을 추출한다.

```

01 adj_list = [[3], [6, 10], [7, 11], [0, 6, 8], [13], [14],
02             [1, 3, 8, 10, 11], [2, 11], [3, 6, 10, 12],
03             [13], [1, 6, 8], [2, 6, 7], [8], [4, 9], [5]]
    
```

그래프 인접리스트

```

04 N = len(adj_list)
    
```

```

05 CCList = []
    
```

```

06 visited = [None] * N
    
```

정점 방문 여부 확인 용

```

07 def dfs(v):
    
```

현재 연결성분에
정점 v 추가

```

08     visited[v] = True
    
```

```

09     cc.append(v)
    
```

```

10     for w in adj_list[v]:
    
```

정점 v에 인접한 w로
dfs() 재귀호출

```

11         if not visited[w]:
    
```

```

12             dfs(w)
    
```

```

13
14 for i in range(N):
    
```

```

15     if not visited[i]:
    
```

새로운 연결성분(cc)을
위한 초기화

```

16         cc = []
    
```

```

17         dfs(i)
    
```

dfs() 호출로 cc 만들기

```

18         CCList.append(cc)
    
```

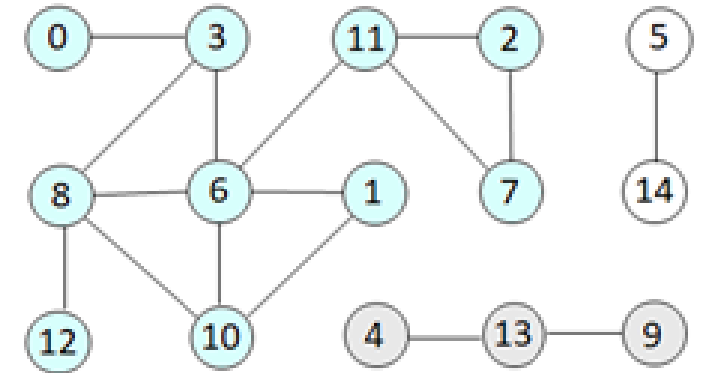
```

19
20 print('연결성분 리스트: ');
    
```

```

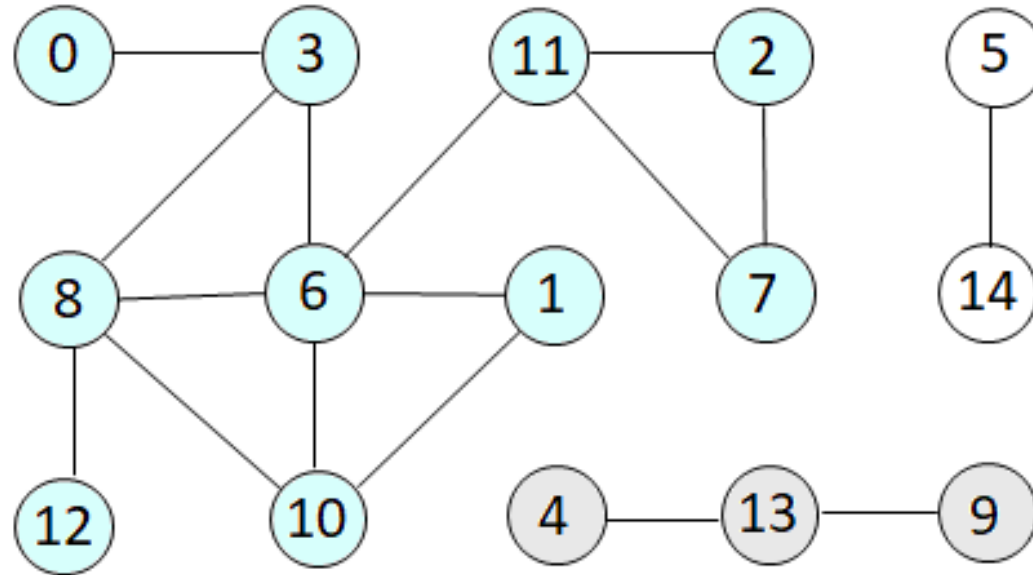
21 print(CCList)
    
```

완성된 cc를 연결성분
리스트에 추가



연결 성분 검사

- 프로그램 수행 결과

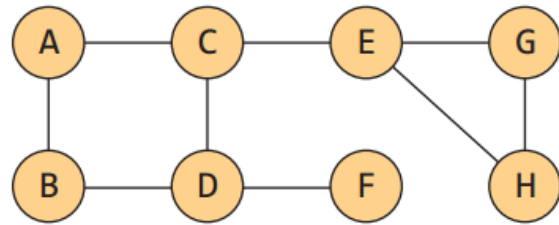


연결성분 리스트:

[[0, 3, 6, 1, 10, 8, 12, 11, 2, 7], [4, 13, 9], [5, 14]]

신장 트리(Spanning tree)

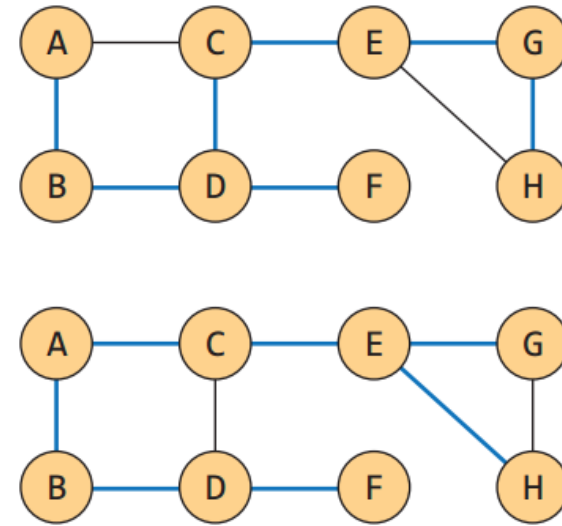
- 그래프 내의 모든 정점을 포함하는 트리
- 사이클을 포함하면 안됨 간선의 수 = $n-1$
- 그래프 탐색 도중에 사용된 간선들만 모으면 됨



연결 그래프

깊이우선
탐색의 간선

너비우선
탐색의 간선



신장 트리의 예

신장 트리 알고리즘

- DFS를 이용한 신장 트리
- 인접 행렬 표현 그래프에 대한 알고리즘

```
def ST_DFS(vtx, adj, s, visited) :  
    visited[s] = True  
    for v in range(len(vtx)) :  
        if adj[s][v] != 0 :  
            if visited[v]==False:  
                print("(" + vtx[s], vtx[v], ")", end=' ')  
                ST_DFS(vtx, adj, v, visited)
```



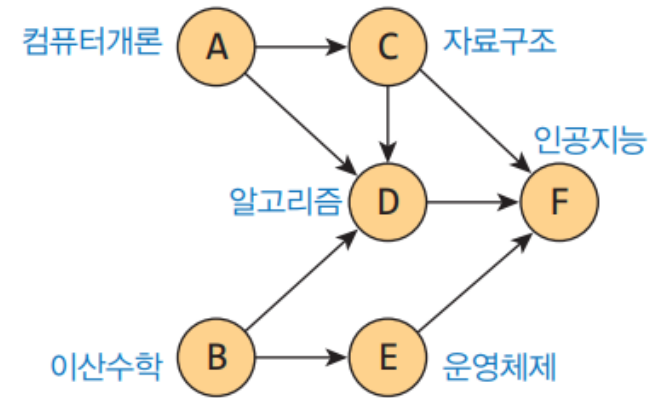
C:\WINDOWS\system32\cmd.exe
신장트리(DFS): (A B) (B D) (D C) (C E) (E G) (G H) (D F)

위상 정렬

- 방향 그래프에 대해 정점들의 선행 순서를 위배하지 않으면서 모든 정점을 나열하는 것

과목번호	과목명	선수과목
A	컴퓨터개론	없음
B	이산수학	없음
C	자료구조	A
D	알고리즘	A, B, C
E	운영체제	B
F	인공지능	C, D, E

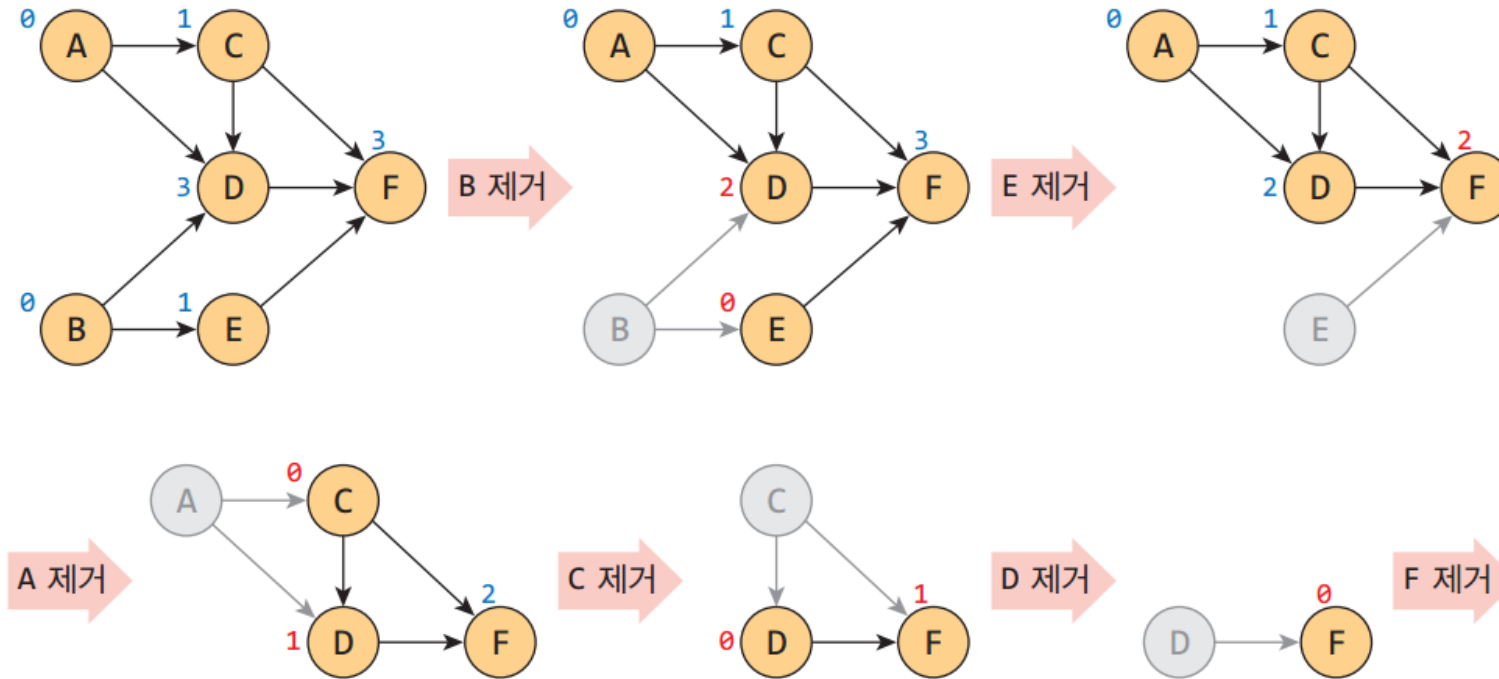
교과목의 선후수 관계 표



방향 그래프로 표시한 선후수 관계

- 가능한 정렬: $\langle A, B, C, D, E, F \rangle$, $\langle B, A, C, D, E, F \rangle$, $\langle A, C, B, E, D, F \rangle$ 등
- 잘못된 정렬: $\langle C, A, B, D, E, F \rangle$

위상 정렬의 예



- 위상 순서(topological order) : <B,E,A,C,D,F>

위상 정렬 알고리즘

```
def topological_sort_AM(vertex, edge) :  
    n = len(vertex)  
    inDeg = [0] * n    # 진입차수 저장  
    for i in range(n) :  
        for j in range(n) :  
            if edge[i][j]>0 :  
                inDeg[j] += 1  
  
    vlist = []  
    for i in range(n) :  
        if inDeg[i]==0 :  
            vlist.append(i)
```

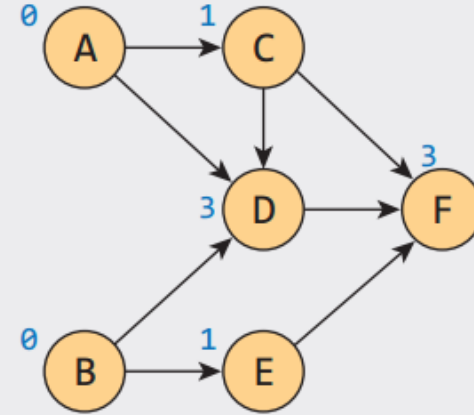
```
while len(vlist) > 0 :  
    v = vlist.pop()  
    print(vertex[v], end=' ')  
  
    for u in range(n) :  
        if v!=u and edge[v][u]>0:  
            inDeg[u] -= 1  
            if inDeg[u] == 0 :  
                vlist.append(u)
```

위상 정렬 알고리즘 테스트

```

vertex = ['A', 'B', 'C', 'D', 'E', 'F']
graphAM = [ [ 0, 0, 1, 1, 0, 0 ],
             [ 0, 0, 0, 1, 1, 0 ],
             [ 0, 0, 0, 1, 0, 1 ],
             [ 0, 0, 0, 0, 0, 1 ],
             [ 0, 0, 0, 0, 0, 1 ],
             [ 0, 0, 0, 0, 0, 0 ] ]

print('topological_sort: ')
topological_sort_AM(vertex, graphAM)
print()
    
```



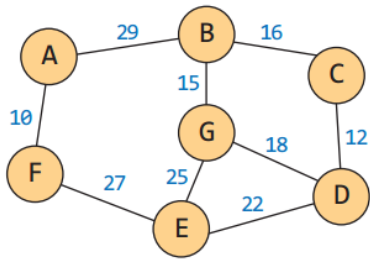
```

C:\WINDOWS\system32\cmd.exe
topological_sort:
B E A C D F
    
```

가중치 그래프(Weighted Graph)

- 간선에 가중치가 할당된 그래프
- $G = (V, E, w)$
 - ✓ w : 비용, 가중치(weight)

인접 행렬을 이용한 가중치 그래프 표현 예



인접 행렬
표현

	A	B	C	D	E	F	G
A	0	29	∞	∞	∞	10	∞
B	29	0	16	∞	∞	∞	15
C	∞	16	0	12	∞	∞	∞
D	∞	∞	12	0	22	∞	18
E	∞	∞	∞	22	0	27	25
F	10	∞	∞	∞	27	0	∞
G	∞	15	∞	18	25	∞	0

파이썬에서 2차원 배열은 리스트의 리스트

```

vertex = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
weight = [
    [None, 29, None, None, None, 10, None],
    [29, None, 16, None, None, None, 15],
    [None, 16, None, 12, None, None, None],
    [None, None, 12, None, 22, None, 18],
    [None, None, None, 22, None, 27, 25],
    [10, None, None, None, 27, None, None],
    [None, 15, None, 18, 25, None, None]
]
    
```

인접 행렬: 간단한 연산

- 예) 인접 행렬에서의 가중치의 합 계산

```
def weightSum( vlist, W ):  
    sum = 0  
    for i in range(len(vlist)) :  
        for j in range(i+1, len(vlist)) :  
            if W[i][j] != None :  
                sum += W[i][j]  
    return sum
```

매개변수: 정점 리스트, 인접 행렬
가중치의 합
모든 정점에 대해(i: 0, ... N-1)
하나의 행에 대해 (삼각영역)
만약 간선이 있으면
sum에 추가
전체 가중치 합을 반환

```
print('AM : weight sum = ', weightSum(vertex, weight))
```



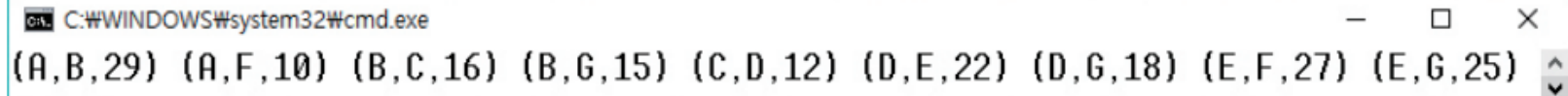
C:\WINDOWS\system32\cmd.exe
AM : weight sum = 174

인접 행렬: 간단한 연산

- 예) 인접 행렬에서의 모든 간선 출력

```
def printAllEdges(vlist, W):  
    for i in range(len(vlist)) :  
        for j in range(i+1, len(W[i])) :  
            if W[i][j] != None and W[i][j] != 0 :  
                print("(%s,%s,%d)"%(vlist[i],vlist[j],W[i][j]), end=' ')  
    print()
```

```
printAllEdges(vertex, weight)
```

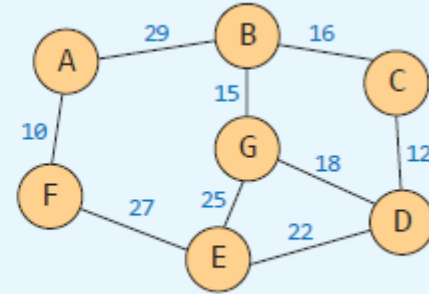


```
C:\WINDOWS\system32\cmd.exe  
(A,B,29) (A,F,10) (B,C,16) (B,G,15) (C,D,12) (D,E,22) (D,G,18) (E,F,27) (E,G,25)
```

인접 리스트를 이용한 표현

- 딕셔너리, 집합, 튜플, 리스트를 이용한 그래프 표현 예

```
graph = {'A' : {('B',29),('F',10)      },
        'B' : {('A',29),('C',16), ('G',15)},
        'C' : {('B',16),('D',12)      },
        'D' : {('C',12),('E',22), ('G',18)},
        'E' : {('D',22),('F',27), ('G',25)},
        'F' : {('A',10),('E',27)      },
        'G' : {('B',15),('D',18), ('E',25)} }
```



가중치의 합 계산 함수

```
def weightSum(graph):
    sum = 0
    for v in graph:
        for e in graph[v]:
            sum += e[1]
    return sum//2
```

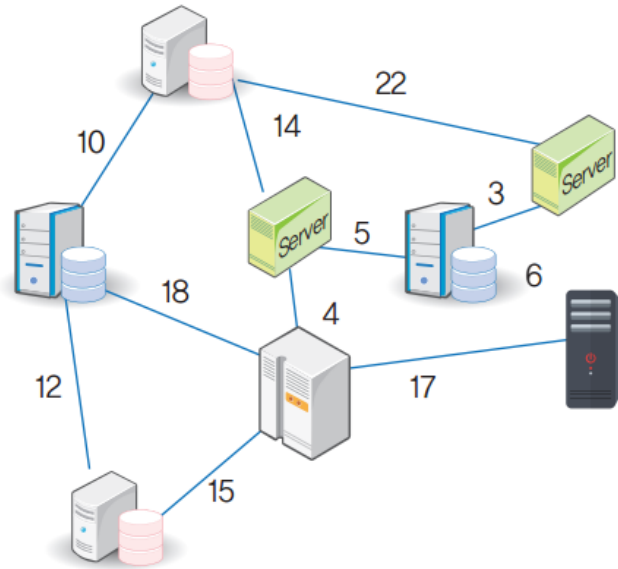
모든 간선 출력 함수

```
def printAllEdges(graph):
    for v in graph:
        for e in graph[v]:
            print("(%s,%s,%d)"%(v,e[0],e[1]), end=' ')
```

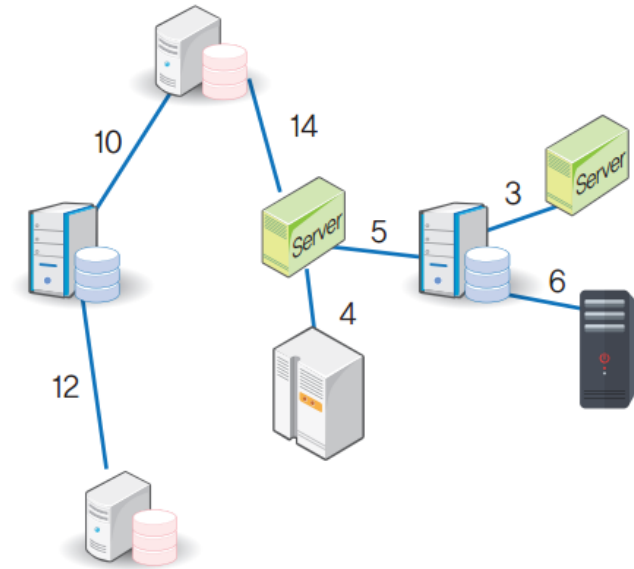
```
C:\WINDOWS\system32\cmd.exe
AL : weight sum = 174
(A,F,10) (A,B,29) (B,A,29) (B,G,15) (B,C,16) (C,B,16) (C,D,12) (D,C,12) (D,G,18)
(D,E,22) (E,F,27) (E,D,22) (E,G,25) (F,A,10) (F,E,27) (G,B,15) (G,E,25) (G,D,18)
```

최소비용 신장트리

- 간선들의 가중치 합이 최소인 신장 트리
- 반드시 $(n-1)$ 개의 간선만 사용
- 사이클이 포함되면 안됨



(a) 사이트 사이의 연결 비용



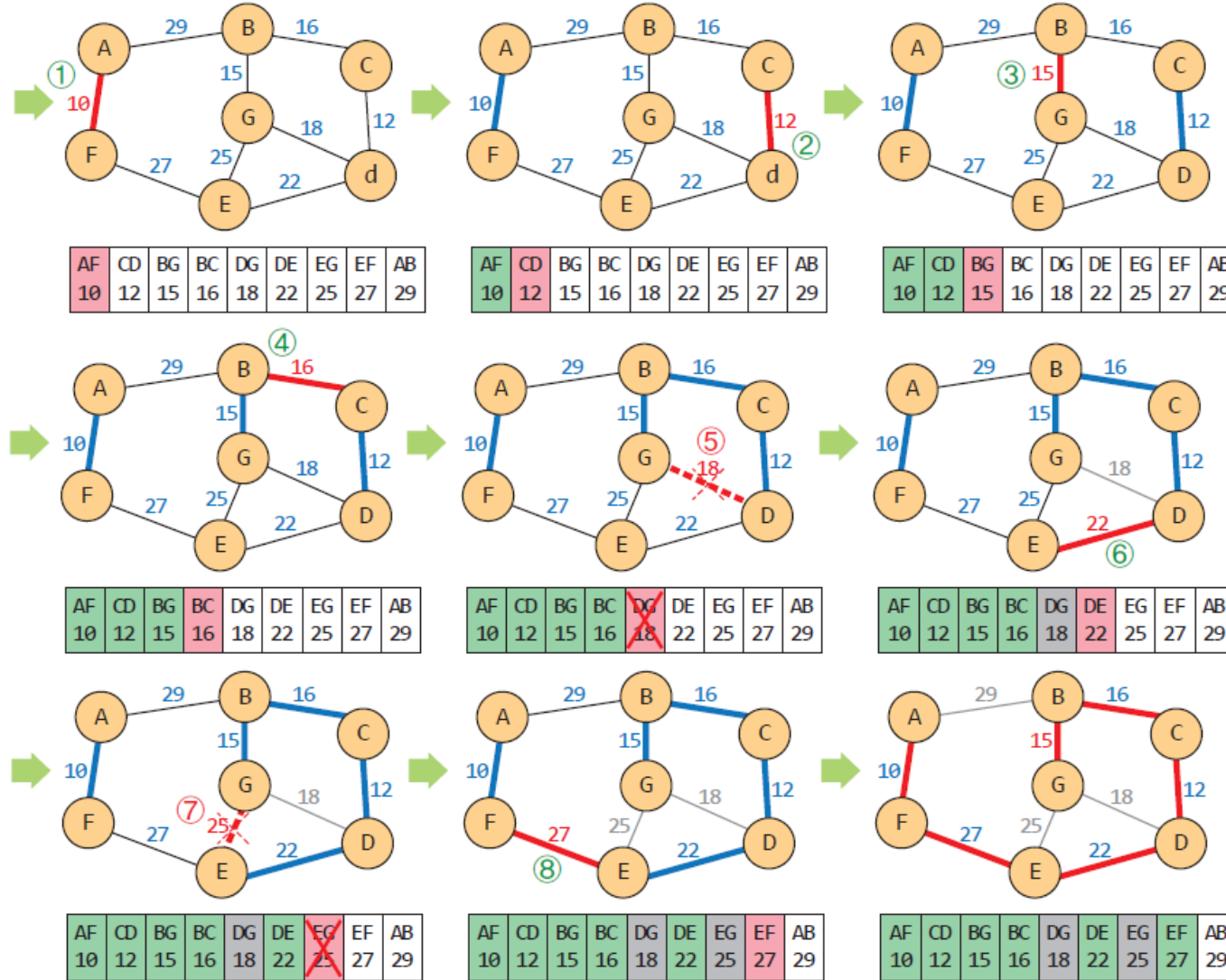
(b) 최소비용 신장트리의 예

- MST의 응용
 - 도로, 통신, 배관 건설: 모두 연결하면서 길이/비용을 최소화
 - 전기 회로: 단자를 모두 연결하면서 전선의 길이를 최소화
- Kruskal 알고리즘, Prim 알고리즘, Sollin 알고리즘

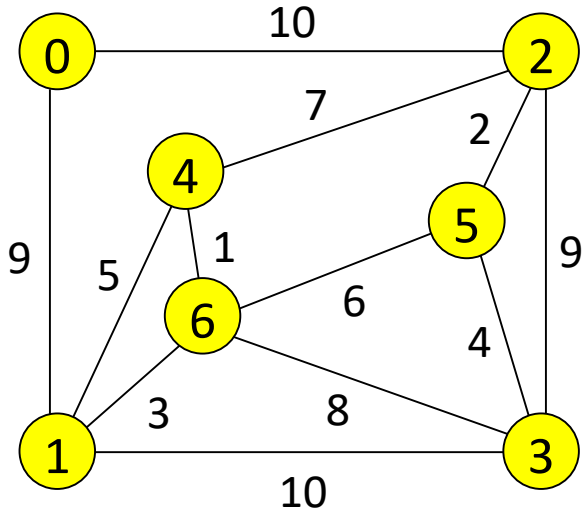
Kruskal의 MST 알고리즘

- 탐욕적인 방법(greedy method)
 - "그 순간에 최적"이라고 생각되는 것을 선택
 - 각 단계에서 최선의 답을 선택 최종적인 해답에 도달
 - 항상 최적의 해답을 주는지 검증 필요함
 - Kruskal MST 알고리즘은 최적의 해답임이 증명됨
- 알고리즘
 1. 그래프의 모든 간선을 가중치에 따라 오름차순으로 정렬
 2. 가장 가중치가 작은 간선 e 를 뽑음
 3. e 를 MST에 넣었을 때 사이클이 생기면 → 2번으로 이동
 4. 사이클이 생기지 않으면 MST에 삽입
 5. $n-1$ 개의 간선이 삽입될 때 까지 2번으로 이동

Kruskal의 MST 알고리즘



Kruskal의 MST 알고리즘 예제



정렬된 L

(0, 1) 9
 (0, 2) 10
 (1, 3) 10
 (1, 4) 5
 (1, 6) 3
 (2, 3) 9
 (2, 4) 7
 (2, 5) 2
 (3, 5) 4
 (3, 6) 8
 (4, 6) 1
 (5, 6) 6

(4, 6) 1
 (2, 5) 2
 (1, 6) 3
 (3, 5) 4
 (1, 4) 5
 (5, 6) 6
 (2, 4) 7
 (3, 6) 8
 (0, 1) 9
 (2, 3) 9
 (0, 2) 10
 (1, 3) 10

Kruskal의 MST 알고리즘 예제

(4, 6) 1

(2, 5) 2

(1, 6) 3

(3, 5) 4

(1, 4) 5

(5, 6) 6

(2, 4) 7

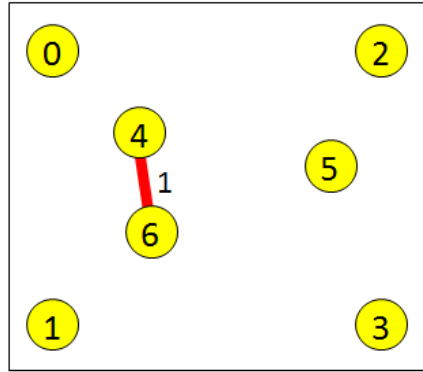
(3, 6) 8

(0, 1) 9

(2, 3) 9

(0, 2) 10

(1, 3) 10



(2, 5) 2

(1, 6) 3

(3, 5) 4

(1, 4) 5

(5, 6) 6

(2, 4) 7

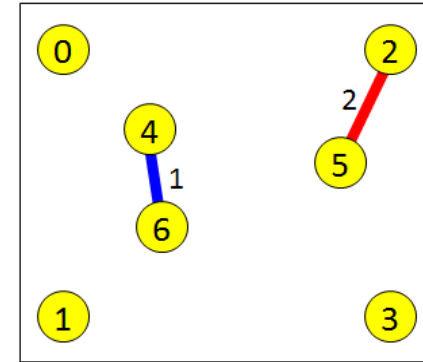
(3, 6) 8

(0, 1) 9

(2, 3) 9

(0, 2) 10

(1, 3) 10



(1, 6) 3

(3, 5) 4

(1, 4) 5

(5, 6) 6

(2, 4) 7

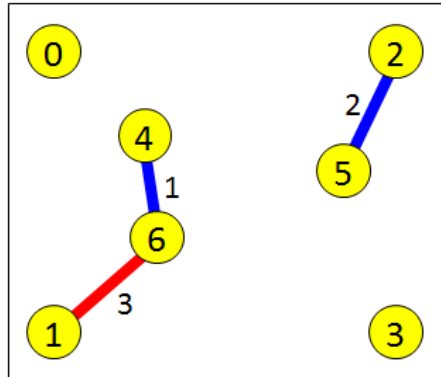
(3, 6) 8

(0, 1) 9

(2, 3) 9

(0, 2) 10

(1, 3) 10



(3, 5) 4

(1, 4) 5

(5, 6) 6

(2, 4) 7

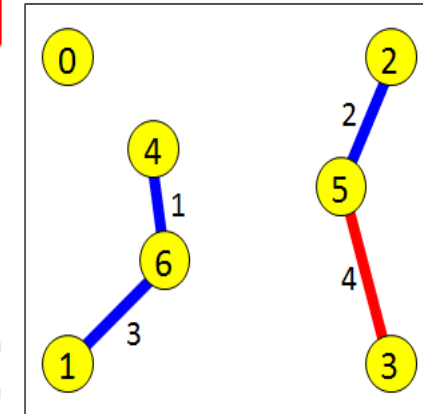
(3, 6) 8

(0, 1) 9

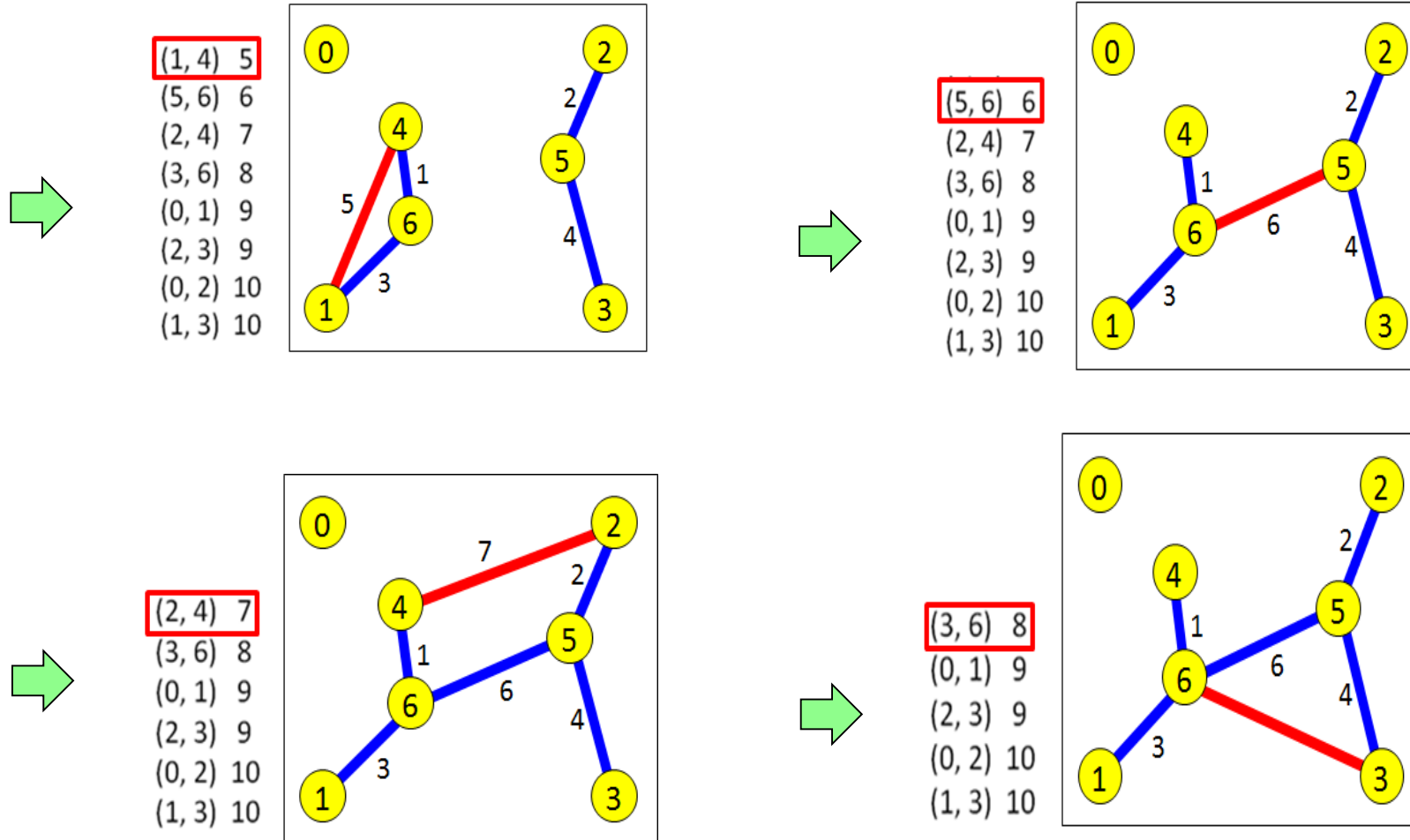
(2, 3) 9

(0, 2) 10

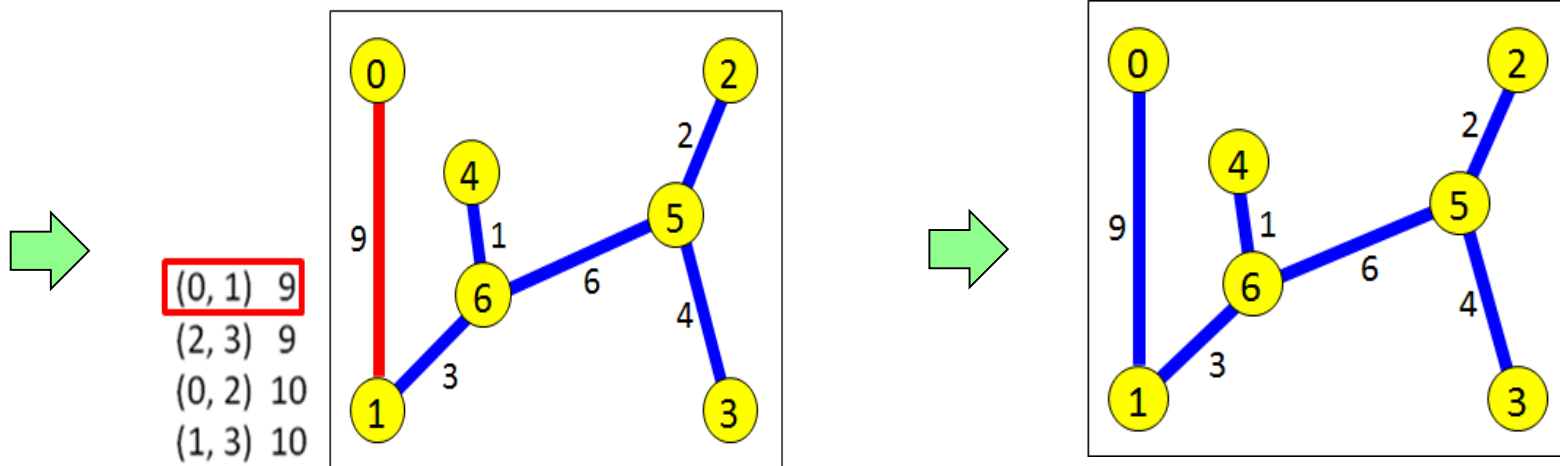
(1, 3) 10



Kruskal의 MST 알고리즘 예제



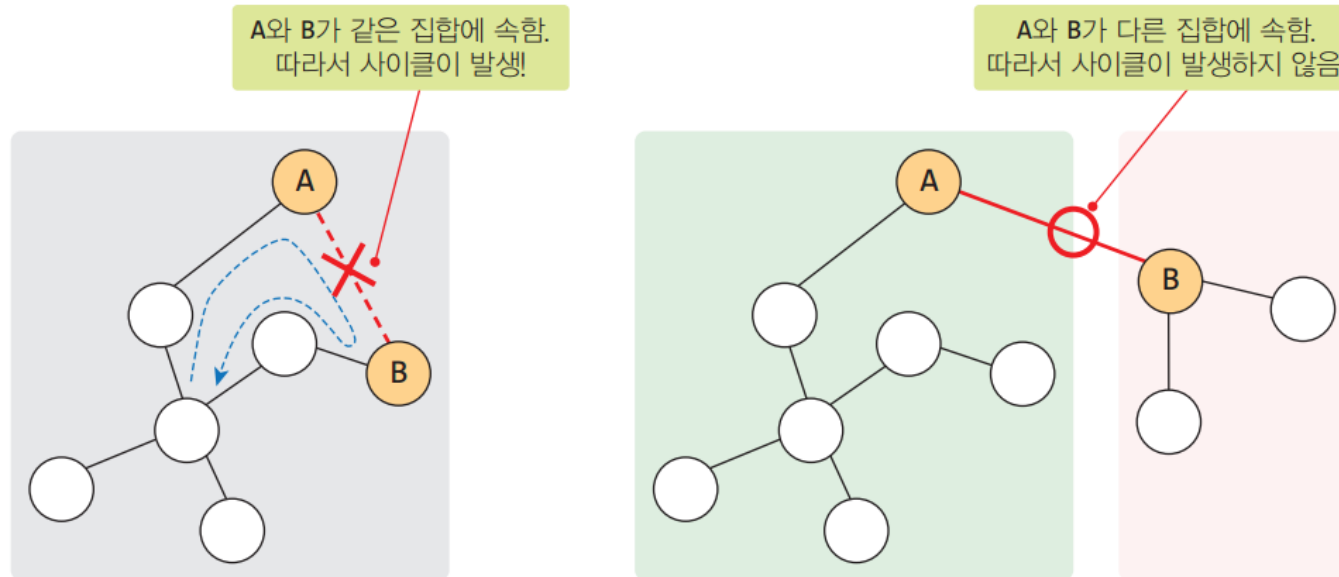
Kruskal의 MST 알고리즘 예제



최소신장트리의 간선의 가중치의 합 = $1 + 2 + 3 + 4 + 6 + 9 = 25$

사이클 검사: union-find 알고리즘

- 간선 추가 시 사이클 검사 과정이 필요

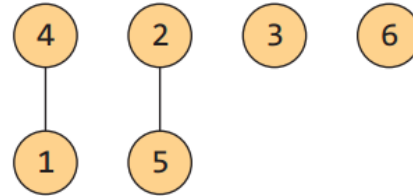


- Union-Find 알고리즘
 - 서로소(disjoint)인 집합들을 표현할 때 사용하는 독특한 형태의 집합 자료구조
 - union 연산: 두 집합의 합집합을 만드는 연산
 - find 연산: 원소가 속한 집합을 찾는 연산(대표 원소를 반환)
 - 트리를 이용해 구현

union과 find 연산 예

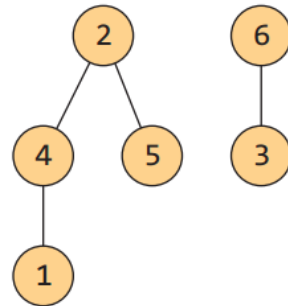

 ← 서로소(disjoint)인 집합들

↓ union(1, 4)
 union(5, 2)



find(1) = 4
 find(2) = 2

↓ union(4, 5)
 union(3, 6)



find(1) = 2
 find(5) = 2
 find(3) = 6

union-find 알고리즘

- 서로소 집합의 초기화
- find(id) 연산
 - 대표 원소의 인덱스 반환
 - 루트의 인덱스 반환
- union(s1, s2) 연산
 - 집합을 병합함
 - s1을 s2에 병합
 - 집합의 수가 1개 감소

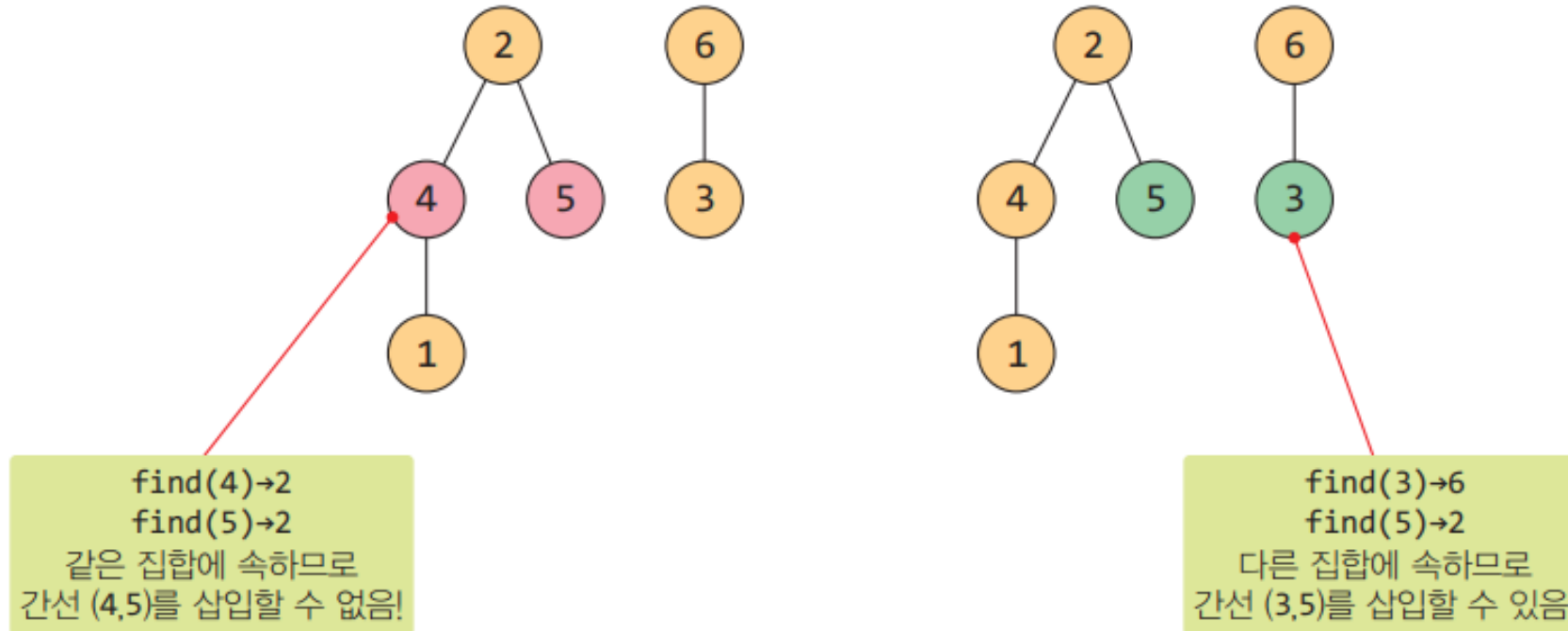
```
parent = []  
set_size = 0  
  
def init_set(nSets) :  
    global set_size, parent  
    set_size = nSets;  
    for i in range(nSets):  
        parent.append(-1)
```

```
def find(id) :  
    while (parent[id] >= 0) :  
        id = parent[id]  
    return id;
```

```
def union(s1, s2) :  
    global set_size  
    parent[s1] = s2  
    set_size = set_size - 1
```

find()를 이용한 간선의 사이클 검사

- 간선 (4, 5)를 추가하면 사이클이 생길까?
- find(4)와 find(5)의 결과가 같으면 사이클이 발생



- 간선 (5, 3)을 추가하면 사이클이 생기지 않음

Kruskal의 MST 알고리즘

```

01 weights = [(0, 1, 9), (0, 2, 10), (1, 3, 10), (1, 4, 5),
02             (1, 6, 3), (2, 3, 9), (2, 4, 7), (2, 5, 2),
03             (3, 5, 4), (3, 6, 8), (4, 6, 1), (5, 6, 6)]
04 weights.sort(key = lambda t: t[2])
05 mst = []
06 N = 7
07 p = [0] * N
08 for i in range(N):
09     p.append(i)
10
11 def find(u):
12     if u != p[u]:
13         p[u] = find(p[u])
14     return p[u]
15
16 def union(u, v):
17     root1 = find(u)
18     root2 = find(v)
19     p[root2] = root1
20

```

입력 그래프
(간선의 두 정점, 가중치)

가중치로 간선 정렬

상호 배타적 집합

각 정점 자신이 집합의 대표(루트)

find 연산

경로압축

union 연산

임의로 root2가
root1의 부모가 됨

Kruskal의 MST 알고리즘

```
21 tree_edges = 0
22 mst_cost = 0
23 while True:
24     if tree_edges == N-1:
25         break
26     u, v, wt = weights.pop(0)
27     if find(u) != find(v):
28         union(u, v)
29         mst.append((u, v))
30         mst_cost += wt
31         tree_edges += 1
32
33 print('최소신장트리: ', end=' ')
34 print(mst)
35 print('최소신장트리 가중치: ', mst_cost)
```

다음 최소 가중치를
가진 간선 가져오기

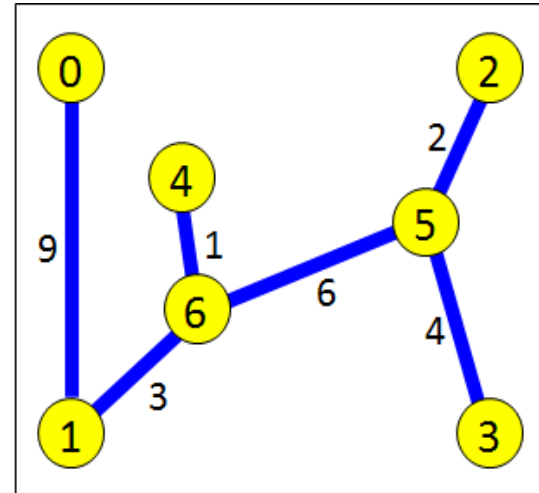
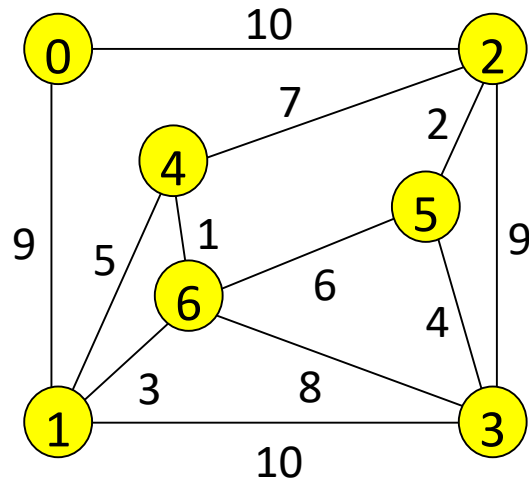
u와 v가 서로 다른
집합에 속해 있으면

트리에 (u, v) 추가

Kruskal의 MST 알고리즘 수행결과

최소신장트리: [(4, 6), (2, 5), (1, 6), (3, 5), (5, 6), (0, 1)]

최소신장트리 가중치: 25



Kruskal의 MST 알고리즘

```
def MSTKruskal(vertex, adj):  
    vsize = len(vertex)  
    init_set(vsize)  
    eList = []  
  
    for i in range(vsize-1):  
        for j in range(i+1, vsize):  
            if adj[i][j] != None:  
                eList.append( (i,j,adj[i][j]) )  
  
    # 간선 리스트를 가중치의 내림차순으로 정렬: 람다 함수  
    eList.sort(key= lambda e : e[2], reverse=True)
```

람다 함수 사용

```
    edgeAccepted = 0  
    while (edgeAccepted < vsize - 1):  
        e = eList.pop()  
        uSet = find(e[0])  
        vSet = find(e[1])  
  
        if uSet != vSet :  
            print("간선 추가 : (%s, %s, %d)" %  
                  (vertex[e[0]], vertex[e[1]], e[2]))  
            union(uSet, vSet)  
            edgeAccepted += 1
```

Kruskal의 MST 알고리즘 테스트

```

vertex = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
weight = [
    [None, 29, None, None, None, 10, None],
    [29, None, 16, None, None, None, 15],
    [None, 16, None, 12, None, None, None],
    [None, None, 12, None, 22, None, 18],
    [None, None, None, 22, None, 27, 25],
    [10, None, None, None, 27, None, None],
    [None, 15, None, 18, 25, None, None]
]
    
```

```

print("MST By Kruskal's Algorithm")
MSTKruskal(vertex, weight)
    
```

```

C:\WINDOWS\system32\cmd.exe
MST By Kruskal's Algorithm
간선 추가 : (A, F, 10)
간선 추가 : (C, D, 12)
간선 추가 : (B, G, 15)
간선 추가 : (B, C, 16)
간선 추가 : (D, E, 22)
간선 추가 : (E, F, 27)
    
```

Prim의 MST 알고리즘

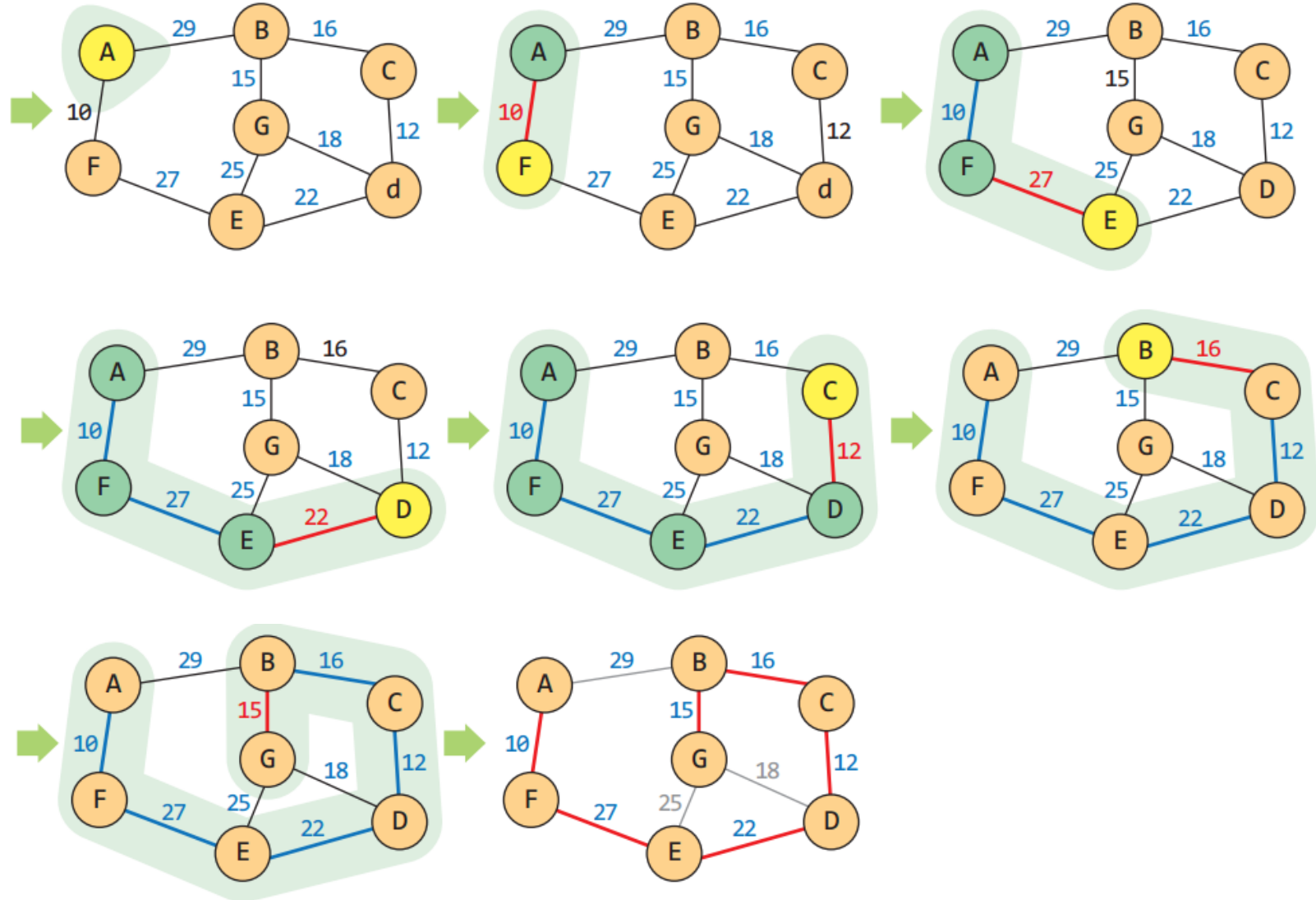
- 하나의 정점에서부터 시작하여 트리를 단계적으로 확장
- 현재의 신장 트리 집합에 인접한 정점 중 최저 간선으로 연결된 정점 선택하여 신장 트리 집합에 추가
- 이 과정을 $n-1$ 개의 간선을 가질 때까지 반복
- 알고리즘
 1. 그래프에서 시작 정점을 선택하여 초기 트리를 만듦
 2. 현재 트리의 정점들과 인접한 정점들 중에서 간선의 가중치가 가장 작은 정점 v 를 선택
 3. 이 정점 v 와 이때의 간선을 트리에 추가
 4. 모든 정점이 삽입될 때 까지 2번으로 이동

Prim의 MST 알고리즘

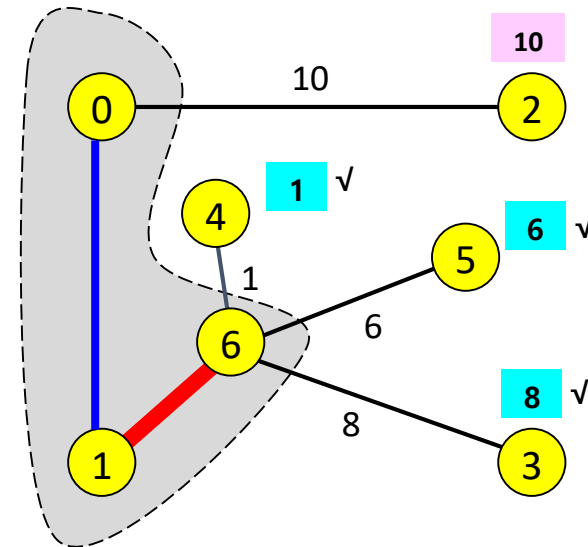
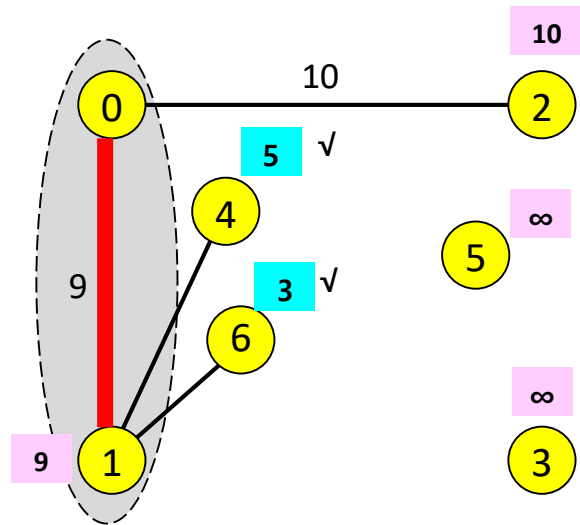
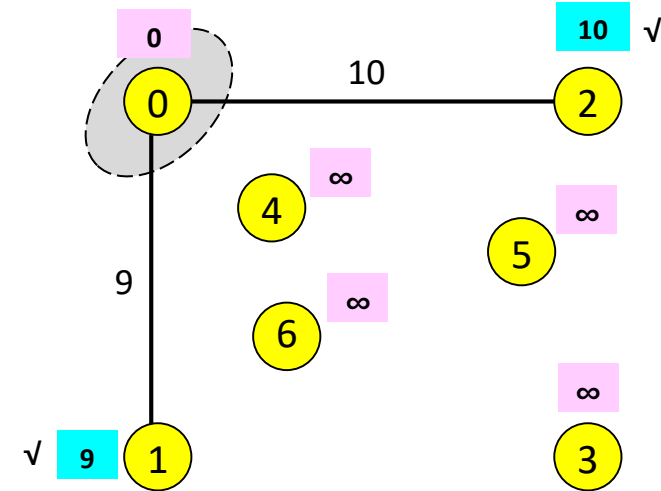
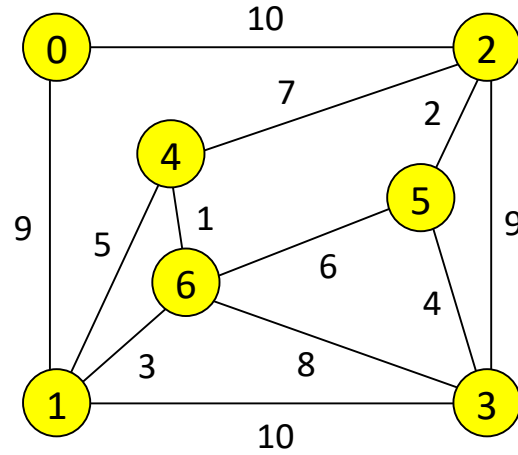
- 하나의 정점에서부터 시작하여 트리를 단계적으로 확장
- 현재의 신장 트리 집합에 인접한 정점 중 최저 간선으로 연결된 정점 선택하여 신장 트리 집합에 추가
- 이 과정을 $n-1$ 개의 간선을 가질 때까지 반복
- 알고리즘
 1. 그래프에서 시작 정점을 선택하여 초기 트리를 만듦
 2. 현재 트리의 정점들과 인접한 정점들 중에서 간선의 가중치가 가장 작은 정점 v 를 선택
 3. 이 정점 v 와 이때의 간선을 트리에 추가
 4. 모든 정점이 삽입될 때 까지 2번으로 이동

Graph

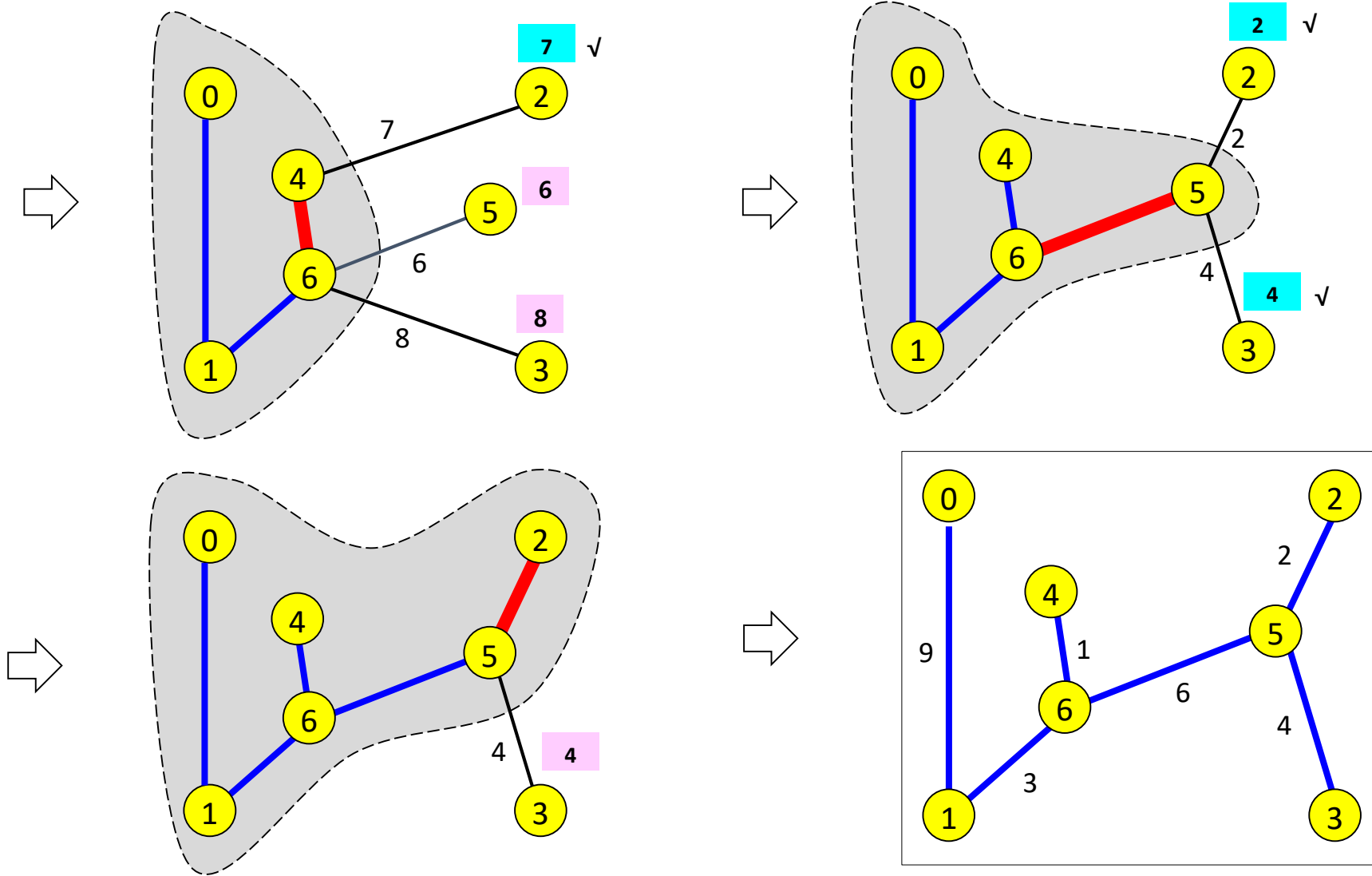
Prim의 MST 알고리즘 처리 과정



Prim의 MST 알고리즘 예제



Prim의 MST 알고리즘 예제



Prim의 MST 알고리즘

```

01 import sys
02 N = 7
03 s = 0
04 g = [None] * N
05 g[0] = [(1, 9), (2, 10)]
06 g[1] = [(0, 9), (3, 10), (4, 5), (6, 3)]
07 g[2] = [(0, 10), (3, 9), (4, 7), (5, 2)]
08 g[3] = [(1, 10), (2, 9), (5, 4), (6, 8)]
09 g[4] = [(1, 5), (2, 7), (6, 1)]
10 g[5] = [(2, 2), (3, 4), (6, 6)]
11 g[6] = [(1, 3), (3, 8), (4, 1), (5, 6)]
12
13 visited = [False] * N
14 D = [sys.maxsize] * N
15 D[s] = 0
16 previous = [None] * N
17 previous[s] = s
18

```

sys.maxsize(최댓값) 사용 위해

입력 그래프의
인접리스트

각 원소를 최댓값으로

초기화

트리 간선 추출을 위해

Prim의 MST 알고리즘

```

19 for k in range(N):
20     m = -1
21     min_value = sys.maxsize
22     for j in range(N):
23         if not visited[j] and D[j] < min_value:
24             min_value = D[j]
25             m = j
26     visited[m] = True
27
28     for w, wt in list(g[m]):
29         if not visited[w]:
30             if wt < D[w]:
31                 D[w] = wt
32                 previous[w] = m
33
34 print('최소신장트리: ', end=' ')
35 mst_cost = 0
36 for i in range(1,N):
37     print('(%d, %d) '% (i, previous[i]), end=' ')
38     mst_cost += D[i]
39 print('\n최소신장트리 가중치: ', mst_cost)
    
```

m = min_vertex

방문 안된 정점들의 D
원소들 중에서 최솟값을
가진 정점 m 찾기

정점 m에 인접한 정점 w와
간선 (m, w)의 가중치 wt에 대해

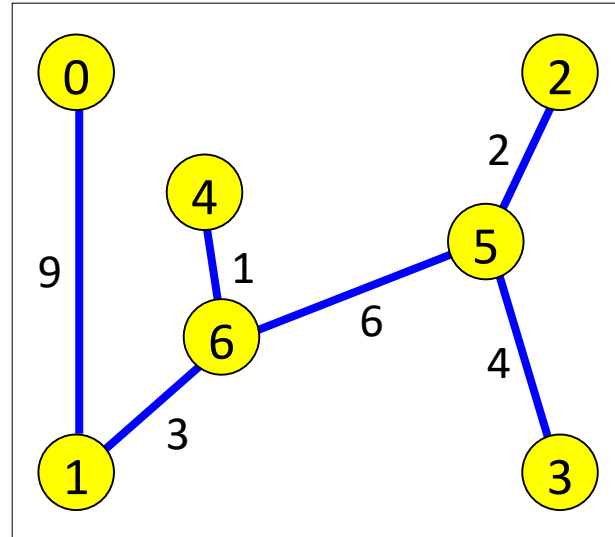
D[w] 갱신

D[w]가 정점 m 때문에
갱신되었음을 기록

Prim의 MST 알고리즘 수행 결과

최소신장트리: (1, 0)(2, 5)(3, 5)(4, 6)(5, 6)(6, 1)

최소신장트리 가중치: 25



Prim의 알고리즘

```
def MSTPrim(vertex, adj) :  
    vsize = len(vertex)  
    dist = [INF] * vsize  
    selected = [False] * vsize  
    dist[0] = 0
```

```
    for i in range(vsize) :
```

```
        u = getMinVertex(dist, selected)  
        selected[u] = True  
        print(vertex[u], end=' ')
```

```
    for v in range(vsize) :
```

```
        if (adj[u][v] != None):  
            if selected[v]==False and adj[u][v]< dist[v] :  
                dist[v] = adj[u][v]
```

```
    print()
```

```
INF = 9999      # 가장 큰 가중치 (무한대)
```

```
def getMinVertex(dist, selected) :
```

```
    minv = 0
```

```
    mindist = INF
```

```
    for v in range(len(dist)) :
```

```
        if not selected[v] and dist[v]<mindist :
```

```
            mindist = dist[v]
```

```
            minv = v
```

```
    return minv
```

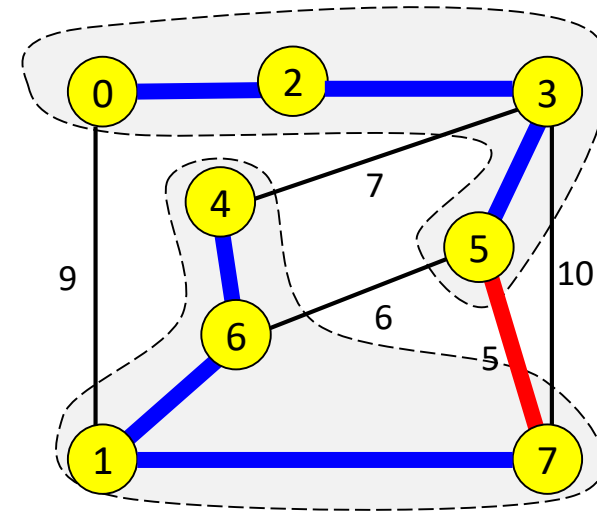
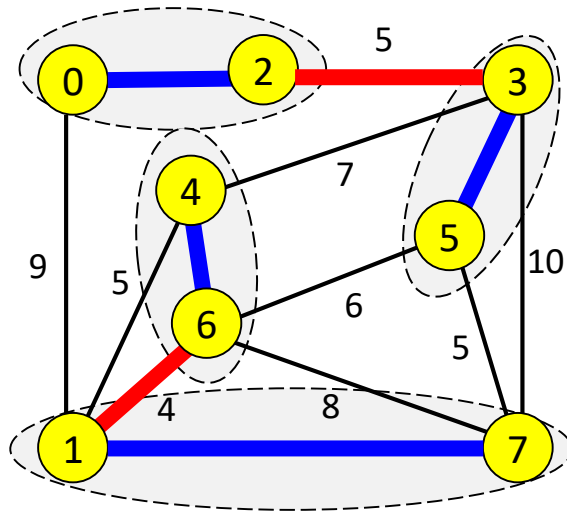
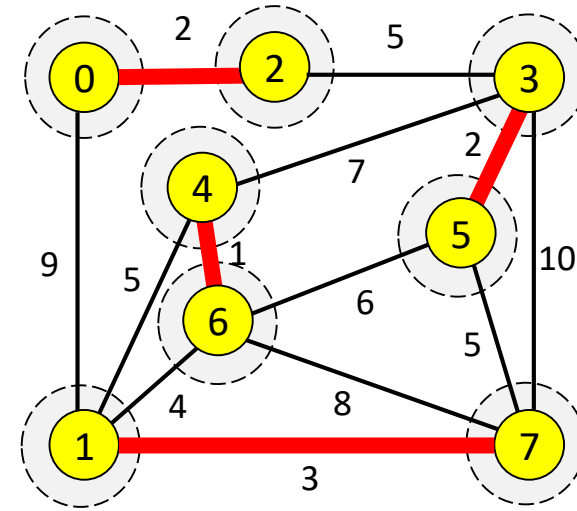
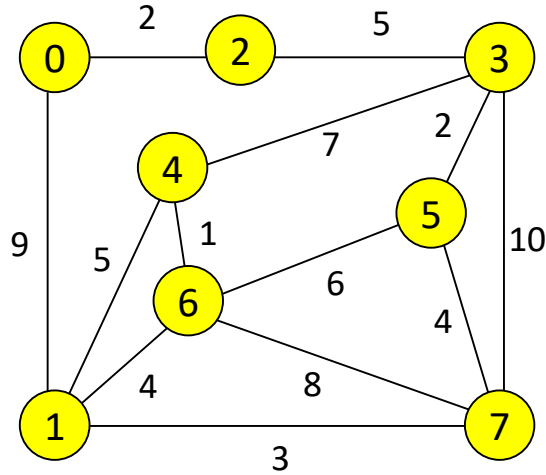
시간 복잡도 비교

- Kruskal 알고리즘: $O(e \log e)$
 - 대부분 간선들을 정렬하는 시간에 좌우됨
 - 간선 e 개를 정렬하는 시간
- Prim의 알고리즘: $O(n^2)$
 - 주 반복문이 n 번, 내부 반복문이 n 번 반복
- 희박한 그래프
 - $O(e \log e)$ 인 Kruskal이 유리
- 밀집한 그래프: 예) 완전그래프
 - $O(n^2)$ 인 Prim이 유리

Sollin 알고리즘

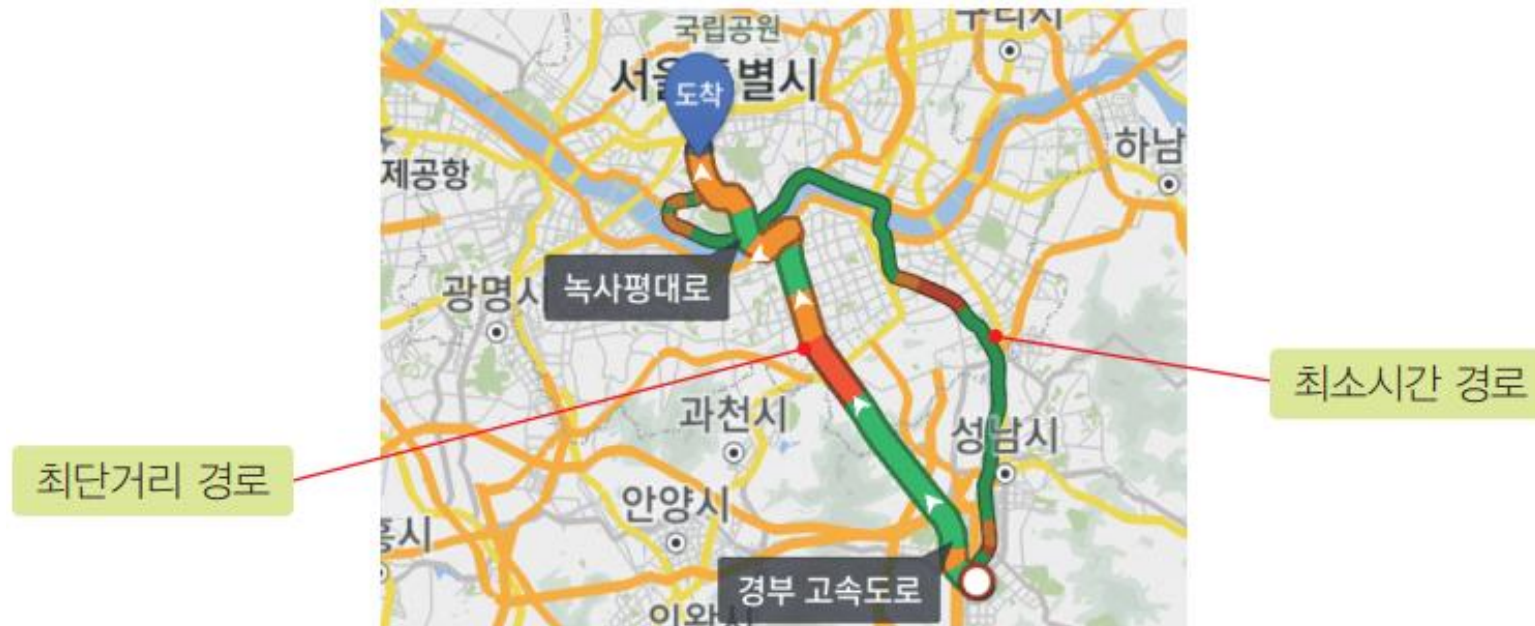
- Sollin 알고리즘은 각 정점을 독립적인 트리로 간주하고, 각 트리에 연결된 간선들 중에서 가장 작은 가중치를 가진 간선을 선택한다. 이때 선택된 간선은 2 개의 트리를 1개의 트리로 만든다.
- 같은 방법으로 한 개의 트리가 남을 때까지 각 트리에서 최소 가중치 간선을 선택하여 연결
- Sollin 알고리즘은 병렬알고리즘(Parallel Algorithm)으로 구현이 쉽다는 장점을 가짐
- 알고리즘
 - [1] 각 정점은 독립적인 트리이다.
 - [2] repeat
 - [3] 각 트리에 닿아 있는 간선들 중에서 가중치가 가장 작은 간선을 선택하여 트리를 합친다.
 - [4] until (1개의 트리만 남을 때까지)

Sollin 알고리즘 예제



최단경로

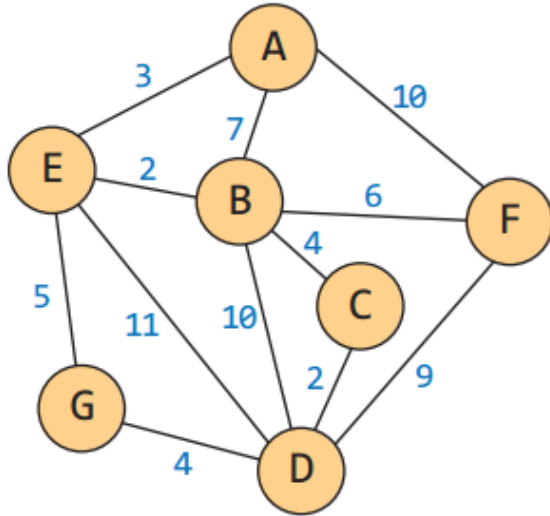
- 두 정점을 연결하는 경로들 중에서 간선의 가중치 합이 최소가 되는 경로
- 간선의 가중치는 비용, 거리, 시간 등



- Dijkstra, Floyd 알고리즘

최단 경로를 위한 그래프 표현

- 간선이 없으면 가중치를 무한대로 처리
- 대각선 요소는 0



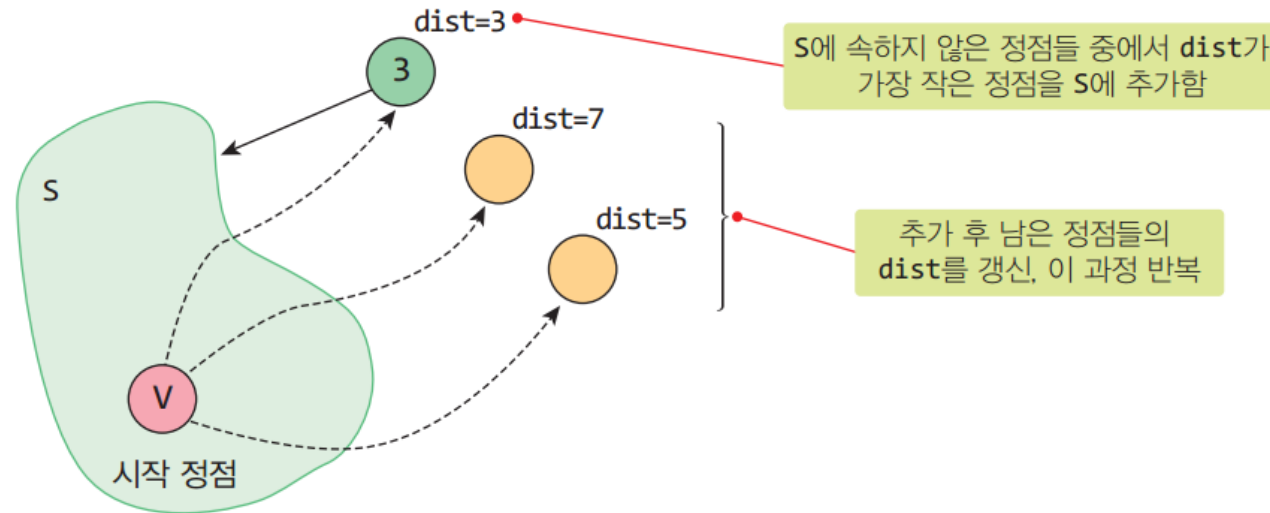
인접 행렬

	A	B	C	D	E	F	G
A	0	7	∞	∞	3	10	∞
B	7	0	4	10	2	6	∞
C	∞	4	0	2	∞	∞	∞
D	∞	10	2	0	11	9	4
E	3	2	∞	11	0	∞	5
F	10	6	∞	9	∞	0	∞
G	∞	∞	∞	4	5	∞	0

간선이 없으면 무한대(∞)

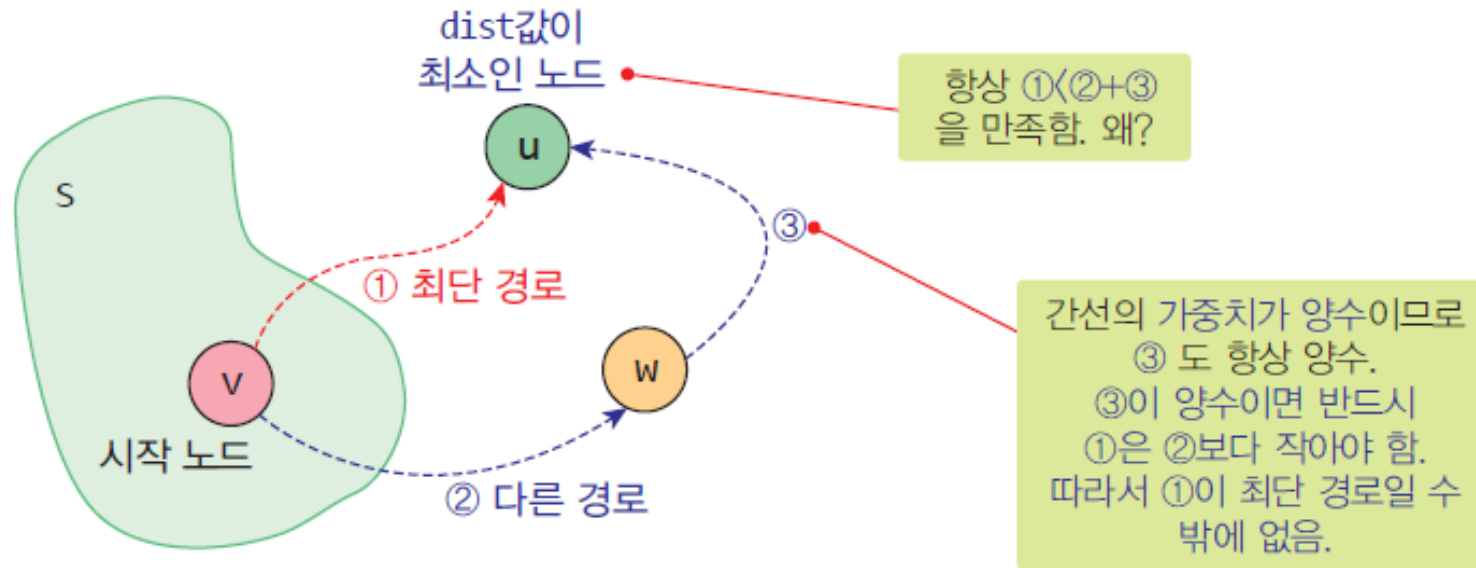
Dijkstra 알고리즘

- Dijkstra 알고리즘: 출발점으로부터 각 정점까지의 최단거리 및 경로를 계산
- Dijkstra 알고리즘은 Prim의 MST 알고리즘과 매우 유사
- 차이점
 - ✓ Dijkstra 알고리즘은 출발점이 주어지지만 Prim 알고리즘에서는 출발점이 주어지지 않는다는 것
 - ✓ Prim 알고리즘에서는 D의 원소에 간선의 가중치가 저장되지만, Dijkstra 알고리즘에서는 D의 원소에 출발점으로부터 각 정점까지의 경로의 길이가 저장됨
- 시작 정점 v 에서 모든 다른 정점까지의 최단 경로 찾기



Dijkstra 알고리즘

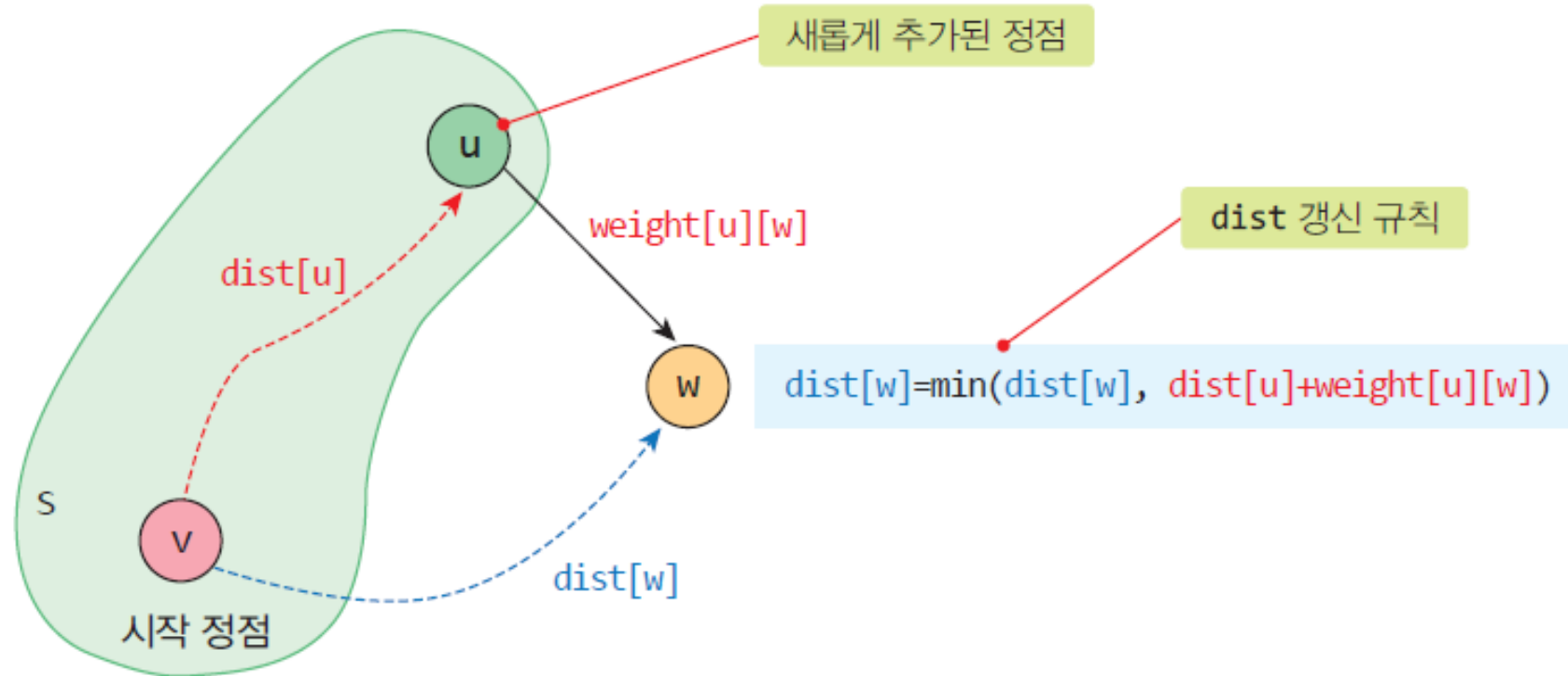
- 최단 경로 증명
- 이렇게 해도 최단 경로가 구해질까?



- 모든 간선의 가중치가 양수라면 최단 경로가 구해짐
- Dijkstra 알고리즘은 입력그래프에 음수 가중치가 있으면 최단 경로 찾기에 실패하는 경우가 발생

Dijkstra 알고리즘

- dist 갱신 규칙
- 새로운 정점이 S에 추가되면 dist 갱신



Dijkstra의 최단경로 알고리즘

```

shortest_path_dijkstra(v)
    S ← {v}
    for 각 정점 w ∈ G do
        dist[w] ← weight[v][w]
    while 모든 정점이 S에 포함되지 않으면 do
        u ← 집합 S에 속하지 않는 정점 중에서 최소 dist 정점
        S ← S ∪ {u}
        for u에 인접하고 S에 있지 않은 모든 정점 w do
            if dist[u] + weight[u][w] < dist[w]
                then dist[w] ← dist[u] + weight[u][w]
    
```

- 알고리즘 결과
 - 경로의 길이 dist를 구함
 - 경로 자체를 구하려면?
 - ✓ path 처리 추가

```

def shortest_path_dijkstra(vtx, adj, start) :
    vsize = len(vtx)
    dist = list(adj[start])
    path = [start] * vsize
    found = [False] * vsize
    found[start] = True
    dist[start] = 0

    for i in range(vsize) :
        print("Step%2d: "%(i+1), dist)
        u = choose_vertex(dist, found)
        found[u] = True

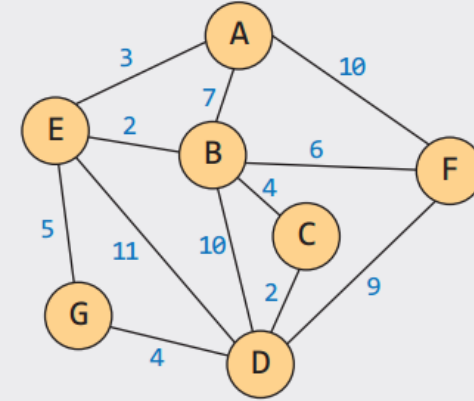
        for w in range(vsize) :
            if not found[w] :
                if dist[u] + adj[u][w] < dist[w] :
                    dist[w] = dist[u] + adj[u][w]
                    path[w] = u

    return path
    
```

Dijkstra의 최단경로 알고리즘 테스트

```
print("Shortest Path By Dijkstra Algorithm")
start = 0      # 시작 정점은 0번, 'A'로 선택
path = shortest_path_dijkstra(vertex, weight, start)

# 최종 경로를 출력하기 위한 코드
for end in range(len(vertex)) :
    if end != start :
        print("[최단경로: %s->%s] %s" %
              (vertex[start], vertex[end], vertex[end]), end='')
        while (path[end] != start) :
            print(" <- %s" % vertex[path[end]], end='')
            end = path[end]
        print(" <- %s" % vertex[path[end]])
```



```
C:\WINDOWS\system32\cmd.exe

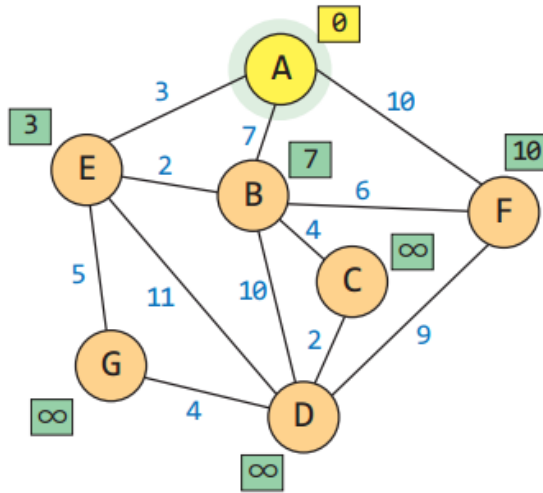
Shortest Path By Dijkstra Algorithm
Step 1: [0, 7, 999, 999, 3, 10, 999]
Step 2: [0, 5, 999, 14, 3, 10, 8]
Step 3: [0, 5, 9, 14, 3, 10, 8]
Step 4: [0, 5, 9, 12, 3, 10, 8]
Step 5: [0, 5, 9, 11, 3, 10, 8]
Step 6: [0, 5, 9, 11, 3, 10, 8]
Step 7: [0, 5, 9, 11, 3, 10, 8]
[최단경로: A->B] B <- E <- A
[최단경로: A->C] C <- B <- E <- A
[최단경로: A->D] D <- C <- B <- E <- A
[최단경로: A->E] E <- A
[최단경로: A->F] F <- A
[최단경로: A->G] G <- E <- A
```

각 단계별 dist[] 배열의 값 변화

A부터 모든 정점까지의 최단 경로

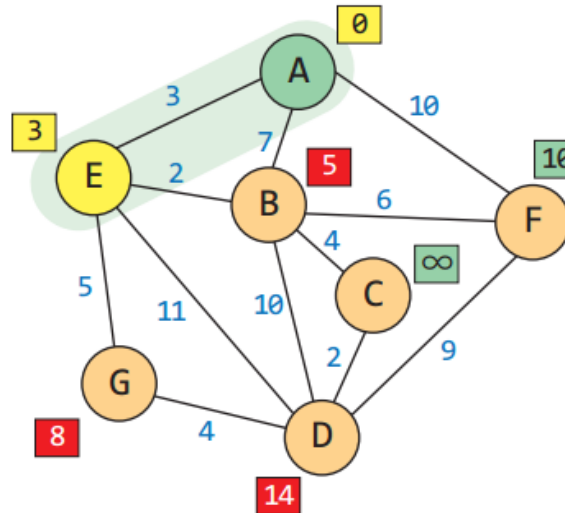
Dijkstra 알고리즘

- 알고리즘 실행 과정: Step1-2



$S = \{ A \}$

$\text{dist}(A) = w(A, A) = 0$
 $\text{dist}(B) = w(A, B) = 7$
 $\text{dist}(C) = w(A, C) = \infty$
 $\text{dist}(D) = w(A, D) = \infty$
 $\text{dist}(E) = w(A, E) = 3$
 $\text{dist}(F) = w(A, F) = 10$
 $\text{dist}(G) = w(A, G) = \infty$

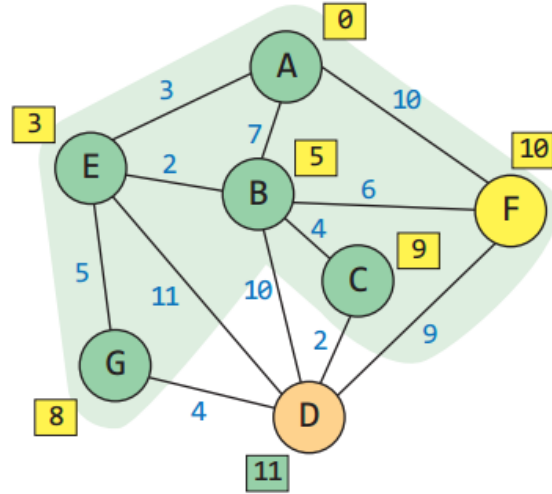


$S = \{ A, E \}$

$\text{dist}(A) = 0$
 $\text{dist}(E) = w(A, E) = 3$
 $\text{dist}(B) = \min(\text{dist}(B), \text{dist}(E) + w(E, B)) = \min(7, 3 + 2) = 5$
 $\text{dist}(C) = \min(\text{dist}(C), \text{dist}(E) + w(E, C)) = \min(\infty, 3 + \infty) = \infty$
 $\text{dist}(D) = \min(\text{dist}(D), \text{dist}(E) + w(E, D)) = \min(\infty, 3 + 11) = 14$
 $\text{dist}(F) = \min(\text{dist}(F), \text{dist}(E) + w(E, F)) = \min(10, 3 + \infty) = 10$
 $\text{dist}(G) = \min(\text{dist}(G), \text{dist}(E) + w(E, G)) = \min(\infty, 3 + 5) = 8$

Dijkstra 알고리즘

- 알고리즘 실행 과정: Step6-최종



$S = \{ A, E, B, G, C, F \}$

$\text{dist}(A)=0$

$\text{dist}(B)=5$

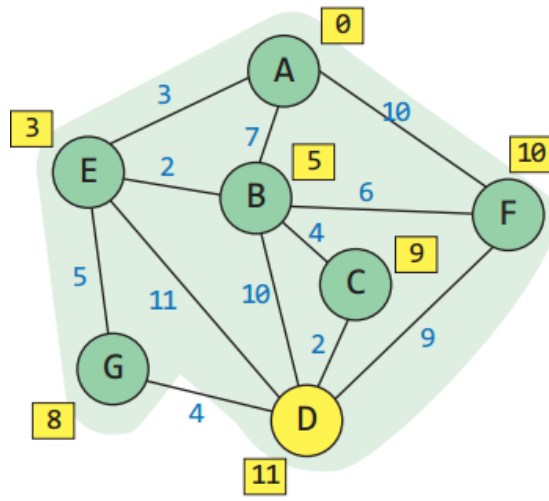
$\text{dist}(C)=9$

$\text{dist}(E)=3$

$\text{dist}(F)=10$

$\text{dist}(G)=8$

$\text{dist}(D)=\min(\text{dist}(D), \text{dist}(F)+w(F,D))=\min(11, 10+9)=11$



$S = \{ A, E, B, G, C, F, D \}$

$\text{dist}(A)=0$

$\text{dist}(B)=5$

$\text{dist}(C)=9$

$\text{dist}(D)=11$

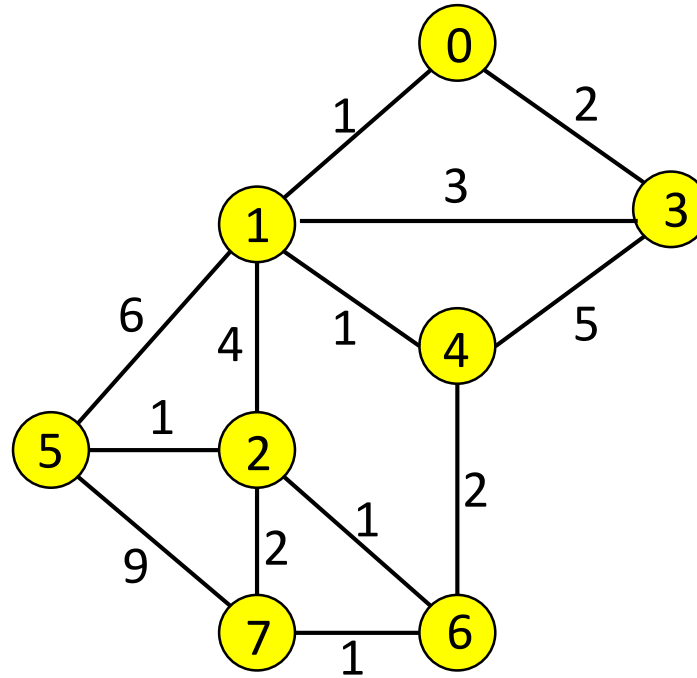
$\text{dist}(E)=3$

$\text{dist}(F)=10$

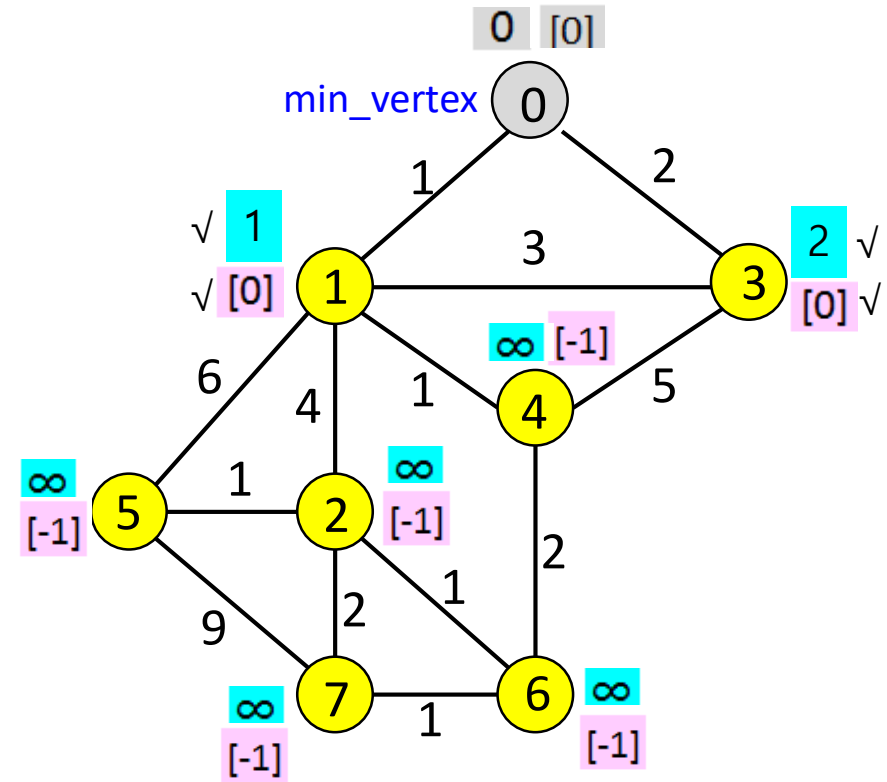
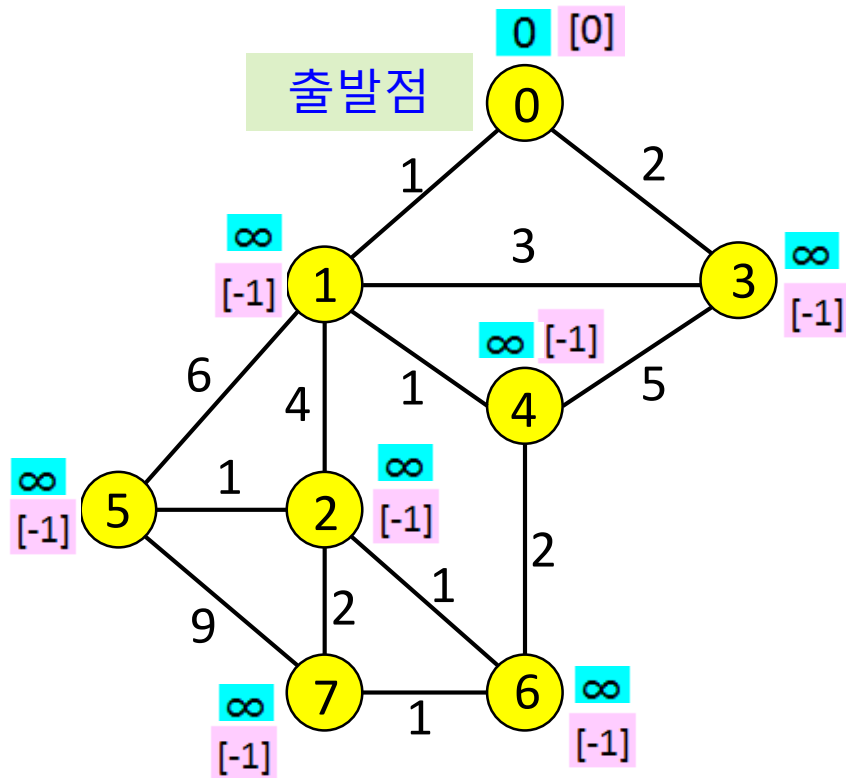
$\text{dist}(G)=8$

Graph

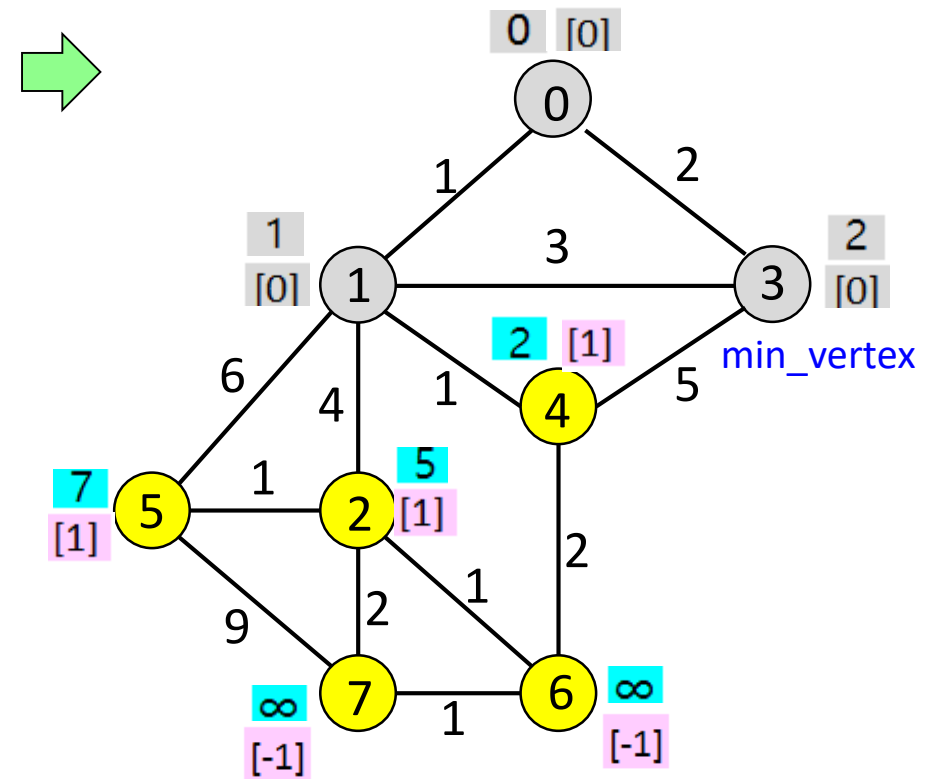
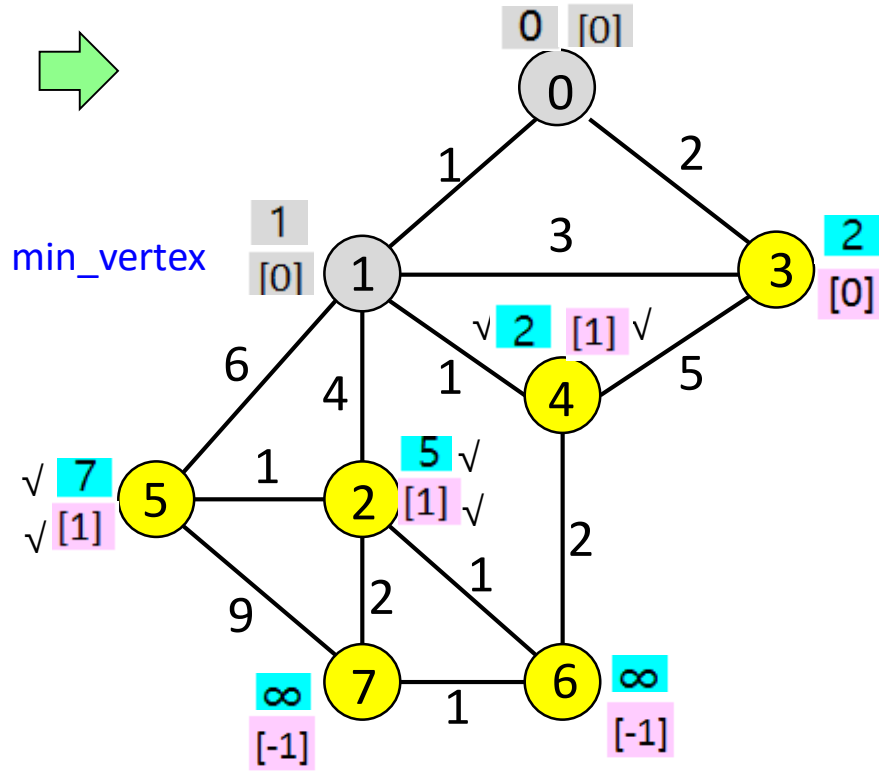
Dijkstra 알고리즘 예제



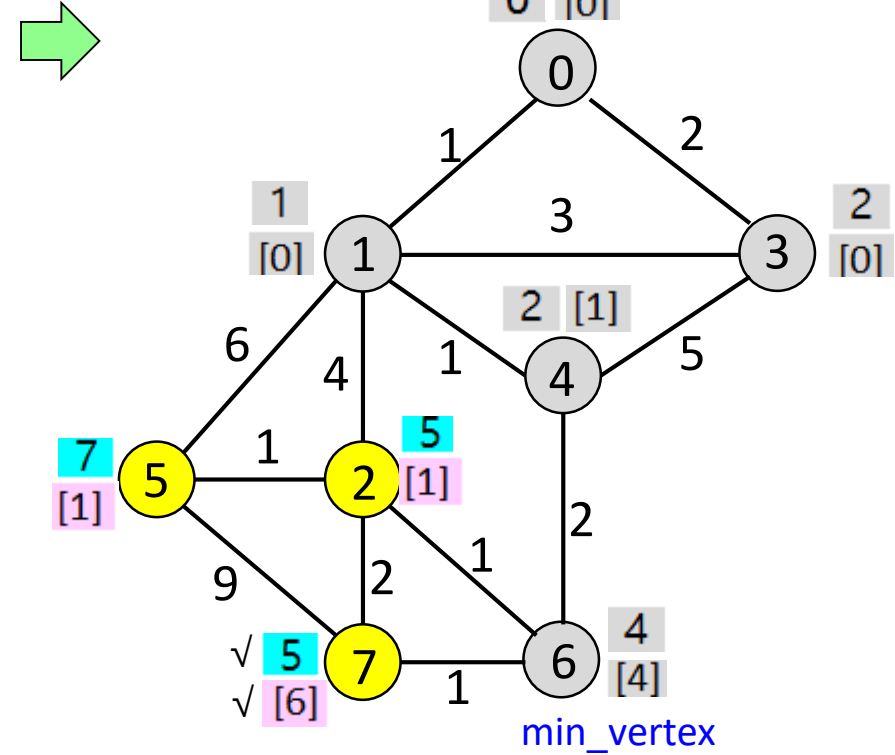
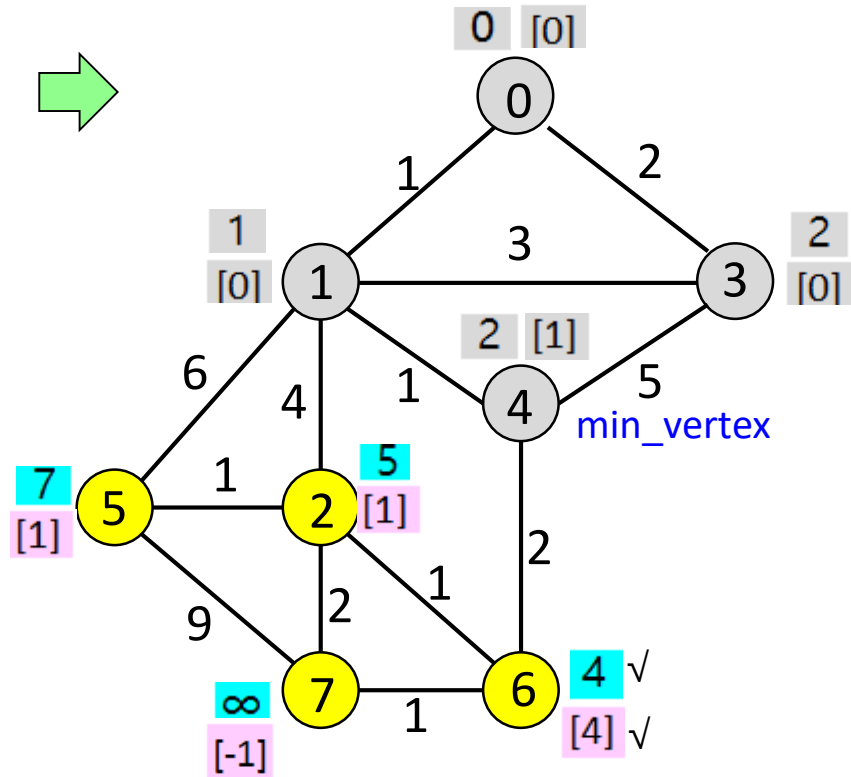
Dijkstra 알고리즘 예제



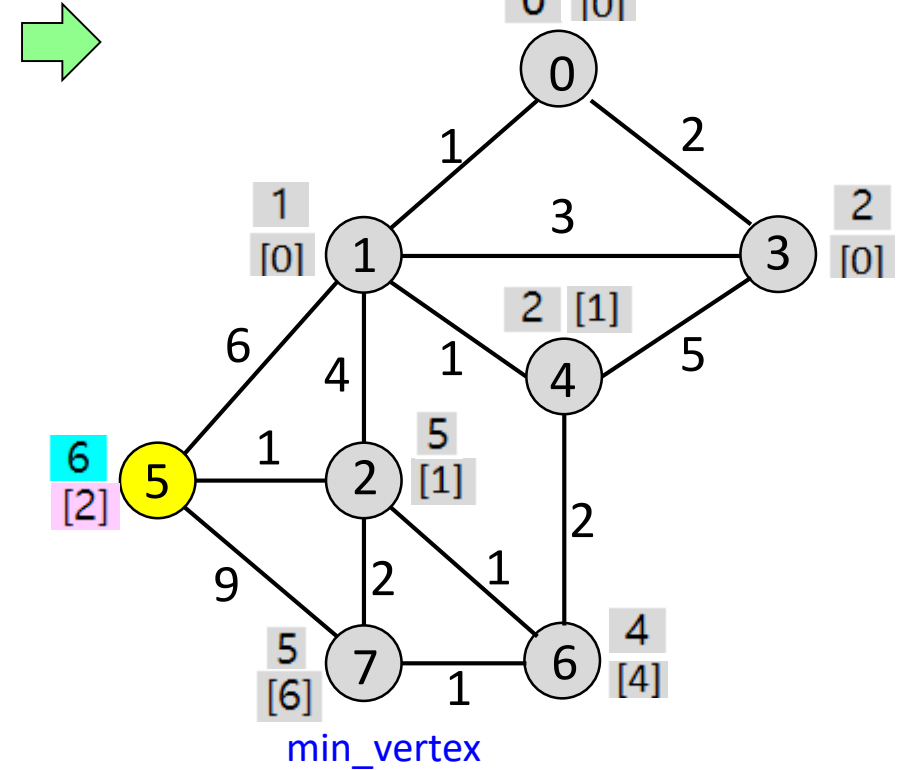
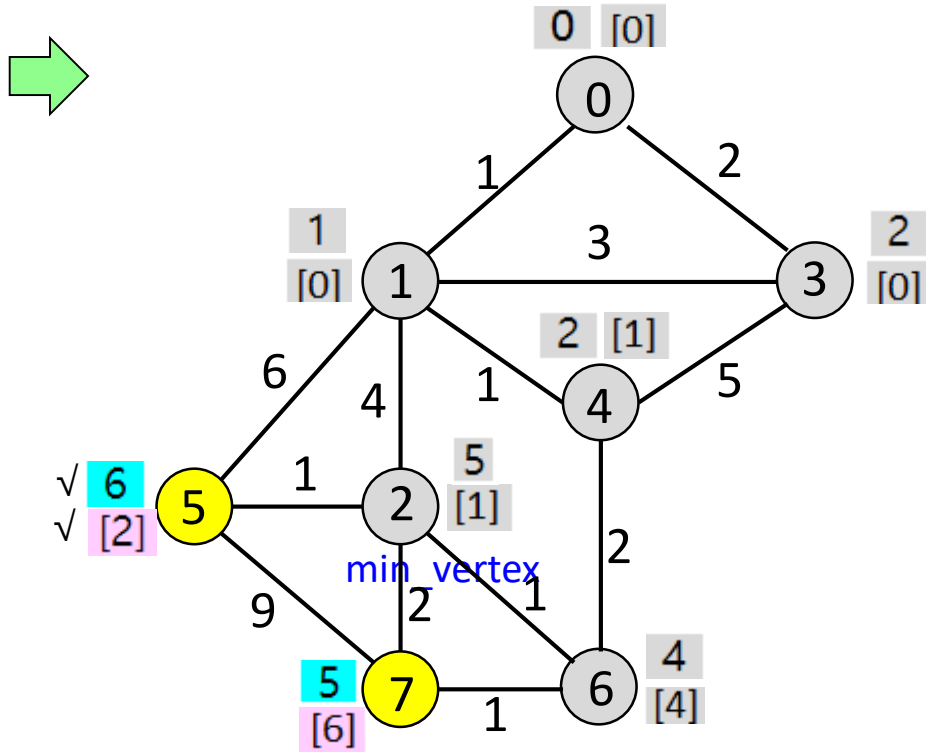
Dijkstra 알고리즘 예제



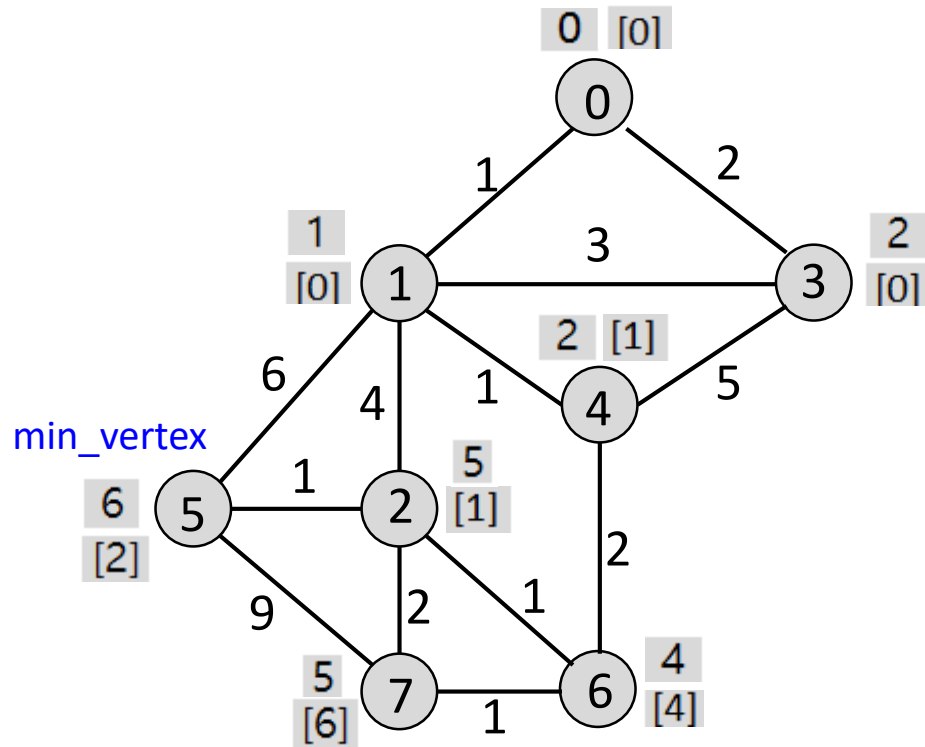
Dijkstra 알고리즘 예제



Dijkstra 알고리즘 예제



Dijkstra 알고리즘 예제



정점 0으로부터의 최단 거리

$[0, 1] = 1$
 $[0, 2] = 5$
 $[0, 3] = 2$
 $[0, 4] = 2$
 $[0, 5] = 6$
 $[0, 6] = 4$
 $[0, 7] = 5$

정점 0으로부터의 최단 경로

$1 < -0$
 $2 < -1 < -0$
 $3 < -0$
 $4 < -1 < -0$
 $5 < -2 < -1 < -0$
 $6 < -4 < -1 < -0$
 $7 < -6 < -4 < -1 < -0$

Dijkstra 알고리즘

```

01 import sys
02 N = 8
03 s = 0
04 g = [None] * N
05 g[0] = [(1, 1), (3, 2)]
06 g[1] = [(0, 1), (2, 4), (3, 3), (4, 1), (5, 6)]
07 g[2] = [(1, 4), (5, 1), (6, 1), (7, 2)]
08 g[3] = [(0, 2), (1, 3), (4, 5)]
09 g[4] = [(1, 1), (3, 5), (6, 2)]
10 g[5] = [(1, 6), (2, 1), (7, 9)]
11 g[6] = [(2, 1), (4, 2), (7, 1)]
12 g[7] = [(2, 2), (5, 9), (6, 1)]
13
14 visited = [False] * N
15 D = [sys.maxsize] * N
16 D[s] = 0
17 previous = [None] * N
18 previous[s] = s
19
    
```

sys.maxsize(최댓값) 사용 위해

입력 그래프의
인접리스트

각 원소를 최댓값으로

초기화

최단경로 추출을 위해

Dijkstra 알고리즘

```

20 for k in range(N):
21     m = -1
22     min_value = sys.maxsize
23     for j in range(N):
24         if not visited[j] and D[j] < min_value:
25             min_value = D[j]
26             m = j
27     visited[m] = True
28     for v, wt in list(g[m]):
29         if not visited[v]:
30             if D[m]+wt < D[v]:
31                 D[v] = D[m] + wt
32                 previous[v] = m
33 
```

`m = min_vertex`

방문 안된 정점들의 D 원소들 중에서 최솟값을 가진 정점 m 찾기

정점 m에 인접한 v와 (m, v)의 가중치 wt에 대해

D[v] 갱신: 간선완화

D[v]가 정점 m 때문에 갱신되었음을 기록

Dijkstra 알고리즘 예제

```
34 print('정점 ', s, '(으)로부터 최단거리:')
35 for i in range(N):
36     if D[i] == sys.maxsize:
37         print(s, '와(과) ', i, ' 사이에 경로 없음.')
38     else:
39         print('[%d, %d]' % (s, i), '=', D[i])
40
41 print('\n정점 ', s, '(으)로부터의 최단 경로')
42 for i in range(N):
43     back = i
44     print(back, end='')
45     while back != s:
46         print(' <-', previous[back], end='')
47         back = previous[back]
48     print()
```

Floyd의 최단경로 알고리즘

- Floyd-Warshall 알고리즘은 모든 정점 쌍 사이의 최단 경로 계산
- 모든 쌍 최단경로(All Pairs Shortest Paths) 알고리즘으로도 불린다.
- 지도에서 도시간 거리를 계산한 표를 볼 수 있는데, Floyd-Warshall 알고리즘을 사용하면 얻을 수 있음
- Floyd-Warshall 알고리즘은 모든 정점 쌍 사이의 최단 경로 계산
- 모든 쌍 최단경로(All Pairs Shortest Paths) 알고리즘으로도 불린다
- Floyd-Warshall 알고리즘은 Dijkstra 알고리즘에 비해 훨씬 알고리즘이 간단하고, 음수 가중치 그래프에서도 최단 경로를 찾을 수 있다는 장점을 가짐
- 지도에서 도시간 거리를 계산한 표를 볼 수 있는데, Floyd-Warshall 알고리즘을 사용하면 얻을 수 있음

Floyd의 최단경로 알고리즘

- Floyd-Warshall의 최단 경로 알고리즘
 - 모든 정점 사이의 최단경로 길이를 찾음
 - 2차원 배열 A를 이용하여 3중 반복을 하는 루프로 구성
 - 배열 A의 초기 값은 인접 행렬의 가중치
- 알고리즘

```
shortest_path_floyd()
```

```
for k ← 0 to n - 1
```

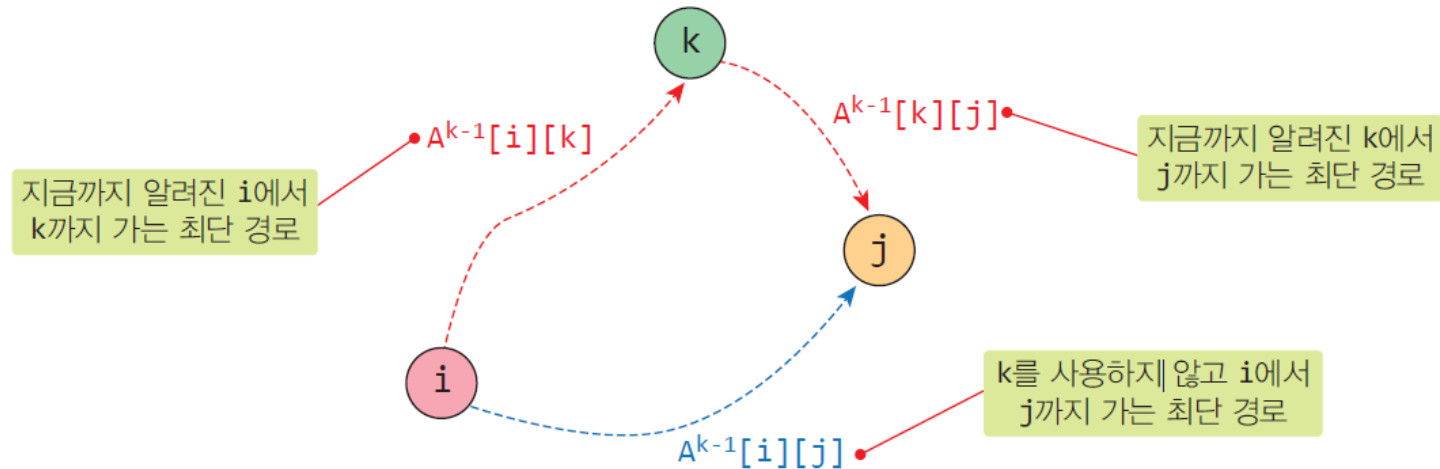
```
for i ← 0 to n - 1
```

```
for j ← 0 to n - 1
```

```
A[i][j] = min(A[i][j], A[i][k] + A[k][j])
```

Floyd의 최단경로 알고리즘

- 아이디어
- $A^k[i][j]$
 - 0~k까지의 정점만을 이용한 정점 i에서 j까지의 최단 경로 길이
- $A^{-1} \rightarrow A^0 \rightarrow A^1 \rightarrow \dots \rightarrow A^{n-1}$ 순으로 최단경로길이를 구함



k번 정점을
사용한다면

$$A^k[i][j] = \min(A^{k-1}[i][j], A^{k-1}[i][k] + A^{k-1}[k][j])$$

i에서 j까지 가는 최단 경로

지금까지 알고있던 경로

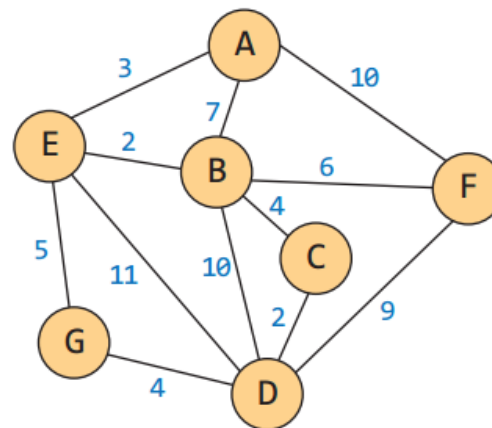
k를 거쳐서 가는 경로

Floyd의 최단경로 알고리즘

```
def shortest_path_floyd(vertex, adj) :  
    vsize = len(vertex)  
    A = list(adj)  
    for i in range(vsize) :  
        A[i] = list(adj[i])  
  
    for k in range(vsize) :  
        for i in range(vsize) :  
            for j in range(vsize) :  
                if (A[i][k] + A[k][j] < A[i][j]) :  
                    A[i][j] = A[i][k] + A[k][j]  
    printA(A)
```

Floyd의 최단경로 알고리즘

실행 결과



C:\WINDOWS\system32\cmd.exe

Shortest Path By Floyd's Algorithm

0	7	INF	INF	3	10	INF
7	0	4	10	2	6	INF
INF	4	0	2	INF	INF	INF
INF	10	2	0	11	9	4
3	2	INF	11	0	13	5
10	6	INF	9	13	0	INF
INF	INF	INF	4	5	INF	0

0	7	11	17	3	10	INF
7	0	4	10	2	6	INF
11	4	0	2	6	10	INF
17	10	2	0	11	9	4
3	2	6	11	0	8	5
10	6	10	9	8	0	INF
INF	INF	INF	4	5	INF	0

0	7	11	13	3	10	INF
7	0	4	6	2	6	INF
11	4	0	2	6	10	INF
13	6	2	0	8	9	4
3	2	6	8	0	8	5
10	6	10	9	8	0	INF
INF	INF	INF	4	5	INF	0

0	7	11	13	3	10	17
7	0	4	6	2	6	10
11	4	0	2	6	10	6
13	6	2	0	8	9	4
3	2	6	8	0	8	5
10	6	10	9	8	0	13
17	10	6	4	5	13	0

0	5	9	11	3	10	8
5	0	4	6	2	6	7
9	4	0	2	6	10	6
11	6	2	0	8	9	4
3	2	6	8	0	8	5
10	6	10	9	8	0	13
8	7	6	4	5	13	0

0	5	9	11	3	10	8
5	0	4	6	2	6	7
9	4	0	2	6	10	6
11	6	2	0	8	9	4
3	2	6	8	0	8	5
10	6	10	9	8	0	13
8	7	6	4	5	13	0

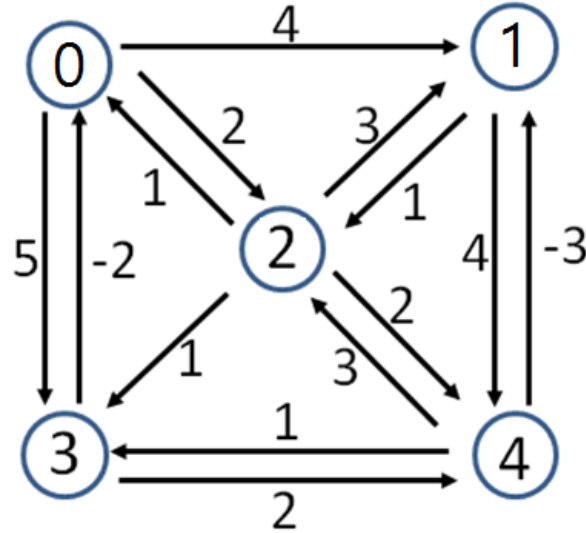
0	5	9	11	3	10	8
5	0	4	6	2	6	7
9	4	0	2	6	10	6
11	6	2	0	8	9	4
3	2	6	8	0	8	5
10	6	10	9	8	0	13
8	7	6	4	5	13	0

계속하려면 아무 키나 누르십시오 . . .

Dijkstra 알고리즘(시작정점0)의
결과와 동일

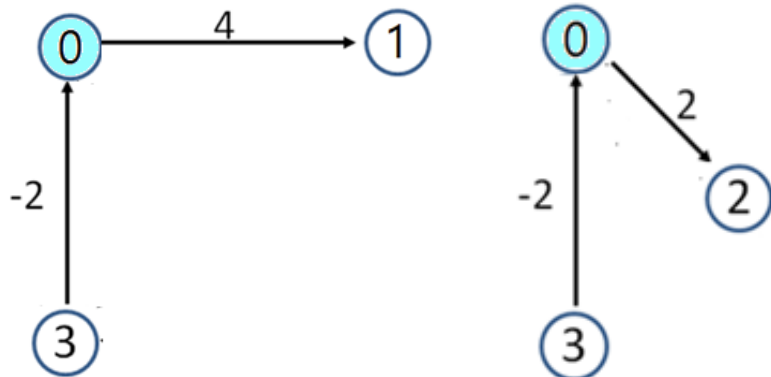
최종 A행렬

Floyd의 최단경로 예제



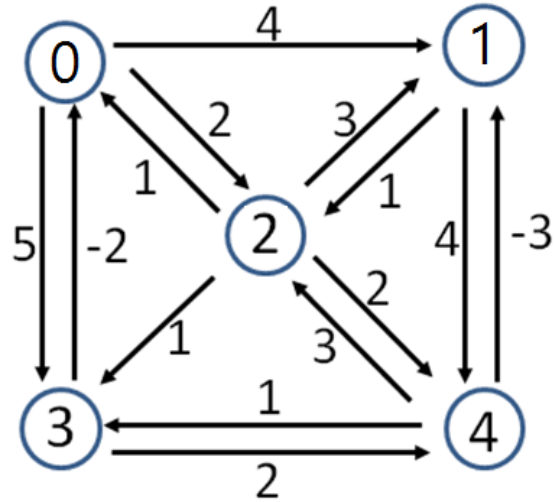
D	0	1	2	3	4
0	0	4	2	5	∞
1	∞	0	1	∞	4
2	1	3	0	1	2
3	-2	∞	∞	0	2
4	∞	-3	3	1	0

k=0

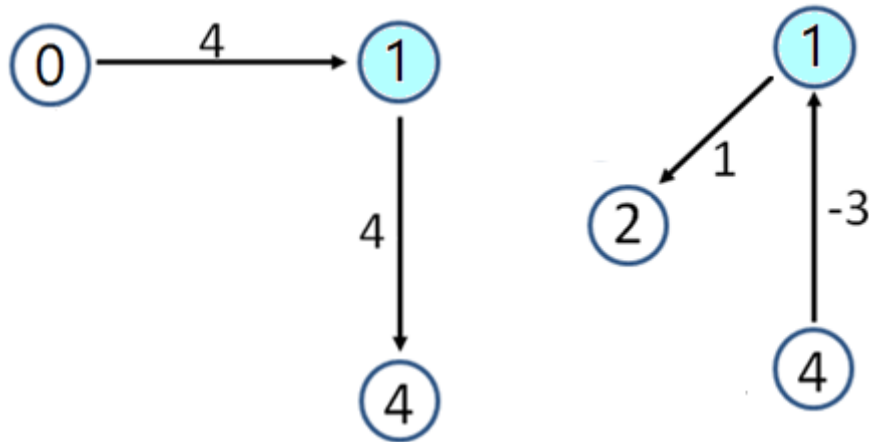


D	0	1	2	3	4
0	0	4	2	5	∞
1	∞	0	1	∞	4
2	1	3	0	1	2
3	-2	2	0	0	2
4	∞	-3	3	1	0

Floyd의 최단경로 예제



- $k=1$
 - $D[0,4] = 8$, due to $0 \rightarrow 1 \rightarrow 4$
 - $D[4,2] = -2$, due to $4 \rightarrow 1 \rightarrow 2$



D	0	1	2	3	4
0	0	4	2	5	8
1	∞	0	1	∞	4
2	1	3	0	1	2
3	-2	2	0	0	2
4	∞	-3	-2	1	0

Floyd의 최단경로 예제

• k=2

D	0	1	2	3	4
0	0	4	2	3	4
1	2	0	1	2	3
2	1	3	0	1	2
3	-2	2	0	0	2
4	-1	-3	-2	-1	0

• k=3

D	0	1	2	3	4
0	0	4	2	3	4
1	0	0	1	2	3
2	-1	3	0	1	2
3	-2	2	0	0	2
4	-3	-3	-2	-1	0

• k=4

D	0	1	2	3	4
0	0	1	2	3	4
1	0	0	1	2	3
2	-1	-1	0	1	2
3	-2	-1	0	0	2
4	-3	-3	-2	-1	0

Floyd의 최단경로 알고리즘

```
[1] D = 입력 그래프의 인접행렬
[2] for k in range(N):
[3]     for i in range(N):
[4]         for j in range(N):
[5]             if D[i][j] > D[i][k] + D[k][j]:
[6]                 D[i][j] = D[i][k] + D[k][j] # 간선완화
```

- 알고리즘의 step [1]에서 입력 그래프의 인접행렬을 모든 쌍 최단거리를 저장할 리스트 D에 복사한다.
- Step [2]의 for-루프는 경유하는 정점 ID를 0부터 N-1까지 변화시킨다.
- 모든 쌍 i와 j에 대하여 step [3]~[4]의 이중 for-루프가 i와 j를 각각 0부터 N-1까지 증가시키며, step [5]~[6]에서 간선완화를 수행한다.

Floyd의 최단경로 알고리즘

```

01 import sys
02 N = 5
03 INF = sys.maxsize
04 D = [[0, 4, 2, 5, INF], [INF, 0, 1, INF, 4],
05      [1, 3, 0, 1, 2], [-2, INF, INF, 0, 2],
06      [INF, -3, 3, 1, 0]]
07
08 for k in range(N):
09     for i in range(N):
10         for j in range(N):
11             D[i][j] = min(D[i][j], D[i][k]+D[k][j])
12
13 for i in range(N):
14     for j in range(N):
15         print('%3d' % D[i][j], end=' ')
16     print()
    
```

sys.maxsize(최댓값) 사용 위해

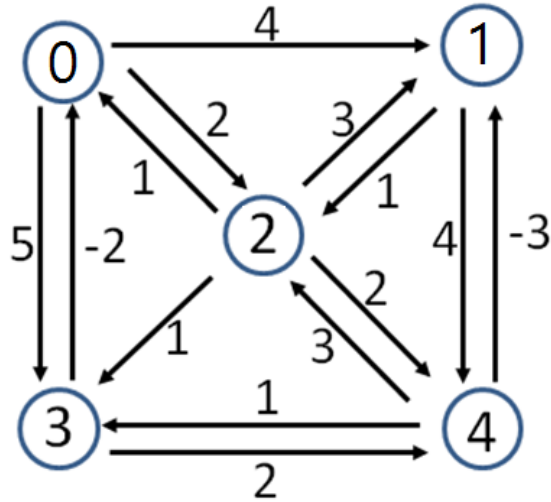
입력 그래프의 인접행렬

간선완화

최단거리 출력

Floyd의 최단경로 알고리즘

- 수행결과



```

Console  PyUnit
<terminated> floyd_warshall.py
  0  1  2  3  4
  0  0  1  2  3
-1 -1  0  1  2
-2 -1  0  0  2
-3 -3 -2 -1  0
    
```

D	0	1	2	3	4
0	0	1	2	3	4
1	0	0	1	2	3
2	-1	-1	0	1	2
3	-2	-1	0	0	2
4	-3	-3	-2	-1	0

최단 경로 알고리즘 시간 복잡도

- Dijkstra: $O(n^2)$
 - 주 반복문을 n 번 반복
 - 내부 반복문을 $2n$ 번 반복
 - 모든 정점 쌍의 최단 경로를 구하려면 n 번 반복 $\rightarrow O(n^3)$
- Floyd-Warshall: $O(n^3)$
 - 모든 정점 쌍의 최단 경로 거리를 구함
 - 3중 반복문을 실행
 - Floyd의 알고리즘은 매우 간결한 반복 구문을 사용

Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- <https://wikidocs.net/195736>
- https://csacademy.com/app/graph_editor/
- <https://www.graphviz.org/>
- <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>